Getting started with the STM32Cube function pack for ST AIoT Craft application using IoT systems

Introduction

The function pack has been updated to Version 1.1.0 with several key improvements and structural changes:

- Code Refactoring and Stability ImprovementsMLC Interrupt Handling:
  - The MLC interrupt has been switched to latched mode for improved stability and better interrupt handling, replacing the previous pulsed mode.
- Telemetry Output:
  - Output telemetry now utilizes $MLC\_STATUS\_MAINPAGE$ to accurately identify the decision tree that generated the output.
- Code Cleanup:
  - The code has undergone cleanup, which includes the removal of useless pointers and redundant inclusions, and the use of extern variables instead of internal, redundant definitions.

Integration of AI-SSM Firmware v1.1.0. The new version integrates AI-SSM firmware v1.1.0, which introduces a significant structural change compared to v1.0.0. The firmware has shifted from a standalone, applicative structure to one that is generated using CubeMX. This change now requires the inclusion of standard ST X-CUBE packs (such as X-CUBE-BLMGR and X-CUBE-NFC), moving away from the previous, simpler internal code structure.

1 FP-SNS-STAIOTCFT software expansion for STM32Cube

1.1 Overview

The Function Pack includes two different firmwares:

- AI-INERTIAL
- AI-SSM (short for AI-SmartSensorMonitoring)

AI-INERTIAL

- **Complete firmware for AI-based inertial applications**: Develop applications with AI inference capabilities on MCU, MLC, and ISPU, utilizing sensors like ISM330IS and LSM6DSV16X.
- **Seamless integration with STM32CubeMX**: Easy project configuration and code generation for STEVAL-MKBOXPRO, STEVAL-STWINBX1, and STEVAL-STWINKT1B boards.
- **Comprehensive sensor support**: Ready-to-use projects for vibration monitoring and smart asset tracking, with seamless sensor data gathering and processing.
- **Multifunctional application**: Integrates AI inference, USB connectivity, and PnPL commands for switching between applications.
- **Robust software pack selection**: Requires X-CUBE-AI, X-CUBE-ISPU, and X-CUBE-MEMS1 for AI, sensor interfacing, and preprocessing support.

This software creates the following functionalities:

1. AI inference on MCU:
   - Direct neural network inference on the microcontroller.
   - Preprocesses inertial data from accelerometers.
   - Standard use-case for vibration monitoring.
2. AI inference on MLC:
   - Machine learning algorithm inference directly on the sensor.
   - Outputs classification results based on selected algorithms.
   - Applications include smart asset tracking and vibration monitoring.
3. AI inference on ISPU:
   - AI algorithm inference on the sensor's core.
   - Standard application for pedometer functionality.
4. PnPL commands via USB:
   - Switch between AI inference functionalities using Plug-&-Play-like commands.
   - Ensures easy interaction and control through USB connectivity.

This package is compatible with STM32CubeMX (V≥6.16.0) and requires additional software packs like X-CUBE-AI, X-CUBE-ISPU, and X-CUBE-MEMS1 for comprehensive support.

AI-SSM

AI Smart Sensor Monitoring (AI-SSM) firmware.

The AI Smart Sensor Monitoring (AI-SSM) firmware is a complete solution for power-saving AIoT applications on the STEVAL-MKBOXPRO board. It leverages the Machine Learning Core (MLC) of the smart sensor to classify motion patterns and transmit inference results wirelessly via Bluetooth Low Energy (BLE).

Key Features and Components:

- Power Saving Application:
  - The core mechanism relies on monitoring motion patterns directly in the sensor's MLC, allowing the host microcontroller (MCU) to remain in a low-power sleep mode. The MCU is only interrupted and wakes up when the MLC detects an event.
- MLC-Based AI Inference:
  - Utilizes the sensor's MLC to perform real-time classification of motion patterns, such as smart asset tracking, directly on the sensor, minimizing MCU load.
- Flexible BLE Communication:

The firmware automatically switches between two modes:

- Connection Mode:
  - Used at boot or upon user trigger (button/NFC) to receive an MLC model from a client (mobile app or gateway). Indicated by a blue LED.
- Beacon Mode (Low Power):
  - Entered after model deployment, where the device operates in low power mode and broadcasts AI prediction results via BLE advertisement frames (beacons). Indicated by a brief orange LED flash upon event detection.
- Event-Driven Architecture:

Processing and transmission are triggered by:

- MLC sensor interrupts upon pattern detection.
- A 1-minute keepalive timer to send the last prediction value if no MLC event occurs.
- User Button 1 or NFC sensor interaction to switch back to Connection Mode.
- Seamless Integration: Designed to work with the STEVAL-MKBOXPRO board and can be managed via the ST AIoT Craft Mobile App or a web-based IoT Gateway for centralized control and monitoring.

Core Functionalities

AI Inference on MLC (Core Function):

- Performs machine learning inference directly on the sensor's MLC.
- Detects and classifies pre-loaded motion patterns, enabling applications like smart asset tracking.

Smart Power Management:

- MCU enters low-power sleep modes (Beacon Mode) to maximize battery life.
- Wakes only upon receiving MLC interrupts or timer events to process and transmit data.

BLE PnPL-like Model Deployment:

- Uses Connection Mode to allow a client device to wirelessly deploy a new MLC model file using the BlueST protocol v2.

BLE Beacon Transmission:

- Uses Beacon Mode to broadcast real-time AI prediction results and device status using the BlueST protocol v3, enabling scalable, low-power monitoring.

1.2 Architecture

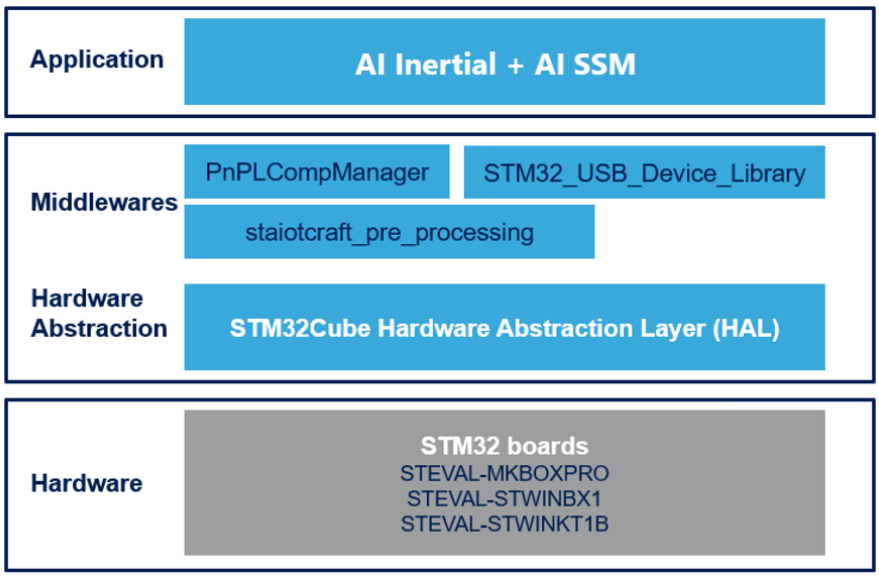
The FP-SNS-STAIOTCFT software package is built on the STM32CubeHAL, the hardware abstraction layer for STM32 microcontrollers. This package extends STM32Cube by providing a board support package (BSP) for the supported development boards and middleware components for AI inference and sensor data processing.

The software architecture includes the following layers:

- **STM32Cube HAL layer**: This layer offers a simple, generic, multi-instance set of application programming interfaces (APIs) to interact with the upper application, library, and stack layers. It is built around a generic architecture, allowing middleware layers to implement functions without requiring specific hardware configurations for a given microcontroller unit (MCU). This structure enhances library code reusability and ensures easy portability across different devices.
- **Board support package (BSP) layer**: The BSP layer supports all peripherals on the STEVAL-MKBOXPRO, STEVAL-STWINBX1, and STEVAL-STWINKT1B boards, except the MCU. This limited set of APIs provides a programming interface for board-specific peripherals like sensors, LEDs, and user buttons, facilitating the identification of specific board versions.
- **Middleware layer**: This layer provides advanced AI and sensor processing libraries. Key components include:
  - **Preprocessing library**: For data preprocessing before AI inference.
  - **AI libraries**: For running neural network inference on the MCU, MLC, and ISPU.
  - **PnPl commands**: For switching between AI functionalities via USB connectivity.

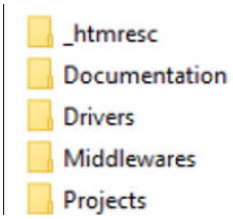
The implementation leverages low power consumption strategies, making it suitable for field applications. It ensures compliance with AI inference requirements and provides a robust framework for developing AI-based inertial applications.

Figure 1. **FP-SNS-STAIOTCFT software architecture**



1.3 Folder structure

Figure 2. **FP-SNS-STAIOTCFT package folder structure**



The following folders are included in the software package:

- Documentation: contains a file generated from the source code which details the software components and APIs, plus the getting started document.
- Drivers: contains the HAL drivers and the board-specific drivers for each supported board or hardware platform, including the on-board components and the CMSIS vendor-independent hardware abstraction layer for Arm Cortex-M processor series.
- Middlewares: contains libraries and protocols for PnPL, preprocessing and USB serial communication.
- Projects: contains two sample applications:
  - "I Inertial": which is the inference application providing 3 AI targets, Neural network inference on MCU and decision tree algorithms on MLC and on ISPU. The folder also contains the libraries generated by the X-CUBE-AI package for the Cortex M33 and M4 as a backup.
  - "AI SSM": the firmware uses the Machine Learning Core (MLC) on the SensorTile.Box Pro board to classify motion patterns and send inference results over BLE.

1.4 APIs

Detailed technical information with full user API function and parameter descriptions are in a compiled HTML file in the “Documentation” folder

1.5 Sample application description

A sample application is provided in the Projects folder for the STEVAL-MKBOXPRO, STIWNBX1 and STWINKT1B. Ready-to-build projects are available for multiple IDEs.

You can set up a terminal window for the appropriate UART communication port to control the initialization phase.

1.6 STEVAL-MKBOXPRO

The STEVAL-MKBOXPRO development board is a versatile platform designed for rapid prototyping and development of advanced sensor-based applications. This board is equipped with a range of inertial sensors, including the LSM6DSV16X, which allows for precise motion tracking and environmental sensing. The firmware architecture is structured to provide a seamless integration of various functional layers, including application, middleware, sensor drivers, and hardware abstraction.

At the application level, the firmware facilitates user interaction through a comprehensive set of commands and functions. Users can communicate with the board via USB using PnPL messages, enabling real-time data exchange and control. The application layer also supports AI inference on the MCU using the X-CUBE-AI pack, allowing for the implementation of machine learning models directly on the device.

To interact with the firmware, users must first flash the firmware onto the board and perform a reboot. Once the board is operational, a terminal application like Tera Term can be used to send commands and view logs. The firmware supports various commands to start and stop AI inference, load configuration files, and retrieve sensor data. For example, users can initiate neural network inference with the “start” command and halt it with the “stop” command.

The firmware for STEVAL-MKBOXPRO is designed to be highly configurable, with support for loading model configuration files to customize the behavior of the machine learning core (MLC) and other sensor functionalities. This flexibility renders the board suitable for a wide range of applications, from smart asset tracking to vibration monitoring.

Overall, the STEVAL-MKBOXPRO provides a robust and user-friendly environment for developers to explore and implement cutting-edge sensor-based applications, leveraging the power of STMicroelectronics' advanced sensor technology and AI capabilities.

1.7 STWINBX1 and STWINKT1B

Other two development boards are supported. They support the same functionalities at an applicative level but leveraging different inertial sensors, like the ISM330DHCX or the IIS2ICLX.

1.8 AI inertial

Introduction to the firmware

The firmware structure can be subdivided into different conceptual levels, each serving a distinct purpose:

- **Application layer**: This is the topmost layer and is common across all development boards. It is crucial for allowing user interaction with the other firmware levels. It wraps commands and general functions from the lower levels and provides the capability to communicate via USB using PnPL messages. It also adopts functions to interact with inertial sensors using the X-CUBE-MEMS1 pack and X-CUBE-ISPU (excluding the STEVAL-STWINKT1B board, which does not provide ISPU support). Additionally, it runs AI inference on the MCU using the X-CUBE-AI pack.
- **Middlewares and libraries**: This layer includes various libraries that provide specialized functionalities required for sensor interaction and AI inference. Key libraries include AI runtime lib, CMSIS DSP lib, preprocessing, USB communication and PnPL.
- **Sensor drivers**: This layer handles the interaction with various sensors. Depending on the development board, different sensors may be supported, with corresponding drivers ensuring proper communication and data acquisition.
- **Development boards**: This layer represents the specific features and capabilities of different development boards such as STEVAL-MKBOXPRO, STWINBX1, and STWINKT1B. Each board has unique characteristics that influence the firmware's behavior.

Application layer functionality

User interaction

The application layer facilitates user interaction by wrapping commands and functions from the lower firmware levels. It allows communication via USB using PnPL messages, enabling users to send commands and receive responses.

Sensor interaction

The application layer includes functions to interact with inertial sensors, leveraging the X-CUBE-MEMS1 pack and X-CUBE-ISPU (excluding the STEVAL-STWINKT1B board). These functions enable sensor data acquisition and processing.

Command wrapping

Commands and general functions are encapsulated within the application layer, providing a simplified interface for user interaction. This abstraction ensures that users can easily communicate with the firmware without needing to understand the underlying complexities.

1.9 Interacting with the firmware

1.9.1 Initial setup and rebooting

Flashing the firmware

To begin interacting with the firmware, it must be flashed onto the chosen target board (e.g., MKBOXPRO, STWINBX1, or STWINKT1B). The flashing process involves loading the firmware binary onto the board using appropriate tools and software.

Rebooting the system

After flashing the firmware, a reboot is necessary to establish communication. This can be done by either turning the board off and on again or unplugging and plugging the USB cable. Once the board has rebooted, users can proceed to interact with the firmware.

1.9.2 Terminal interaction

Using Tera Term

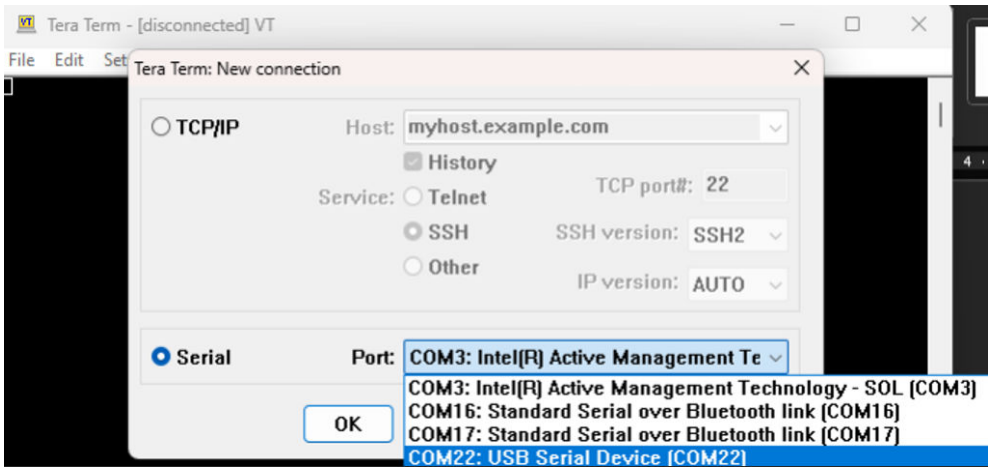
Tera Term is a terminal application that allows users to view logs and interact with the firmware. To use Tera Term:

Step 1. Open the application.

Step 2. Connect to the appropriate COM port corresponding to the development board.

Step 3. Configure the terminal settings to enable communication.

Figure 3. Terminal settings



Configuration settings

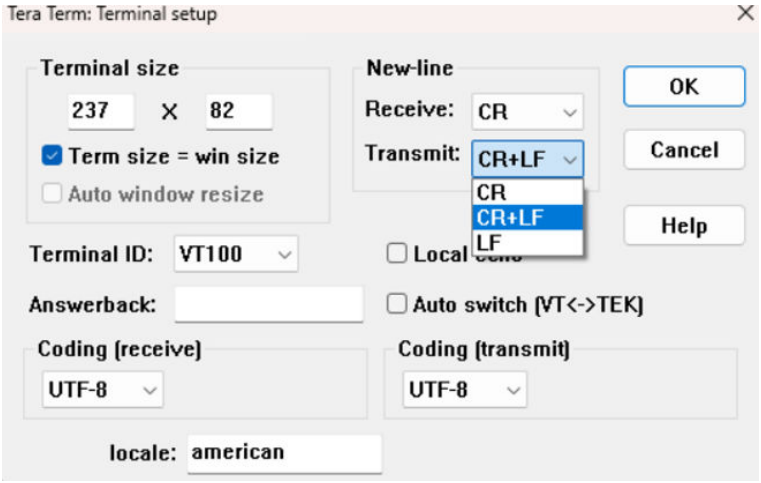
To send commands to the firmware and trigger applications, the CR+LF option must be enabled in Tera Term:

Step 4. Go to Setup -> Terminal -> Transmit.

Step 5. Enable the CR+LF option.

Step 6. Ensure that a new-line character is appended after every command.

Figure 4. Tera Term setup



Help messages

Upon rebooting the board and opening Tera Term, the first message displayed will be a help message. This message provides a set of possible commands depending on the chosen application and the sensors present on the board. For example, the STEVAL-MKBOXPRO board will reference the LSM6DSV16X inertial sensor.

1.10 Executing commands

Starting and stopping inference

Users can start or stop neural network inference on the MCU using commands such as "start_inference" and "stop_inference". These commands initiate or halt the inference process, allowing users to control the execution.

Classification outputs

The firmware outputs classification results for example applications like smart asset tracking. The classification labels include:

- **0** – Stationary upright (board on the table with the ST logo facing up)
- **4** – Stationary not upright (board on the table facing down)
- **8** – Moving (board lightly moving)
- **12** – Shaken (board shaken)

Changing inference applications

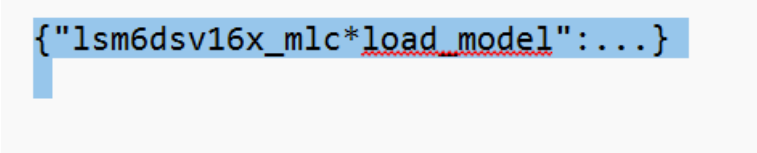
To change the inference application running on the MLC, users can trigger the "load_model" command. This command requires the PnPL message fields to be correctly formatted:

- **filename**: Name of the file.
- **size**: Size of the content.
- **content**: Text containing the compressed model.

Note: the compressed model is a sequence of numbers in hexadecimal format. For example:

```
{
    "lsm6dsv16x_mlc*load_model": {
        "arguments": {
            "filename": "model_name",
            "size": 1234,
            "content": "1000F021...5102"
        }
    }
}
```

Figure 5. **load_model command**



1.11 Advanced firmware operations

1.11.1 Customizing inference applications

PnPL message structure

The PnPL message structure includes several fields that must be correctly formatted to execute commands:

- filename: Specifies the name of the file to be loaded.
- size: Indicates the size of the content in characters.
- content: Contains the model text.

Example - 6D Position monitoring:

```
{
    "lsm6dsv16x_mlc*load_model":{
        "arguments":{
            "filename":"6dposition_model",
            "size":506,
            "content":"10001100018004000500174002110 8EA097809030982090309000900090A0901091202
1108FA095C0903098E0903099A09030231085C093F090009000984090009000900098809000900900098C0900090
0090009040900090009000908090009000900090C09000900091F09000231088E090009000900090009000900090
0900090009000900010001801740023108 9A09CD0934090509800 9CD0934090309810 9CD0934095609E509CD09340
90009A209CD0934093409E409CD0934090009A209CD0934090009A109CD0934091209E3018017000400051002010 1
005E0201800D016015450201001500170010041100"
        }
    }
}
```

Note: Many more commands (targeting also the MCU) and properties can be found in the Documentation section of the FP-SNS-STAIOTCFT package.

The commands and properties of the firmware are related to the FP-SNS-STAIOTCFT v1.0.0.

To create the load_model command, follow these steps:

1. Export the Model File:
   Use the ST AIoT Craft Portal or your preferred tool to generate and export the MLC model file (typically with a .json extension).

2. Exported JSON example


```
{
    "json_format": {
        "type": "reg_config",
        "version": "2.0"
    },
    "application": {
        "name": "MLC Tool",
        "version": "2.3.0"
    },
    "date": "2025-07-08 10:16:36",
    "description": "Generated sensor configuration for MLC core",
    "sensors": [
        {
            "name": [
                "LSM6DSV16X"
            ],
            "configuration": [
                {
                    "type": "write",
                    "address": "0x10",
                    "data": "0x00"
                },
                {...
                },
                {
                    "type": "write",
                    "address": "0x17",
                    "data": "0x01"
                }
            ],
            "outputs": [
                {
                    "name": "Categorical output",
                    "core": "MLC",
                    "type": "uint8_t",
                    "len": "1",
                    "reg_addr": "0x70",
                    "reg_name": "MLC1_SRC",
                    "results": [
                        {
                            "code": "0x00",
                            "label": "none"
                        },
                        {
                            "code": "0x04",
                            "label": "forward"
                        },
                        {
                            "code": "0x08",
                            "label": "left"
                        },
                        {
                            "code": "0x0C",
                            "label": "right"
                        }
                    ]
                }
            ],
            "mlc_identifiers": [
                {
                    "fifo_tag": "0x1C",
                    "id": "0x03C6",
                    "label": "F1_ENERGY_GYR_Z"
                },
                {...
                },
                {
                    "fifo_tag": "0x1C",
```

```
                    "id": "0x03E2",
                    "label": "F15_MINIMUM_ACC_Y"
                }
            ]
        }
    ],
    "staiotcraft_info": {
        "project_name": "joystick-project2",
        "project_id": "0f44196d-9b32-4fef-98bb-2d6d614d66e7",
        "model_name": "joystick-model2",
        "model_id": "6aec31cb-6616-498a-9127-0a4ac9430d07"
    }
}
```

Compress, Format the Content and Insert it into a Json Payload: This process can be automated using a script. For example, here is an AWK command that extracts the required string from a JSON file:
AWK script to extract <MLCx_SRC> and address/data pairs and create a load_model command Save the following script as a file named extract_mlc.awk:

```
# Save as extract_mlc.awk
BEGIN { header=""; out=""; sensor=""; model=""; }
# Find reg_name
/"reg_name"[ \t]*:[ \t]*"MLC[0-9]+_SRC"/ {
    match($0, /"reg_name"[ \t]*:[ \t]*"(MLC[0-9]+_SRC)"/, arr)
    header = "<" arr[1] ">"
}
# Find address/data pairs
/"address"[ \t]*:[ \t]*"0x([0-9A-Fa-f]{2})"/ {
    match($0, /"address"[ \t]*:[ \t]*"0x([0-9A-Fa-f]{2})"/, a)
    getline
    if ($0 ~ /"data"[ \t]*:[ \t]*"0x([0-9A-Fa-f]{2})"/) {
        match($0, /"data"[ \t]*:[ \t]*"0x([0-9A-Fa-f]{2})"/, d)
        out = out a[1] d[1] "\\n"
    }
}
# Find model_name
/"model_name"[ \t]*:[ \t]*"/ {
    match($0, /"model_name"[ \t]*:[ \t]*"([^"]+)"/, m)
    model = m[1]
}
# Find sensor name (fix regex for [)
/"name"[ \t]*:[ \t]*\[/ {
    getline
    if ($0 ~ /"([^"]+)"/) {
        match($0, /"([^"]+)"/, s)
        sensor = tolower(s[1])
    }
}
END {
    # Remove last \n
    sub(/\\n$/, "", out)
    # Compose content string
    content = header "\\n" out
    # Count characters (length in awk is bytes, which is fine for this ASCII content)
    size = length(content)
    # Print compact JSON (no spaces/newlines except inside content), add a real newline at
 the end
    printf "{\"%s_mlc*load_model\":{\"arguments\":{\"filename\":\"%s\",\"size\":%d,\"conte
nt\":\"%s\"}}}\n", sensor, model, size, content
}
```

3. Verify you have awk installed in your pc.
   Then position in a folder where you have:
   – the awk script
   – the json file

   and run the command:
   – awk -f extract_mlc.awk your_model.json

   The awk script is able to take in input the json downloaded from the ST AIoT Craft web portal and extract the load_model command needed for your application.
4. Then copy and paste it on the Teraterm command prompt

1.12 Troubleshooting and optimization

Common issues

Common issues encountered during firmware interaction include communication errors and incorrect command formatting. Identifying and resolving these issues is crucial for smooth operation.

1.13 Application AI SSM

ai_ssm

The AI Smart Sensor Monitoring (ai_ssm) firmware uses the Machine Learning Core (MLC) on the SensorTile.Box Pro board to classify motion patterns and send inference results over BLE.

This firmware is provided as an example of power saving application: since motion patterns are monitored in the smart sensor, the microcontroller is put in low power mode. Whenever an event is detected, the MCU is interrupted to broadcast it using BLE advertisement frames.

At boot time, the firmware places the board in BLE connectable mode, ready to receive an MLC model from a client device such as a gateway or mobile app. Upon client disconnection, the board reboots and enters a low-power mode, awaiting prediction events from the smart sensor. When an event occurs (or after a 1-minute keepalive timeout), the firmware reads the prediction data and transmits the results via BLE in beacon mode.

Get to know AI Smart Sensor Monitoring Firmware

Before starting, ensure you have the following:

Hardware:

- **STEVAL-MKBOXPRO** development board.

Software:

- **Firmware binary** retrievable from "*My IoT Systems*" section of ST AIoT Craft
  Access My IoT Systems
- **ST AIoT Craft** mobile application

GooglePlay:(https://play.google.com/store/apps/details?id=com.st.staiotcraft&hl=it&pli=1)

App Store:(https://apps.apple.com/af/app/st-aiot-craft/id6736846158**)**

Tip: *The blue LED serves as a visual confirmation that the device is ready to communicate. Ensure your mobile device's Bluetooth is enabled for seamless connection.*

Note: *When the board is sending beacons, it operates in **low power mode** to conserve energy. During this state, an **orange LED** briefly lights up for about one second **ONLY when an interrupt is detected**.The device can be set in connectable mode if a user:*

- ***Press User Button 1***
- ***Trigger the NFC sensor***

*In both cases, the board will **reactivate in connectable mode**, allowing a new model deployment or interaction.*

How the ai_ssm Firmware Works

Firmware Overview

At its core, the firmware orchestrates the following:

- **Sensor Initialization & Configuration:** Supports Machine Learning Core (MLC).

- **BLE Communication Modes:** Switches between **Beacon Mode** (broadcasting sensor inferences) and **Connection Mode** (interactive BLE communication).
- **Power Management:** Enters low-power sleep modes to maximize battery life.
- **NFC Interaction:** Detects NFC field changes to influence BLE behavior.
- **Event-Driven Processing:** Reacts to sensor interrupts and timers to process and transmit AI predictions.

Step-by-Step Operation

1. **Initialization**
   - **Initialize MLC:**
     - *MLC Sensor:* Loads AI models from flash or default configurations.
   - **Initialize NFC Tag:** Sets up NFC for secure, event-driven interactions.
   - **Set BLE Mode:** Chooses between Beacon or Connection mode based on firmware status.
2. **BLE Modes Explained**

Table 1. BLE Modes Explained

Mode	Purpose	Key Features
Connection Mode	Interactive BLE communication with clients	- Enables BLE to load a model file
		- Uses BlueST protocol v2
		- Blue LED indicates active connection
Beacon Mode	Broadcast AI inference results periodically	- Sends BLE beacons with sensor predictions
		- Uses BlueST protocol v3
		- Uses timers and MLC interrupts to trigger events
		- Orange LED indicates beacon activity
		- User button or NFC detection switches to Connection mode

3. **Event-Driven Processing**
   - **Sensor Interrupts:** Events from MLC, setting flags for processing.
   - **User Button and NFC detection:** Allows manual switching from Beacon to Connection BLE mode.
   - **Timers:**
     - The **1-second timer** specifies the timespan duration of the interval during which BLE advertisements are sent in beacon mode.
     - The **1-minute timer** triggers an interrupt every minute that sends the last available prediction value in case of inactivity of the MLC sensor, effectively scheduling periodic beacon transmissions.
4. **Power Optimization**
   - **Low-Power Sleep Modes:**
     - Enters Beacon Mode to suspend peripherals, reducing power.
     - Wakes on interrupts to resume processing.
5. **BLE Beacon Transmission**
   - **Prepare Payload:** Packages AI prediction results with identifiers.
   - **Advertise Data:** Sends beacon packets containing inference data and device name read from flash.
   - **Visual Feedback:** Orange LED signals beacon transmission activity.
6. **Firmware Status & Mode Logic**
   - Firmware tracks its status as:
     - **Flashed:** Basic firmware loaded.
     - **Named:** Device name assigned.
     - **Ready:** AI model loaded and name assigned, ready for inference.
   - The firmware can find itself in one of the above statuses when rebooted after a disconnection.
   - BLE mode (Beacon or Connection) is selected automatically based on this status.

Summary

The ai_ssm firmware delivers:

- **Smart AI sensor monitoring** with real-time inference from the MLC.
- **Flexible BLE communication**, toggling between broadcasting and connection modes.
- **Robust power management** for extended battery life.
- **NFC-enabled interactions** for seamless user experience.
- **Event-driven architecture** ensuring responsive and efficient operation.

This makes it an ideal solution for AIoT applications requiring intelligent sensing, wireless communication, and energy efficiency.

Quick Start: Using AI SSM Firmware on STEVAL-MKBOXPRO (via Mobile App)

1. **Flashing the Firmware**
   - **Open the web portal** or use the **ST AIoT Craft Mobile App**.
   - **Flash the firmware binary** referenced above from the portal or app.

Note: Reboot the board after flashing to be sure the firmware is starting.

   - Once flashing is complete, a **blue LED will light up**, indicating the board is now in **connection mode** and ready for pairing.

2. **Testing and Connecting**

Via mobile app:

- **Scan the QR code** using the **ST AIoT Craft Mobile App**.
- **Connect to the device** from the app interface.
- **Monitor telemetry data** in real-time directly on your mobile device.
- View the **beacon messages** related to the deployed project straight from the app.

Via gateway:

- **Access the web portal** where your gateway is managed.
- **Start the deployment process** of your IoT system
- Once deployed, click **"Start Monitoring"** to begin observing live data streams.

For step-by-step instructions on this process, see: (https://staiotcraft.st.com/index.html#/wiki?id=3_2_iot_step_by_step_ble.md:~:text=the%20deployment%20details.-,Leaf%20device%20setup,-This%20step%20prepares)

Why Use the Gateway?

- **Centralized Control:** Easily manage your current devices with a single interface, with plans to support multiple leaf devices for expanded network management in the near future.
- **Robust Connectivity:** Enjoy stable and continuous data flow without depending on mobile device proximity, ensuring reliable operation even as your device ecosystem grows.
- **Enhanced Monitoring:** Access detailed analytics and logs directly from the web portal, with upcoming features designed to provide comprehensive insights across multiple connected leaf devices.

Pro Tip: Monitor device status indicators and logs within the portal to quickly troubleshoot or optimize your deployment. The gateway method is perfectly suited for scalable AI SSM applications, and future enhancements will enable seamless management of multiple leaf devices, streamlining large-scale deployments.

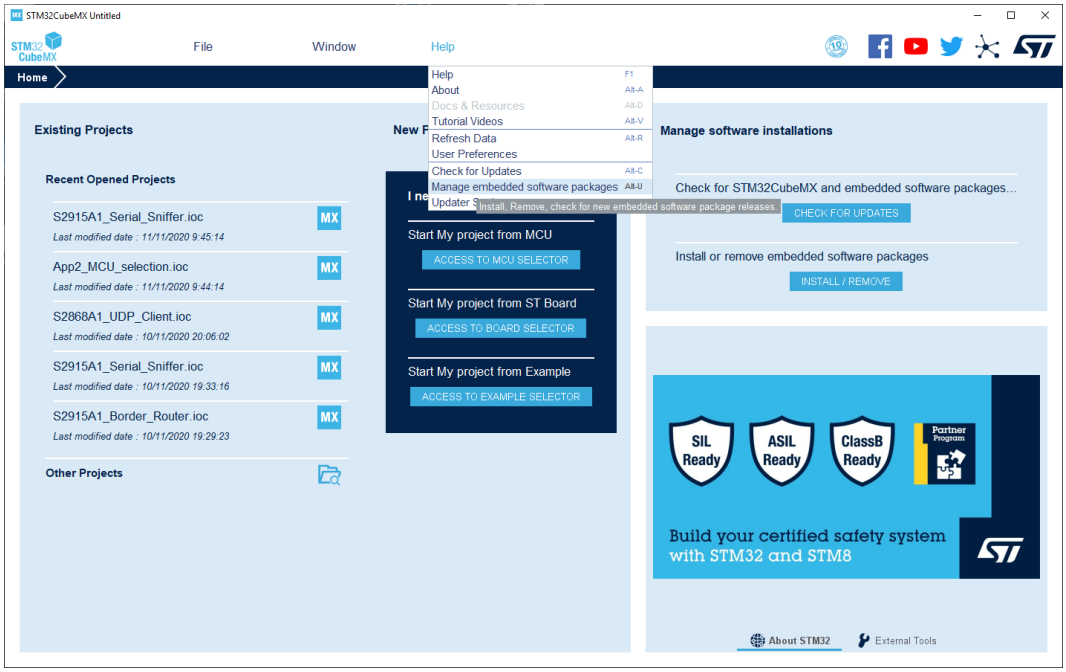
For BLE IoT systems step-by step-by step guide
https://staiotcraft.st.com/index.html#/wiki?id=3_2_iot_step_by_step_ble.md

1.14 Installing the FP-SNS-STAIOTCFT pack in STM32CubeMX

After downloading (from www.st.com), installing and launching the STM32CubeMX (V≥6.13.0), the FP-SNS-STAIOTCFT pack can be installed in few steps.

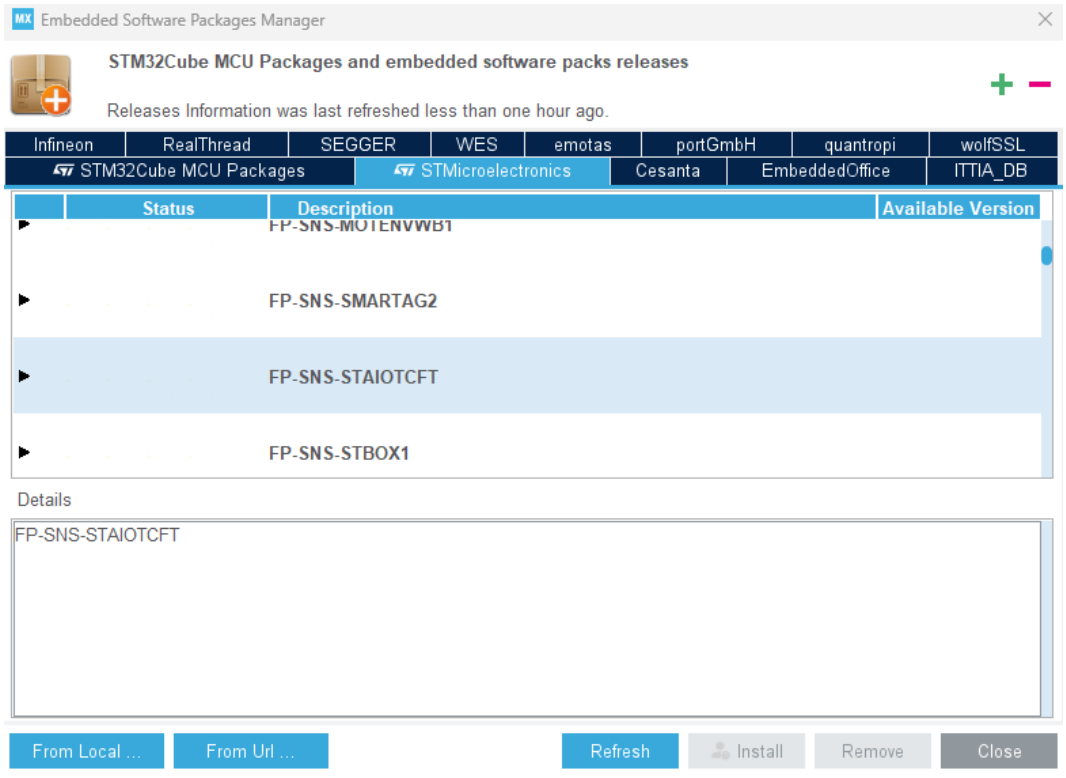
1. From the menu, select Help > Manage embedded software packages

Figure 6. Managing embedded software packs in STM32CubeMX



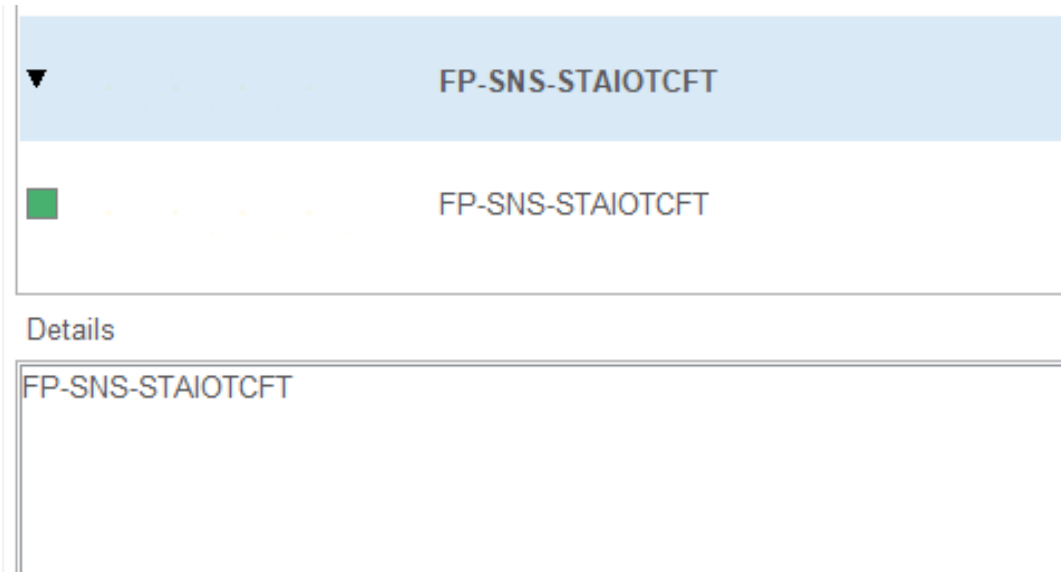
2. From the Embedded Software Packages Manager window, press the ‘Refresh’ button to get an updated list of the add-on packs. Go to the ‘STMicroelectronics’ tab to find the FP-SNS-STAIOTCFT pack.

Figure 7. Installing the FP-SNS-STAIOTCFT pack in STM32CubeMX



3. Select it checking the corresponding box and install it pressing the ‘Install’ button. Once the installation is completed, the corresponding box will become green, the ‘Close’ button can be pressed and the configuration of a new project can start.

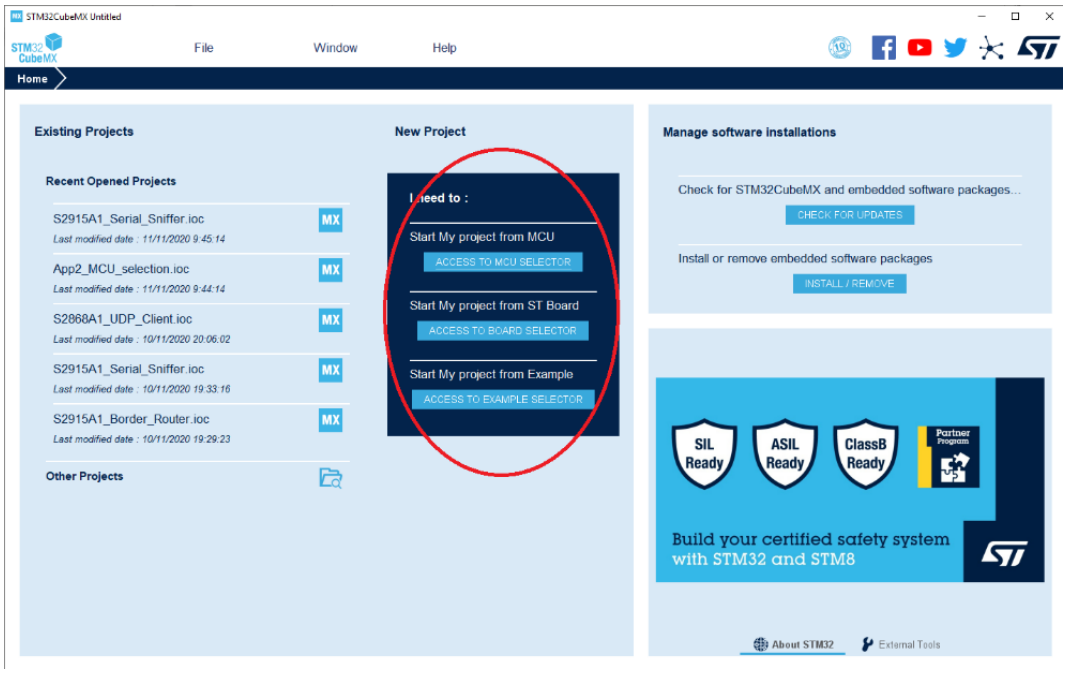
Figure 8. The FP-SNS-STAIOTCFT pack in STM32CubeMX



1.15 Starting a new project

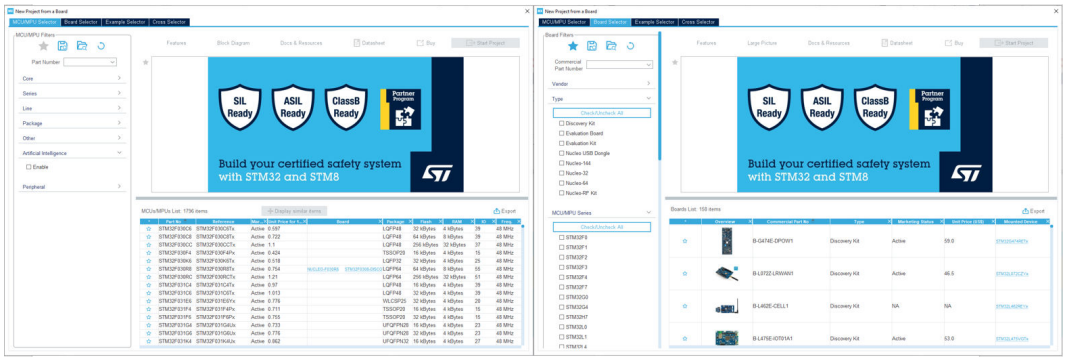
After launching the STM32CubeMX, you can choose if starting a New Project from the MCU Selector or from the Board Selector.

Figure 9. STM32CubeMX main page



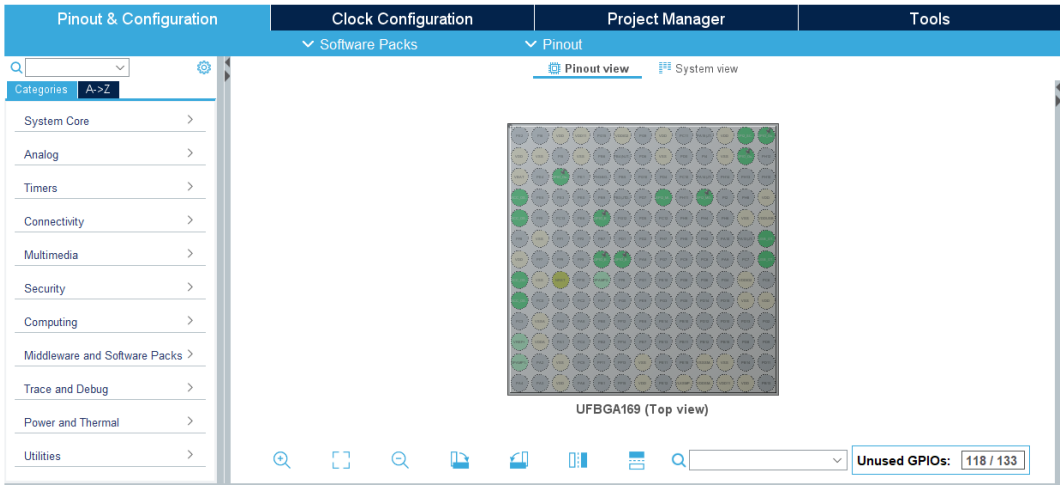
The **MCU/Board selector** window will pop up. From this window, the STM32 MCU or platform can be selected.

Figure 10. STM32CubeMX MCU/Board Selector windows



After selecting the MCU or the Board, the selected STM32 pinout will appear. From this window the user can set up the project, by adding one or more Additional Software and peripherals and configuring the clock.

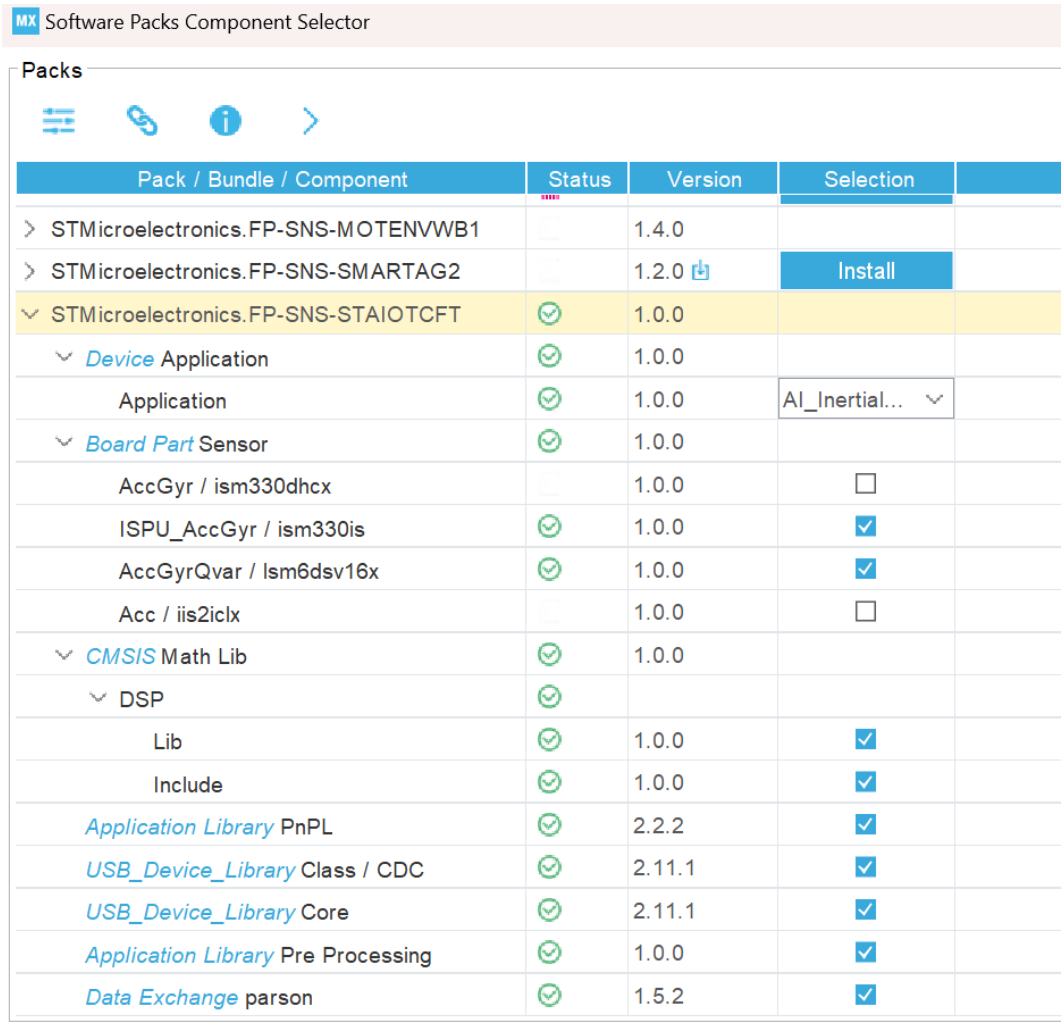
Figure 11. STM32CubeMX Pinout & Configuration window



To add the FP-SNS-STAIOTCFT additional software to the project, the “Software Packs” and then “Select Components” buttons must be clicked.

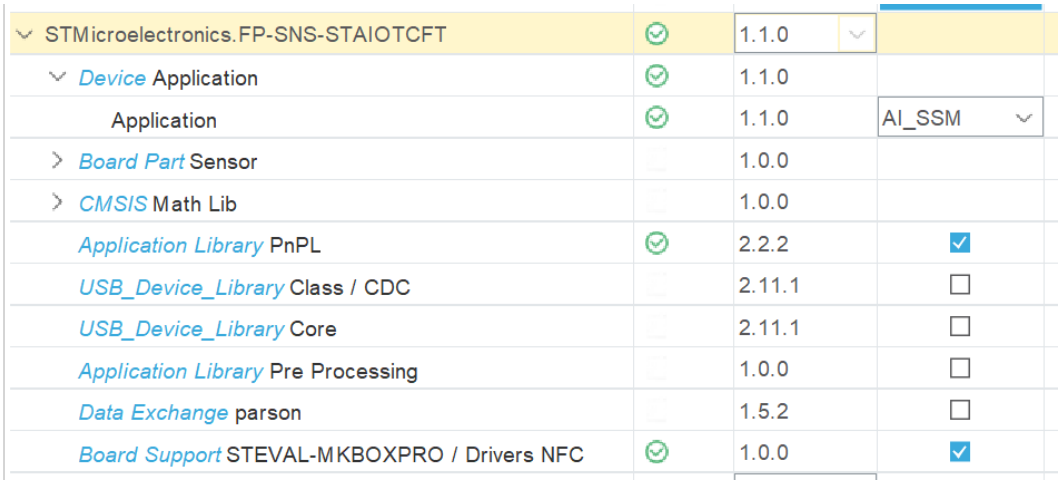
From the Software Pack Component Selector window, the user can either choose to generate, for the selected MCU/Board, one of the enclosed sample applications or a new project. In this latter case, the user must just implement the main application logic without bothering with the pinout and peripherals configuration code that will be automatically generated by STM32CubeMX.

Figure 12. STM32CubeMX Software Pack Component Selector window



To choose the **AI SSM** application follow the above steps except select “AI_SSM” from the drop down list menu and check the components boxes as illustrated below:

Figure 13. STM32CubeMX Software Pack Component Selector window AI_SSM



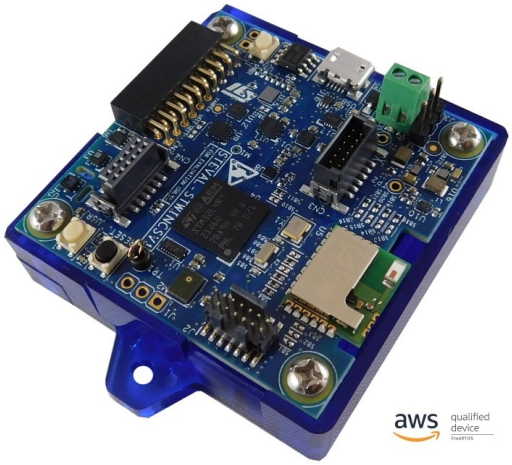
1.16 STM32 Configuration steps

Depending on the chosen development board, the FP-SNS-STAIOTCFT is addressed in different ways. However, from an application point of view all the boards have the same AI inference capabilities, using the MCU, MLC or ISPU, as mentioned in Section 1.17.

Figure 14. STEVAL-MKBOXPRO development board

Figure 15. STEVAL-STWINBX1 development board

Figure 16. STEVAL-STWINKT1B development board



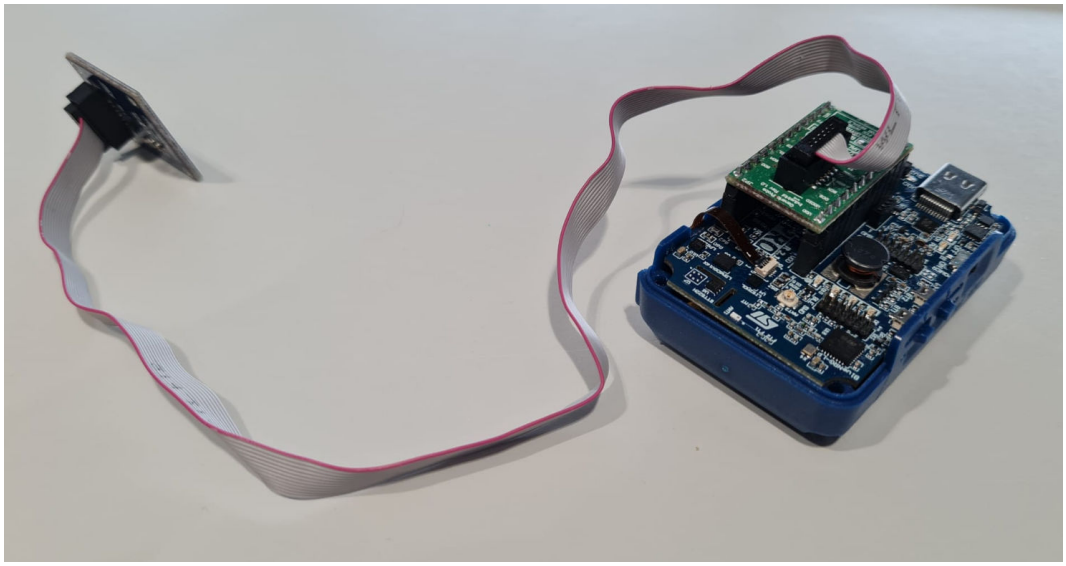
1.16.1 STEVAL - MKBOXPRO (ai_inertial and ai_ssm)

1.16.1.1 *ai_inertial application settings*

This section outlines how to configure STM32CubeMX with the STEVAL-MKBOXPRO development board starting directly from the STM32U585 micro-controller.

Before starting, if using the ISPU, it is important to properly set up the hardware to ensure that the ISM330IS sensor, which is embedded with the ISPU and connected via DIL24, is correctly connected. Please refer to the picture below for the correct setup.

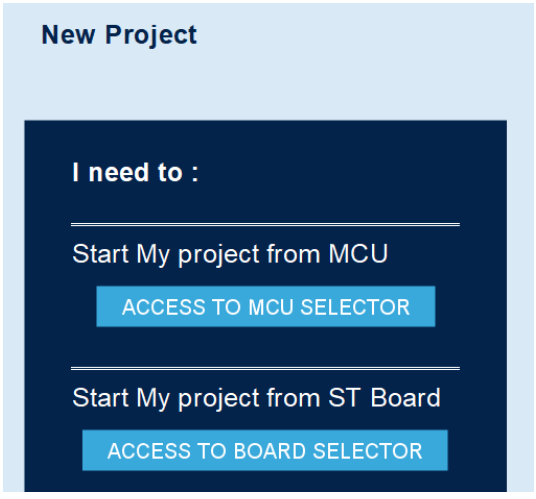
Figure 17. STEVAL-MKBOXPRO development board with ISM330IS sensor connected



It's possible to start a project directly from the MCU or choosing a specific target board.

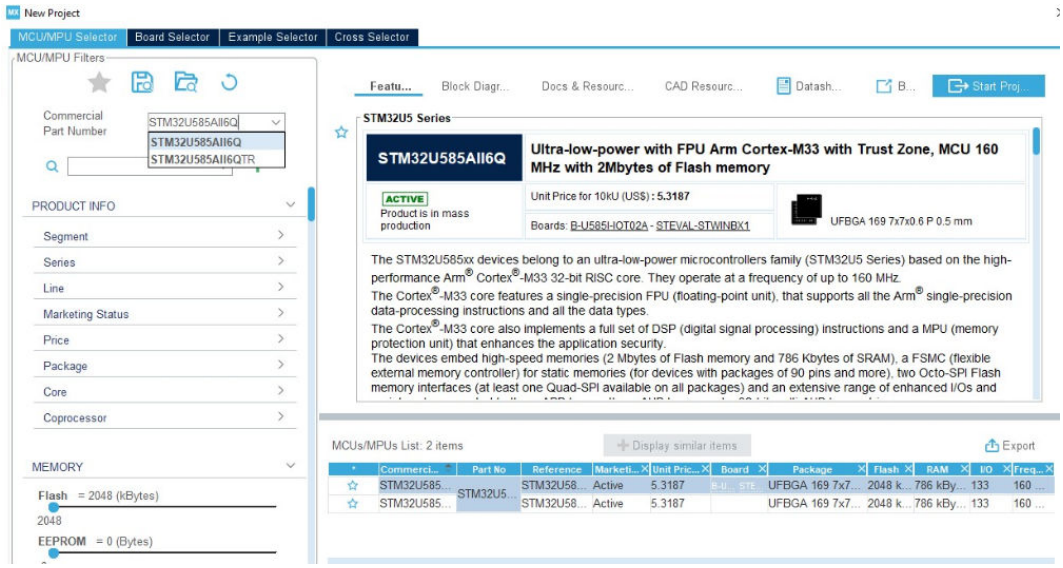
Let's start with the MCU, later we will address the access to the board selector

Figure 18. Starting point: MCU or BOARD selection



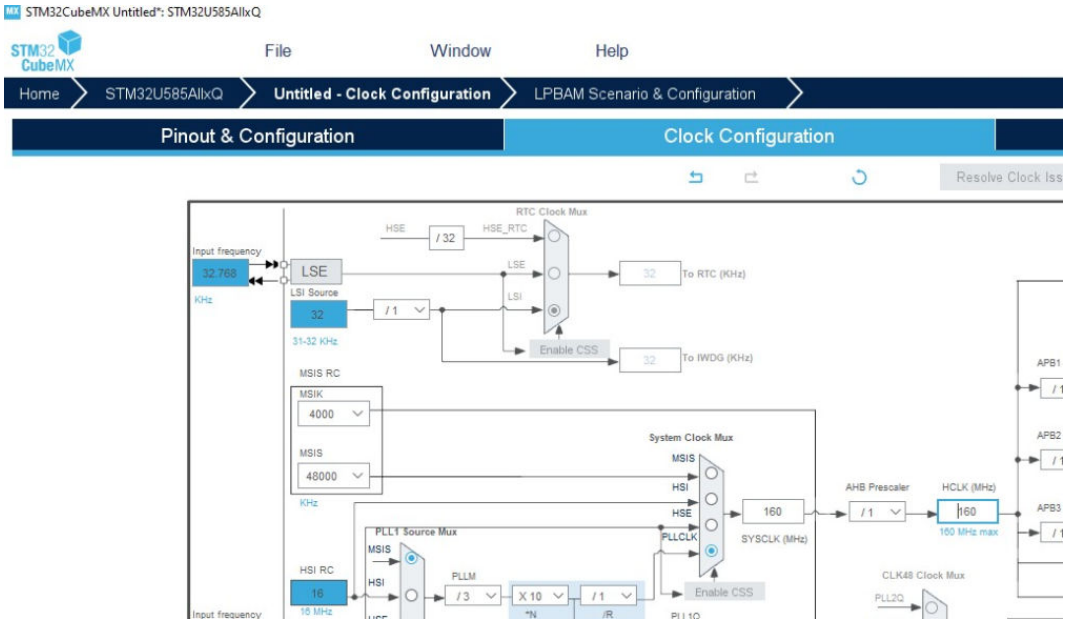
Create a new project for the U585 MCU.

Figure 19. Starting point: Micro-controller selection



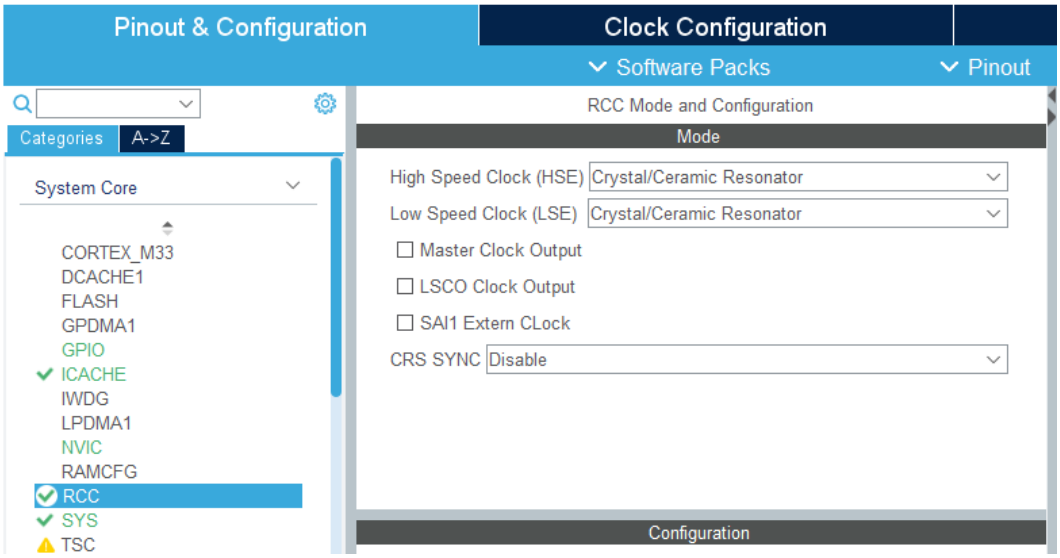
Set the clock **HCLK** to **160MHz** to utilize the maximum frequency.

Figure 20. Clock Frequency selection



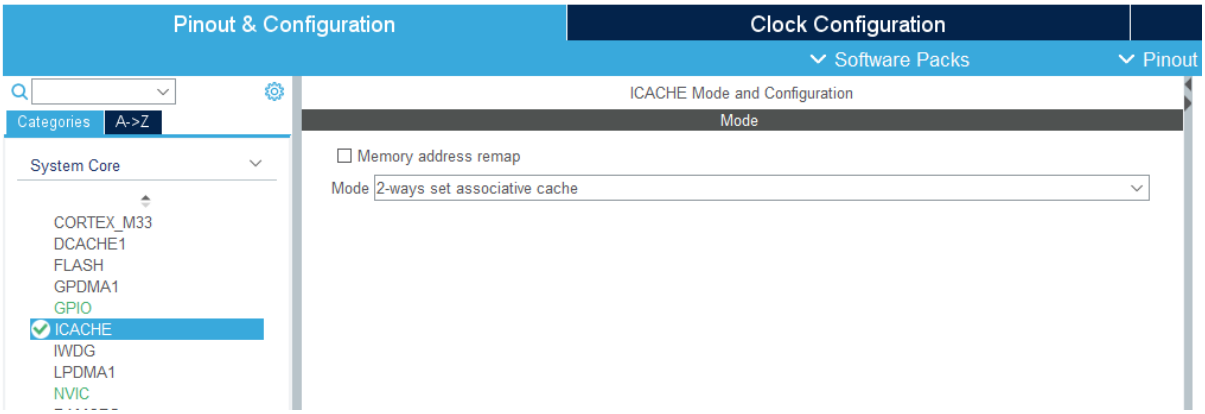
In the **RCC** section set the HSE and the LSE to **Crystal/Ceramic Resonator**.

Figure 21. **HSE and LSE clock selection**



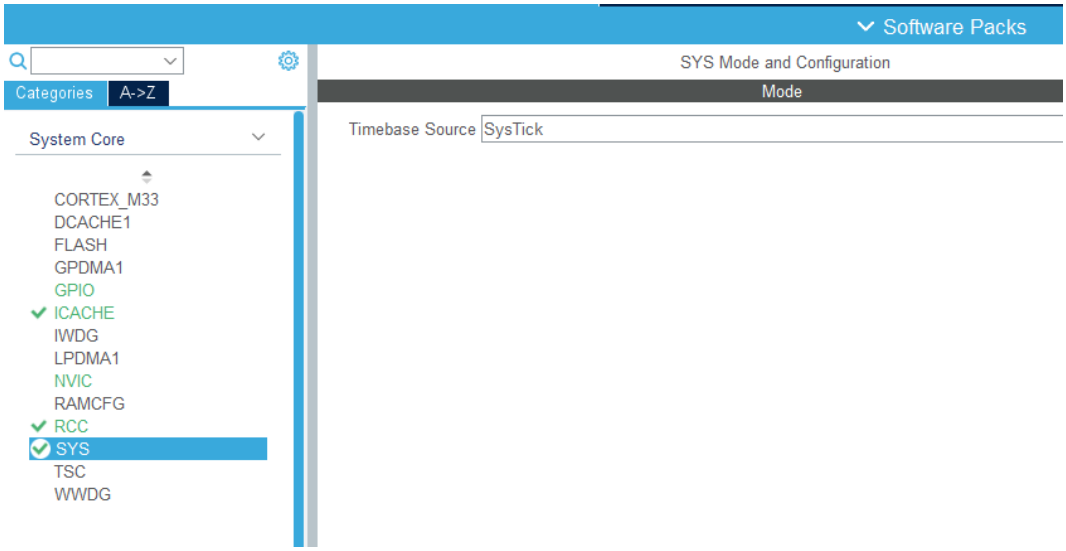
Set the Cache mode to 2-ways set associative cache.

Figure 22. **ICACHE options**



Set the SysTick.

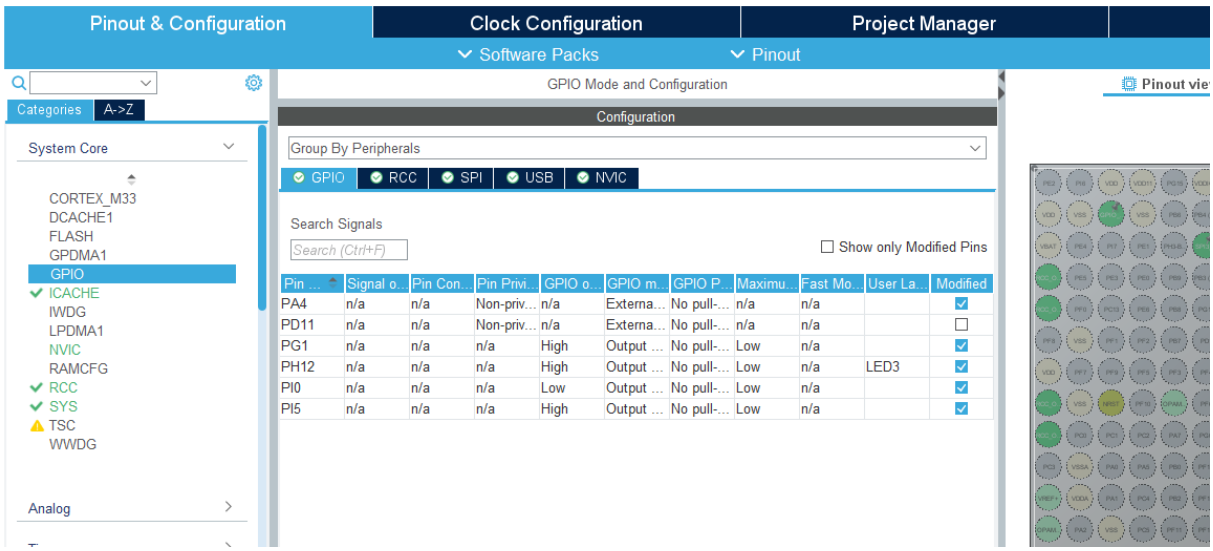
Figure 23. **SysTick options**



GPIO Configuration

Set the **PD11** pin to **GPIO_EXTI11**, **PA4** pin to **GPIO_EXTI4**, **PG1**, **PI0** and **PI5** pins to **GPIO_Output**. Then going to **NVIC** section enable the interrupt pin. The reason for the use of this pin is explained in the picture below. The **PD11** is linked to the **IMU_INT2** pin, which is excited when the FIFO of the LSM6DSV16X is full of samples. While the **PA4** is linked to the **IMU_INT1** pin, launching an interrupt directly from the MLC. For this reason is important to activate the interrupt pins of the sensor and link it to the MCU pin. **PH12** is used for the LED.

Figure 24. **GPIO selection**



Moreover set the **PI5** and the **PG1**, CS (chip select) of the LSM6DSV16X and the ISM330IS sensors respectively, to **High**, leaving **PI0** to **Low**. This is needed to disable the CS pins, they will be turned on once the SPI connection starts (which turns the pins from High to Low). While the **PI0** is used to activate the SPI, since by default this pin is connected to MCU_SEL, configured for I2C.

Timers

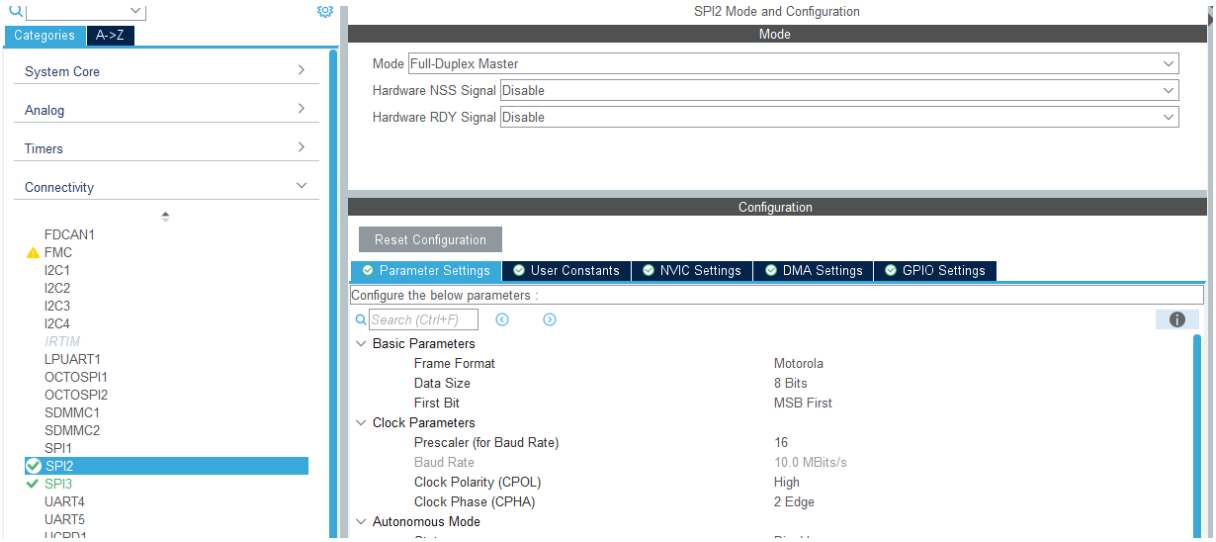
In this section select Tim1, Clock Source: Internal Clock, then in Parameter Settings set the Prescaler to 2400 and the Period to 65534. Then check the TIM1 Update Interrupt in the NVIC settings. This to obtain an interrupt every 1s, since the system clock frequency is 160MHz and to obtain almost the 1 Hz timer frequency the product between prescaler and period must be equal to the system frequency.

Connectivity Configuration

SPI2

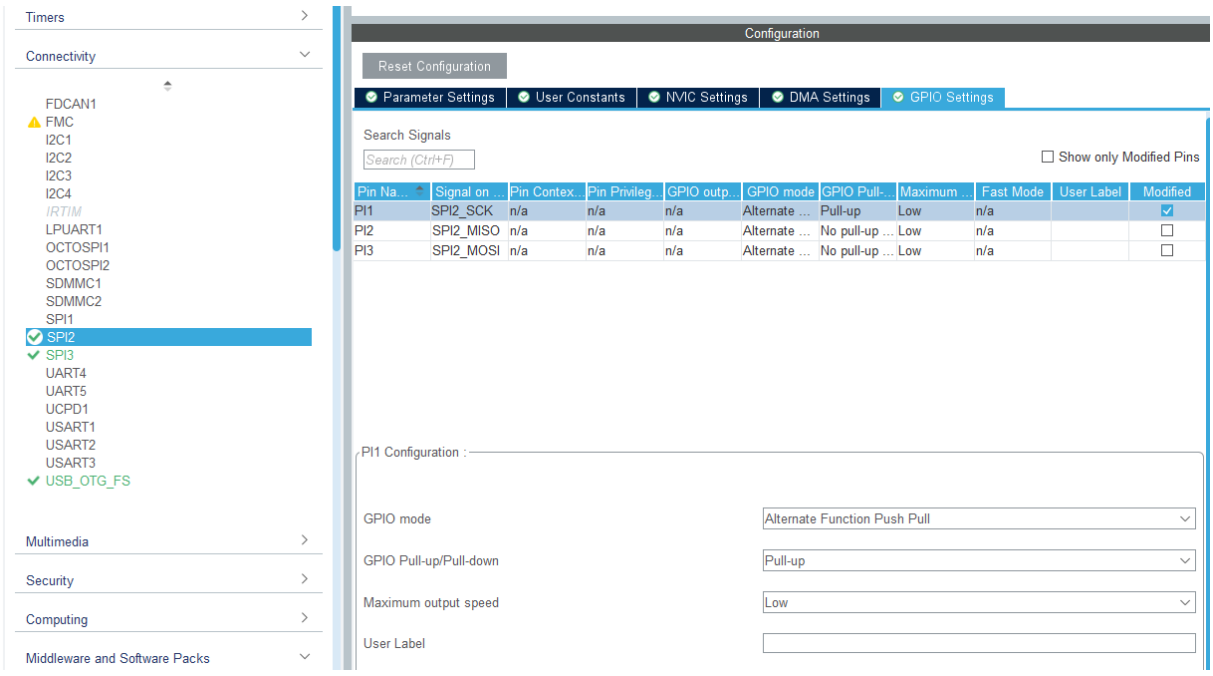
Under the **SPI2** section set the Mode to **Full Duplex Master**. Then in the **Parameter Settings**: set Data Size to 8 Bits, change the Prescaler such that the Baud Rate is lower or equal to 10.0 MBits/s, then choose CPOL High and CPHA 2 Edge. All these settings are standard and are needed to work with the **X-CUBE-MEMS1** pack, with the CUSTOM APIs, more information about this can be retrieved on the X-CUBE-MEMS1 manual (Chapter 7.3, page 64).

Figure 25. SPI2 selection - Parameter Settings



In the **GPIO Settings** of the SPI2, change the pins such that **PI2** is the MISO, **PI1** the SCK and the **PI3** the MOSI signals. This because the LSM6DSV16X sensor is connected to the SPI bus through these pins. Moreover **Pull up** the **SPI2_SCK** pin as reported in the manual discussed earlier.

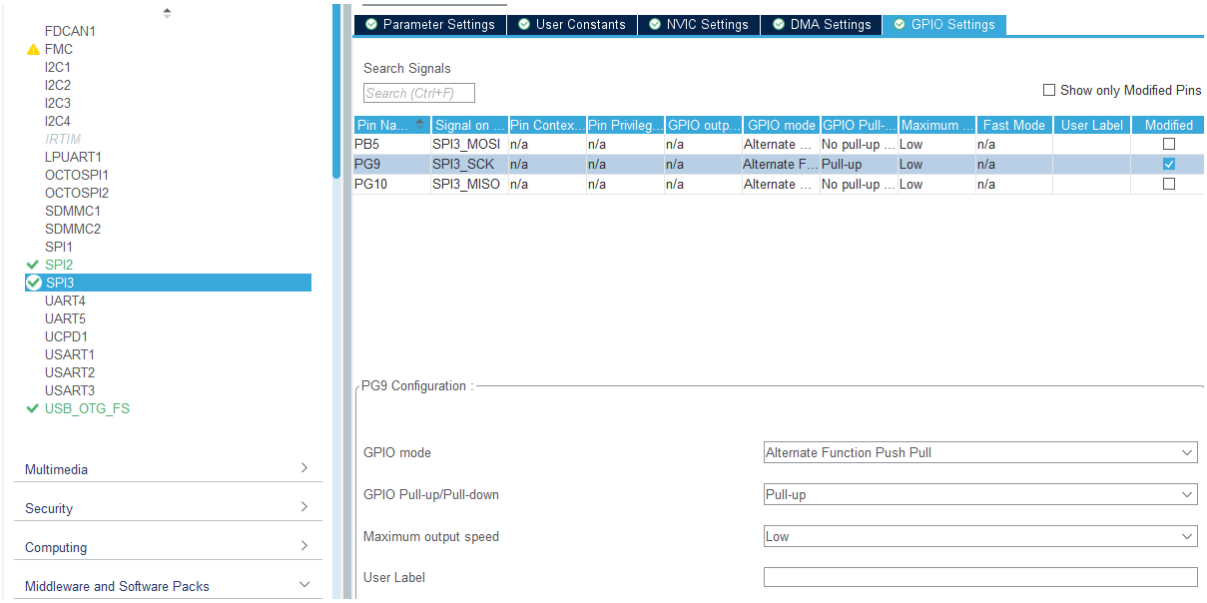
Figure 26. SPI2 selection - GPIO Settings



SPI3

The same configuration must apply for SPI3, considering the Parameter Settings. Regarding the GPIO Settings instead, consider **PG10** as the MISO, **PG9** as the SCK and the **PB5** as the MOSI signals. Remember also in this case to **Pull up** the SCK.

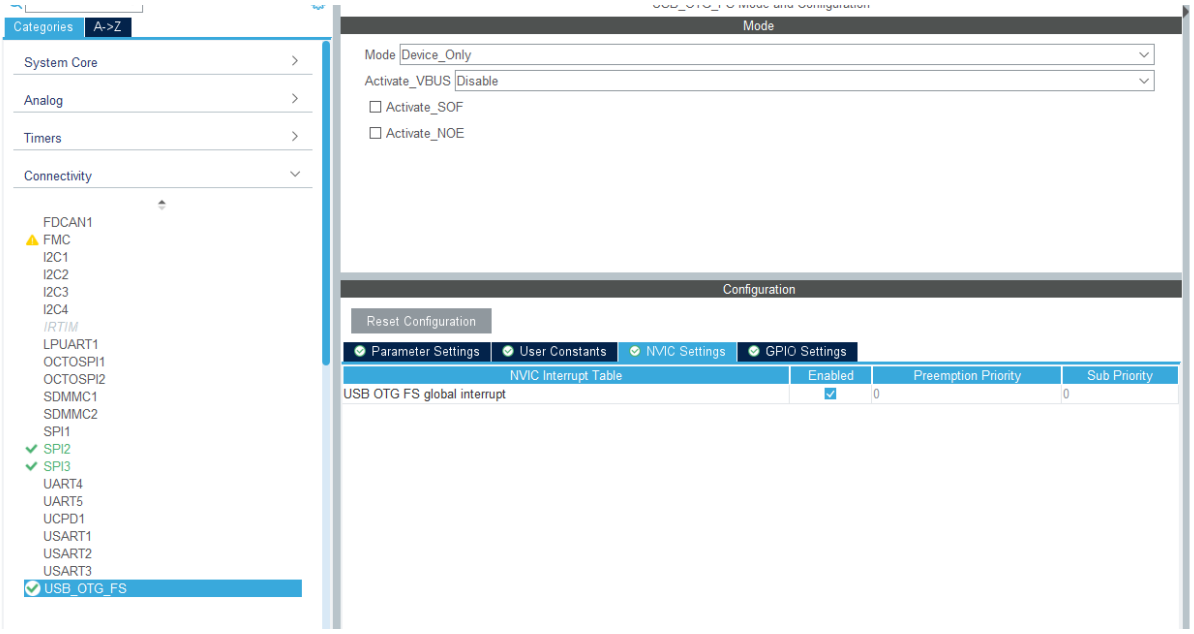
Figure 27. **SPI3 selection – GPIO Settings**



USB

Enable the **USB_OTG_FS** and select the **Device Only** Mode. Moreover enable the **global interrupt** under the NVIC Settings, the reason for this is to exploit the USB functionality at lower level. The interrupt is needed to receive and transmit data using the USB.

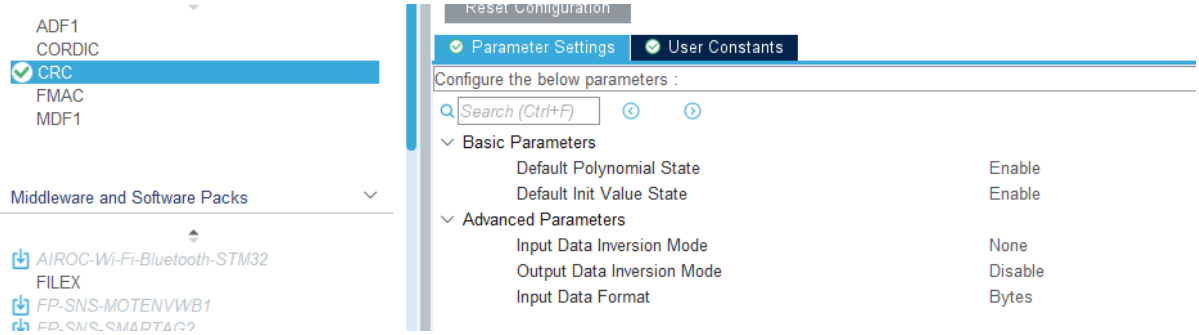
Figure 28. **USB selection - NVIC Settings**



CRC

The **CRC** must be activated to work with Neural Networks. Overall, using it can help ensuring the accuracy and reliability of trained models.

Figure 29. **CRC selection**



Software pack selection

In this part 4 software packs must be selected:

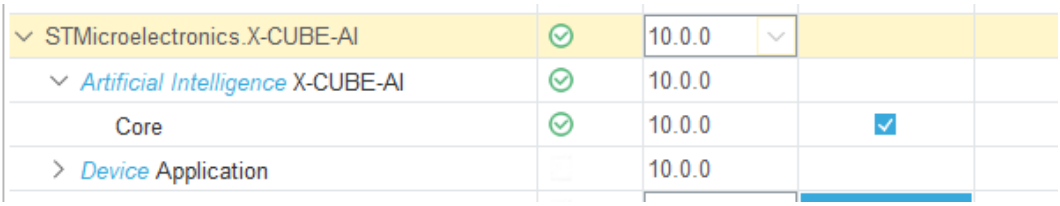
- **X-CUBE-AI**
- **X-CUBE-ISPU**
- **X-CUBE-MEMS1**
- **FP-SNS-STAIOTCFT**

The first three are needed to work with the Function Pack FP-SNS-STAIOTCFT.

Select “Software Packs” and then “Select Components”. From the “Software Packs Component Selector” window, the user has to select all the needed packs. In our case the ones above mentioned.

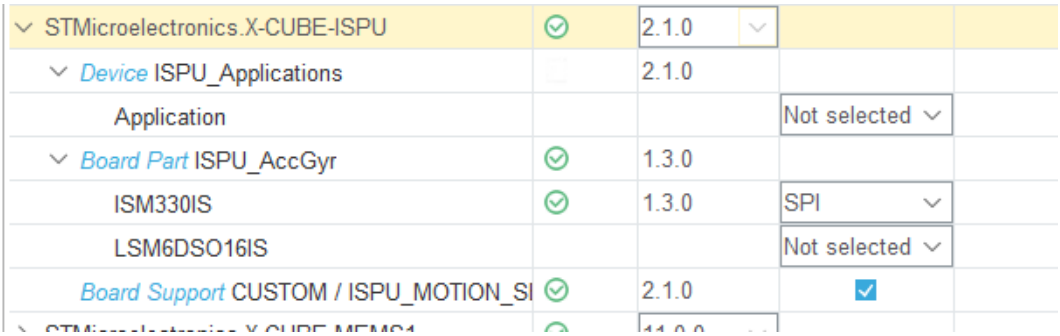
Under the X-CUBE-AI pack select only the **Core**.

Figure 30. **X-CUBE-AI pack selection**



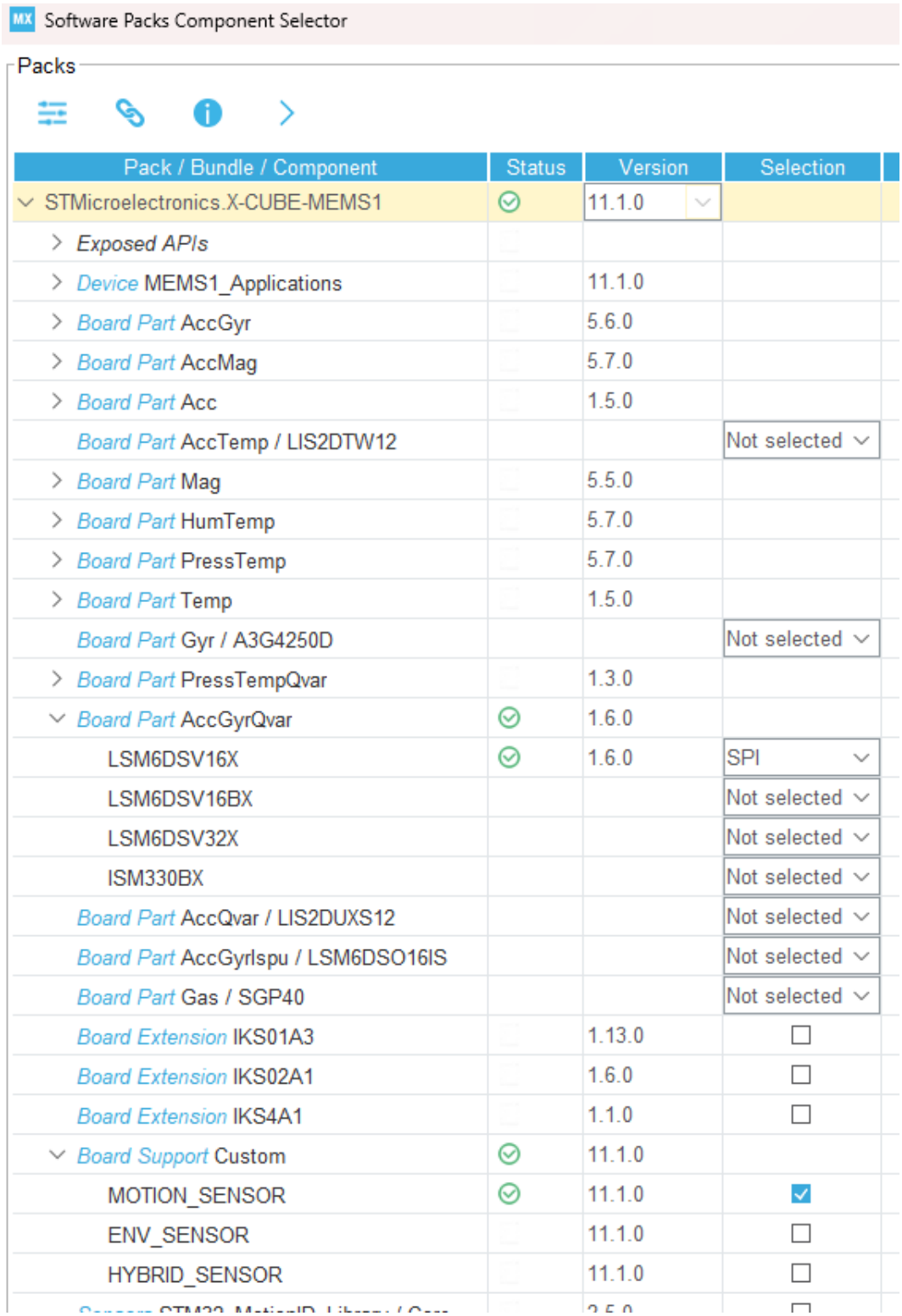
Under the X-CUBE-ISPU pack select the **SPI** for the ISM330IS sensor and the Board Support Custom.

Figure 31. **X-CUBE-ISPU pack selection**



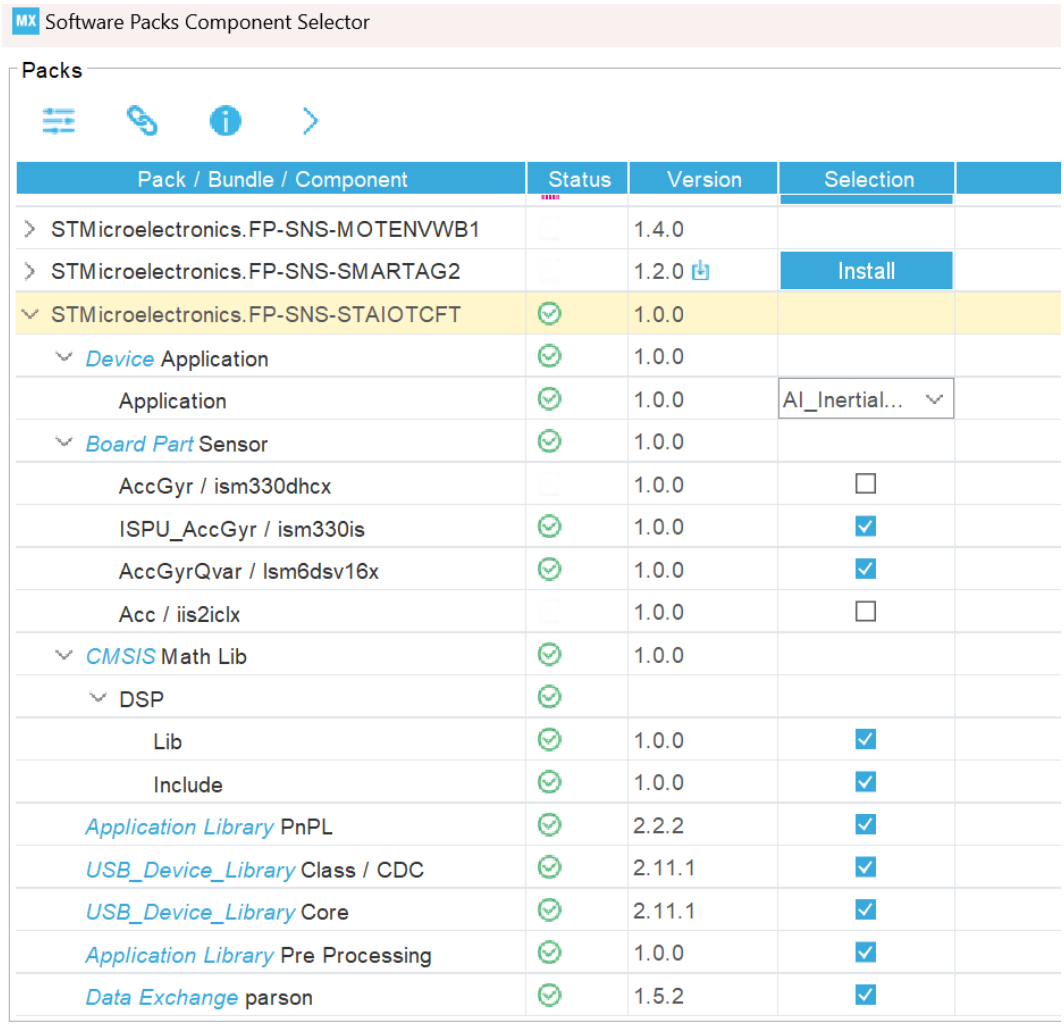
Under the X-CUBE-MEMS1 pack select the **SPI** for the LSM6DSV16X sensor and choose **MOTION_SENSOR** for the Board Support Custom.

Figure 32. **X-CUBE-MEMS1 pack selection**



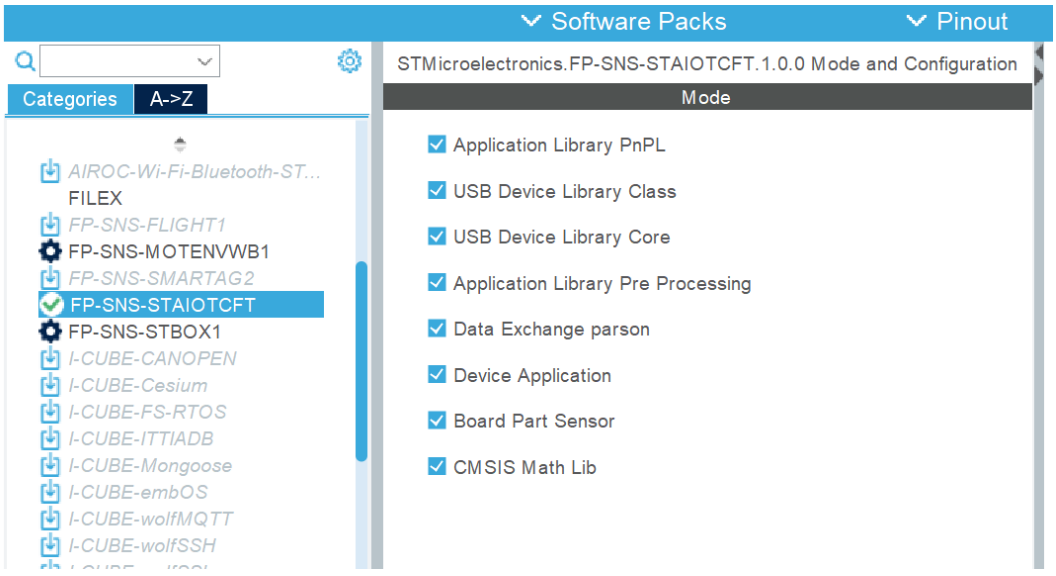
For the FP-SNS-STAIOTCFT select the Application, the sensors required and all the libraries (those include support for the PnPL, pre-processing, AI and USB). The required selections for the MKBOXPRO are provided here below.

Figure 33. FP-SNS-STAIOTCFT pack selection



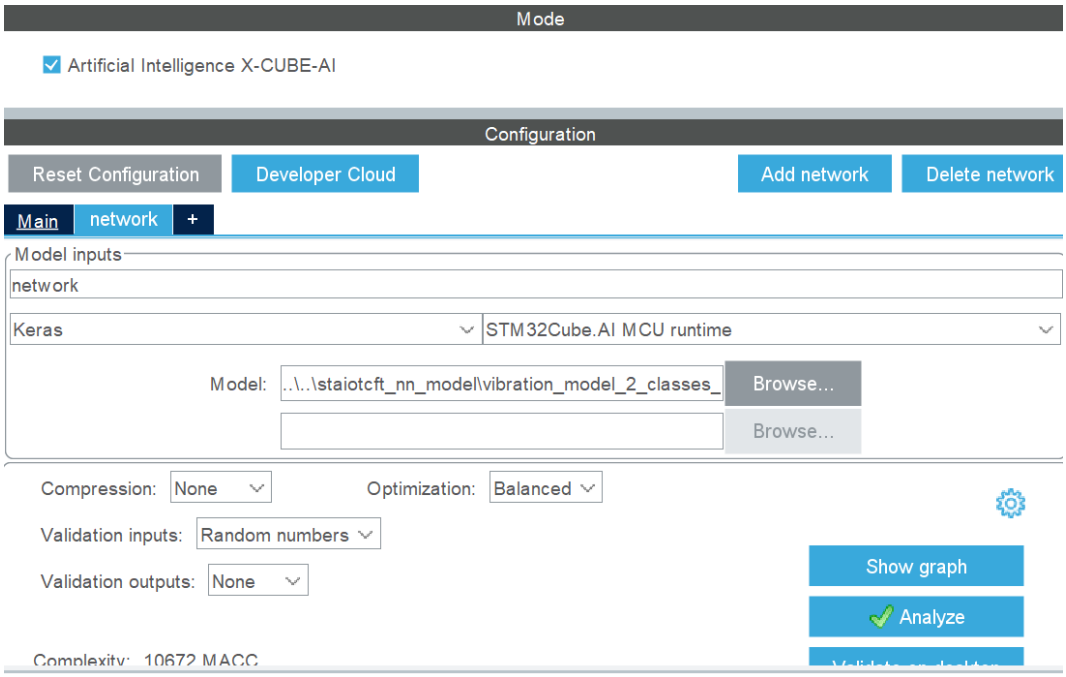
Then the packs must be activated enabling the check boxes.

Figure 34. FP-SNS-STAIOTCFT pack selection



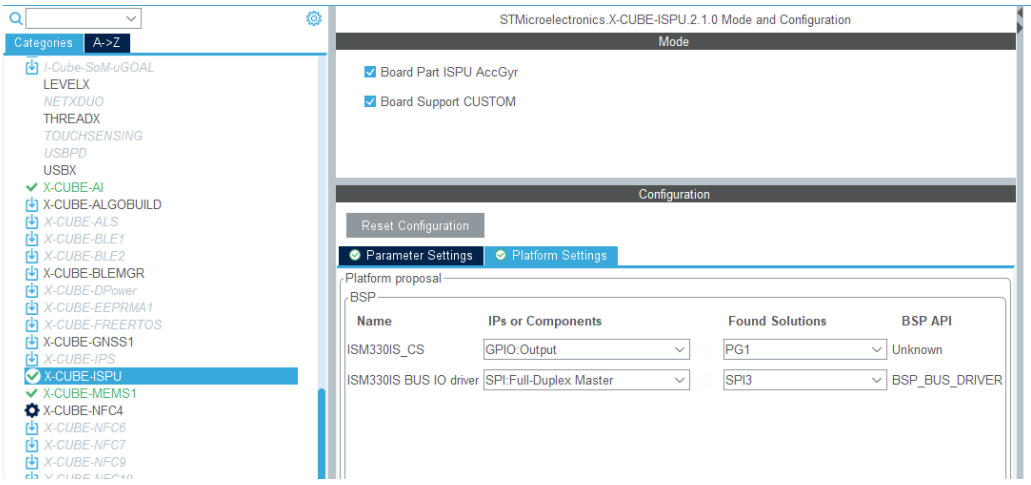
In the X-CUBE-AI section, add the network through the **h5** file (present in the repository under the Projects section in "*staiotcft_nn_model*") and click **Analyze**.

Figure 35. X-CUBE-AI pack selection



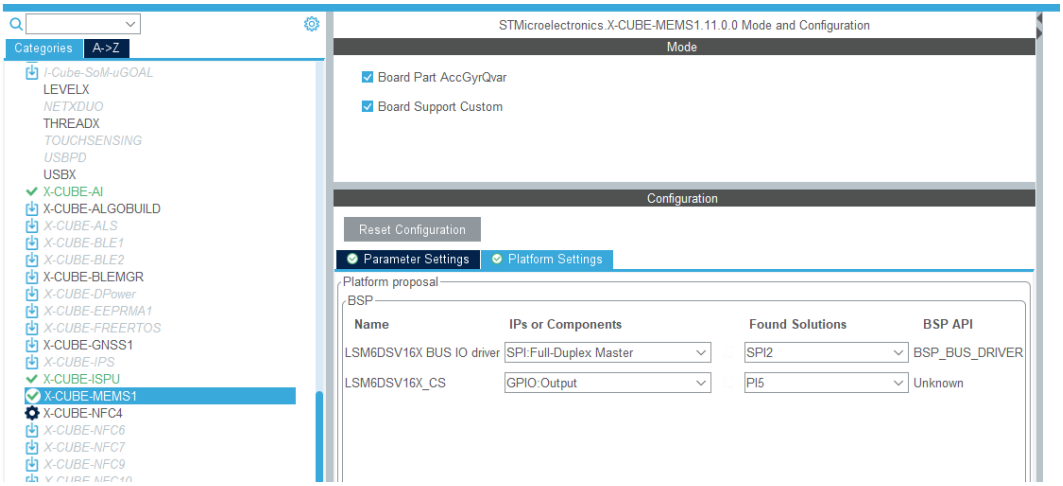
In the Platform Settings of the X-CUBE-ISPU connect the sensor to the MCU pins initialized at the beginning (in the GPIO Configuration).

Figure 36. X-CUBE-ISPU pack selection



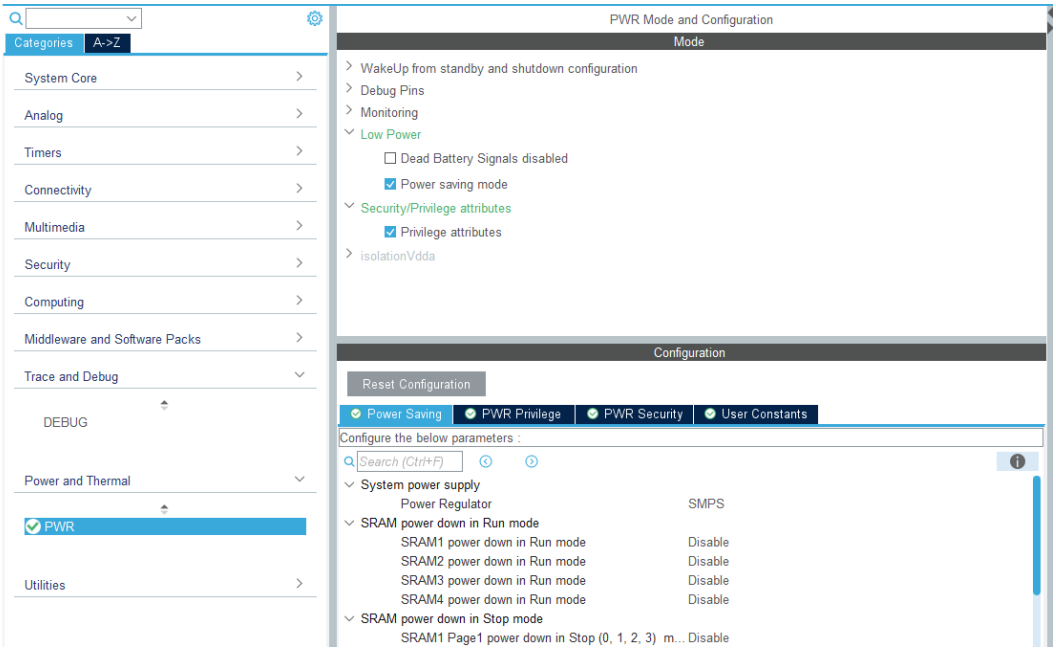
In the Platform Settings of the X-CUBE-MEMS1 connect the sensor to the MCU pins initialized at the beginning (in the GPIO Configuration).

Figure 37. X-CUBE-MEMS1 pack selection



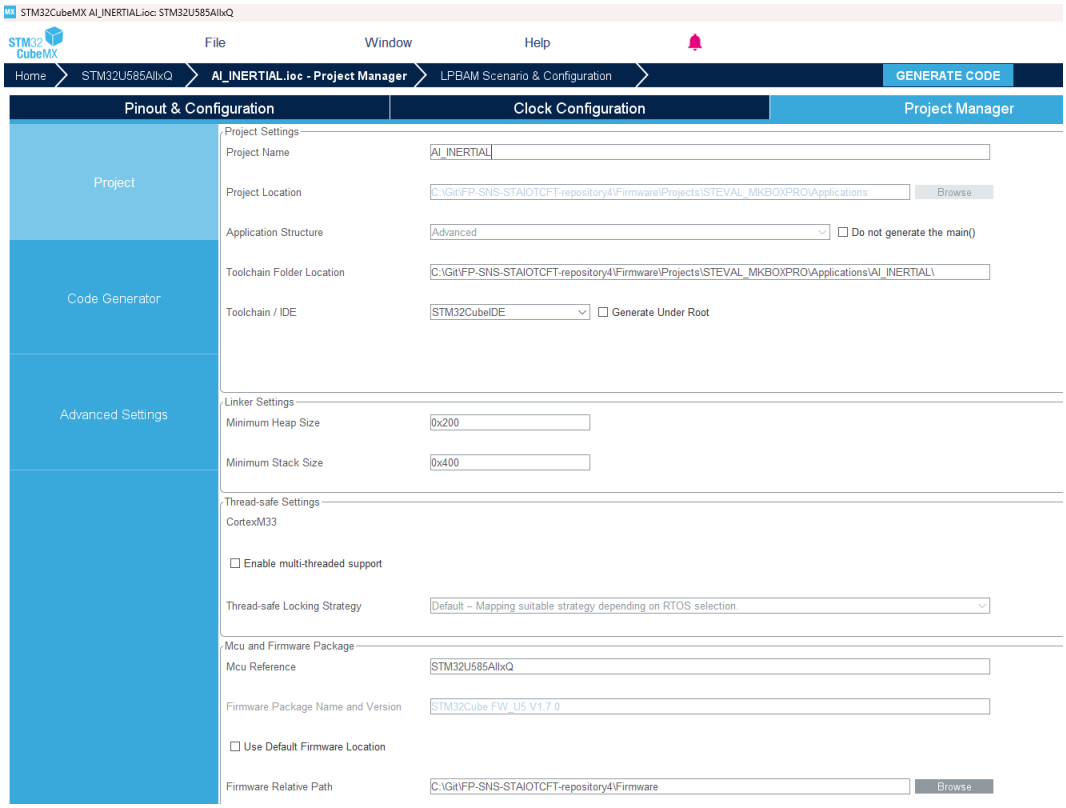
In the Power and Thermal section select **PWR** and configure it like in the picture below. This is required to avoid warning messages in the generation step.

Figure 38. PWR selection



Finally, in the **Project Manager** section, choose Project Name and Location, Toolchain/IDE STM32CubeIDE (Generate Under Root not selected) or EWARM or MDK-ARM. Check the Linker settings as the configuration of the Heap and Stack in the picture below, as well as the Thread-safe settings and the MCU and Firmware Package. Then click **GENERATE CODE**.

Figure 39. Project generation



1.16.1.2 *AI SSM application settings*

This section outlines how to configure STM32CubeMX for the STEVAL-MKBOXPRO development board with the AI SSM application starting directly from the STEVAL MKBOXPRO development board.

The other boards described in this guide are not yet supported by the AI SSM application.

Start

To start configuring the project, open STM32CubeMX and navigate to the “New Project” window. Select the “Board Selector” tab and search for STEVAL-MKBOXPRO in the Commercial Part Number field. Once the board appears in the list, select it and click “Start Project” to initialize a new project with the appropriate configuration for this evaluation board.

Set the clock HCLK to 160MHz to utilize the maximum frequency.

In the RCC section set the HSE and the LSE to Crystal/Ceramic Resonator.

GPIO Configuration

PA4 is set as GPIO_EXTI to handle external interrupts. PA4 is configured to manage events from the MLC. This configuration ensure that the sensor can trigger interrupts independently, allowing the firmware to respond to new predictions efficiently.

PB11 is configured as GPIO_EXTI for BLE events, alongside PA4 for MLC. In the NVIC settings, the corresponding interrupt lines must be enabled: EXTI Line 4 for PA4 and EXTI Line 11 for PB11. This ensures that the firmware can handle interrupts from all the sources effectively.

PA2 and PD4 are both configured as GPIO_Output. PA2 is used for the BLE_SPI_CS_line to control the chip select during SPI communication, while PD4 is assigned to the BLE_SPI_Resetline, enabling the microcontroller to reset the BLE module when required.

PI5 and PG1 are configured as GPIO_Output, serving as the chip select (CS) lines for the LSM6DSV16X and ISM330IS sensors, respectively. Both are set to High by default, while PI0 is set to Low. This configuration disables the CS pins initially and prepares them to be activated (switched from High to Low) when the SPI2 connection starts. PI0, connected to MCU_SEL, is configured to activate the SPI by overriding its default configuration for I2C. This setup ensures proper initialization and control of the SPI communication.

Timers

In this section, select TIM1, and set the Clock Source to Internal Clock. Then, in the Parameter Settings, set the Prescaler to 65535 and the Period to 1279. This configuration ensures that the timer generates an event at the desired interval, controlling the interval between two consecutives BLE packets. Finally, ensure that the TIM1 Update Interrupt is enabled in the NVIC settings to handle the timer event. Since the system clock frequency is 160 MHz, the combination of the prescaler and period is chosen to achieve the required timing interval for BLE advertising.

Also, select TIM3, and set the Clock Source to Internal Clock. Then, in the Parameter Settings, set the Prescaler to 16000 - 1 and the Period to 10000 - 1. This configuration ensures that the timer generates an event at the desired interval, controlling the duration of BLE advertising packets. Finally, ensure that the TIM3 Update Interrupt is enabled in the NVIC settings to handle the timer event. Since the system clock frequency is 160 MHz, the combination of the prescaler and period is chosen to achieve the required timing interval for BLE advertising.

Connectivity Configuration

I2C2

Under the I2C2 section set the I2C Mode. Then in the Parameter Settings leave everything as is. Just check the PB13 and PB14 as SCL and SDA and to enable both the NVIC settings.

SPI1

To configure SPI1 for the BLE component, set the SPI Mode to Full-Duplex Master. In the Parameter Settings, configure the following: set the Data Size to 8 Bits, the Clock Polarity (CPOL) to High, and the Clock Phase (CPHA) to 2 Edge. Adjust the Prescaler to achieve a Baud Rate of 1.25 MBits/s.

For the GPIO configuration, the SPI1 pins are as follows: PA5 is configured as SPI1_SCK (clock) with a Pull-up resistor to ensure signal stability, PA6 as SPI1_MISO (Master In Slave Out) without pull-up or pull-down resistors, and PA7 as SPI1_MOSI (Master Out Slave In) also without pull-up or pull-down resistors. This setup ensures reliable communication with the BLE component through SPI1.

SPI2

Under the SPI2 section set the Mode to Full Duplex Master. Then in the Parameter Settings: set Data Size to 8 Bits, change the Prescaler such that the Baud Rate is lower or equal to 10.0 MBits/s, then choose CPOL High and CPHA 2 Edge. All these settings are standard and are needed to work with the X-CUBE-MEMS1 pack, with the CUSTOM APIs.

The SPI2 pins used are PI1, PI3, and PI2, which correspond to SPI2_SCK (clock), SPI2_MOSI (Master Out Slave In), and SPI2_MISO (Master In Slave Out), respectively. SPI2_SCK on PI1 is configured with a Pull-up resistor to ensure proper signal integrity during operation. These pins are configured to facilitate reliable SPI communication for BLE functionality.

UART4

To configure UART4 for communicating with the serial terminal, set the mode to Asynchronous and disable hardware flow control. In the Parameter Settings, set the Baud Rate to 230400 Bits/s, Word Length to 8 Bits, Parity to None, and Stop Bits to 1. These settings ensure efficient and reliable communication with the serial terminal.

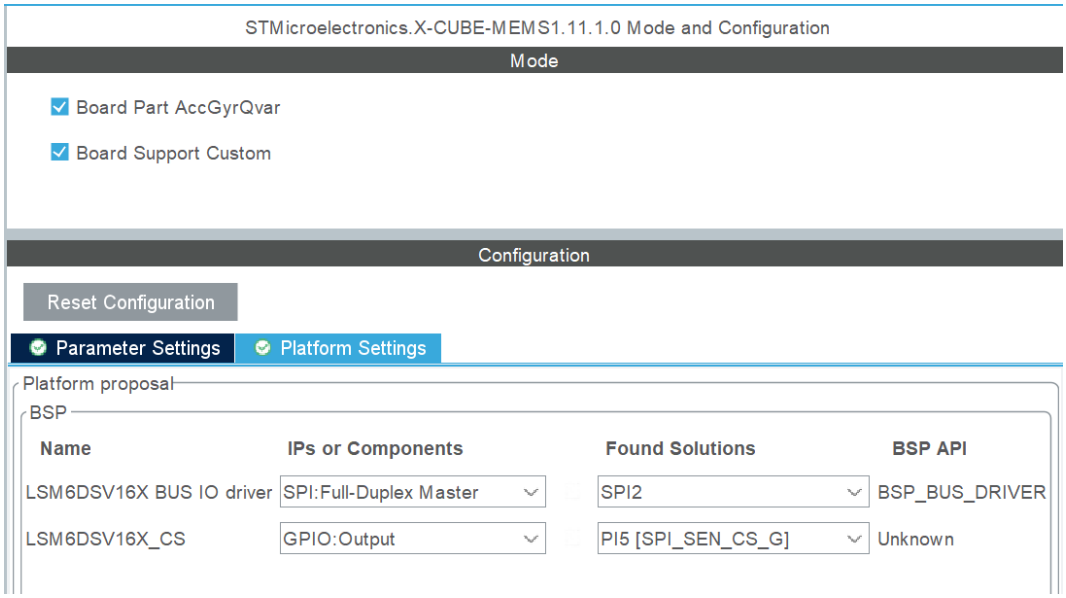
The GPIO configuration for UART4 uses PA0 and PA1 as the TX and RX signals, respectively. Both pins are set to Alternate Function mode with no pull-up or pull-down resistors applied. This setup facilitates proper data transmission and reception between the microcontroller and the serial terminal.

Software pack selection

In this part 5 software packs must be chosen with the configuration shown:

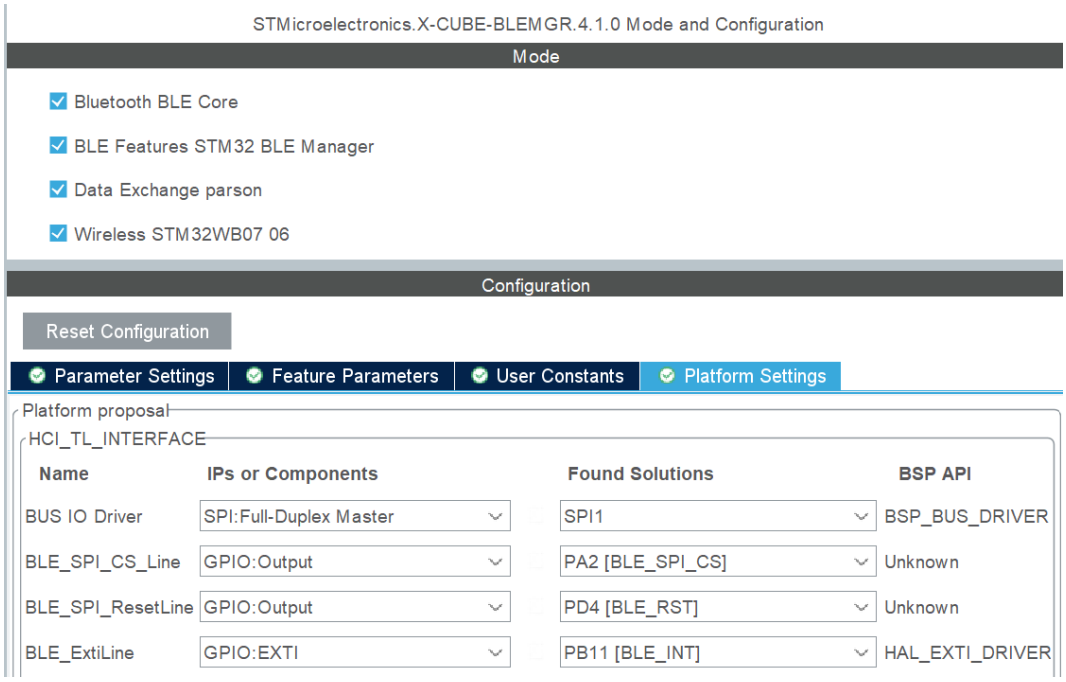
- X-CUBE-BLEMGR
- X-CUBE-MEMS1
- X-CUBE-NFC4 and NFC7
- FP-SNS-STAIOTCFT
- In the Platform Settings of the X-CUBE-MEMS1 connect the sensor to the MCU pins previously chosen.

Figure 40. X-CUBE-MEMS1 Configuration



- And also, for X-CUBE-BLEMGR:

Figure 41. X-CUBE-BLEMGR Configuration

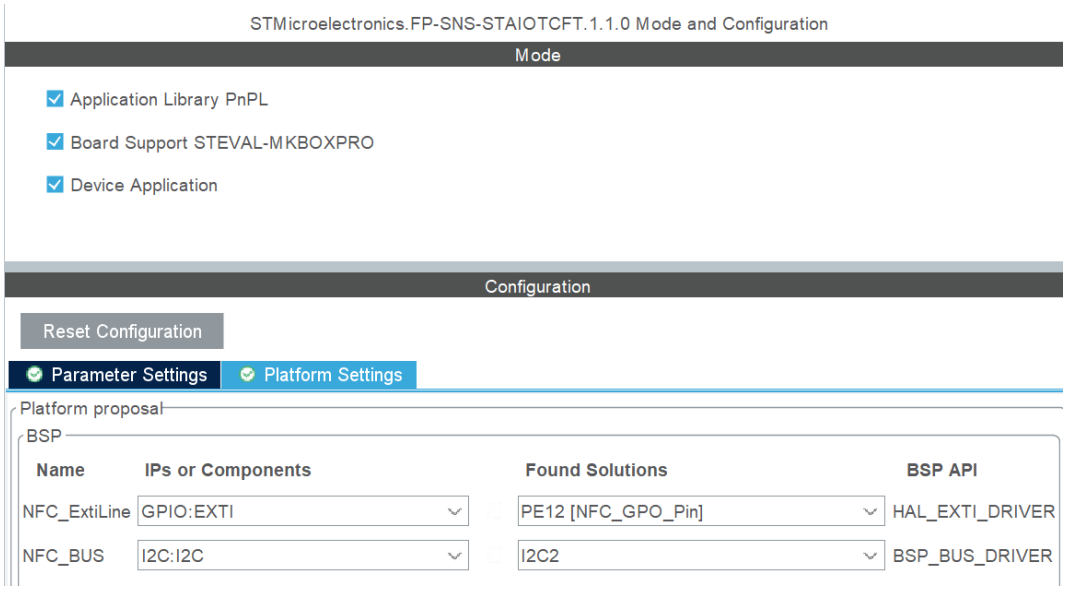


Moreover in the Parameter Settings set the HCI_READ_PACKET_SIZE to 260 and leave the rest unchanged. Finally in the Feature Parameters enable the ENABLE_CONFIG/CONSOLE/EXT_CONFIG and Banks Fw Id, Banks Swap, and Version Fw.

Finally, in the Project Manager section, choose Project Name and directory, Toolchain STM32CubeIDE and deselect the Generation under root.

- For X-CUBE-NFC4 and NFC7 just select the Board Part NFC (don't worry anout the Warning).
- Regarding the FP-SNS-STAIOTCFT select:

Figure 42. **FP-SNS-STAIOTCFT Configuration**

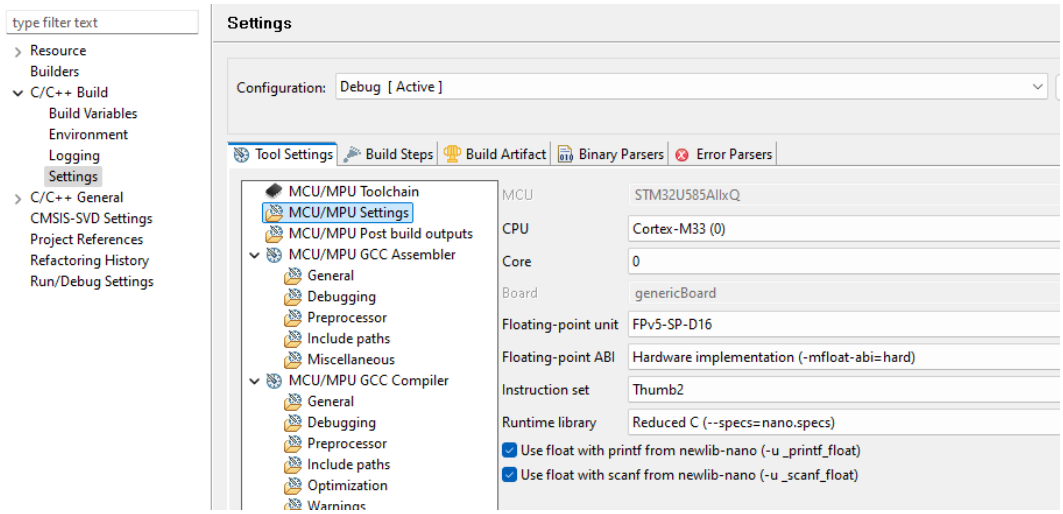


Finally, in the Project Manager section, choose Project Name and directory, Toolchain STM32CubeIDE and deselect the Generation under root. Note: leave Heap and Stack size to 0x8000

STM32CUBEIDE

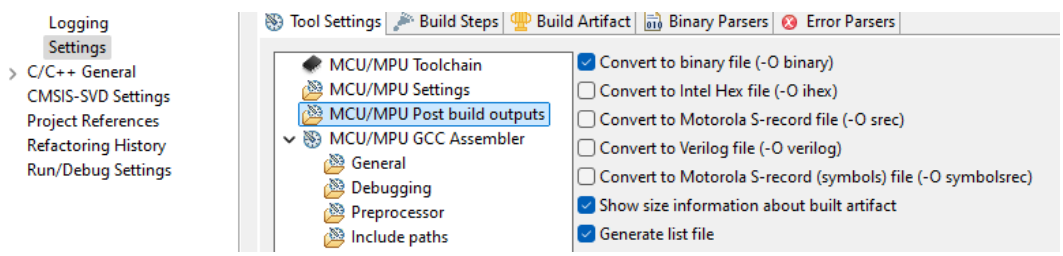
Before building the generated firmware a few steps need to be performed. First of all enabling the **printf** and **scanf** options from the **Properties** settings of the project, like in the pic here below.

Figure 43. **Enabling printf and scanf**



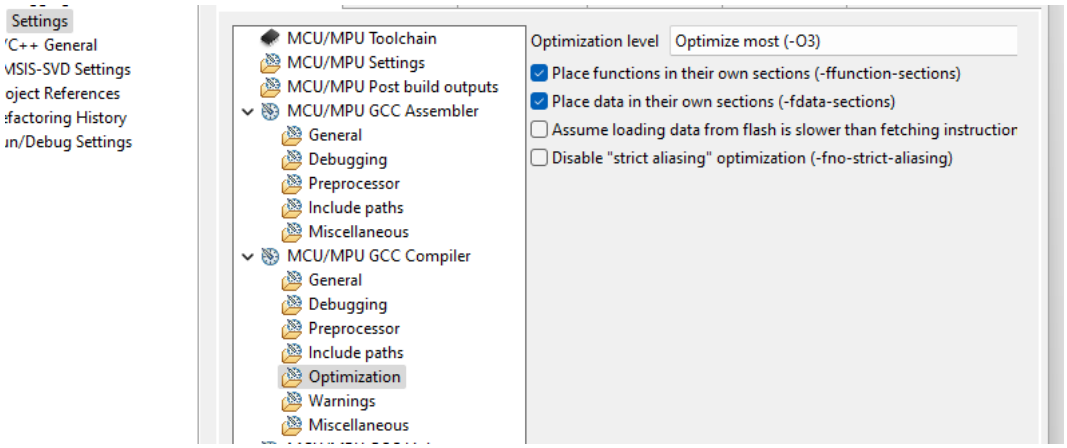
Enable the binary conversion, such that the compilation will produce a .bin file.

Figure 44. **Binary settings**



In the **Optimization** settings enable the -O3 (Optimize most) to enable a balanced set of optimizations.

Figure 45. Optimization settings

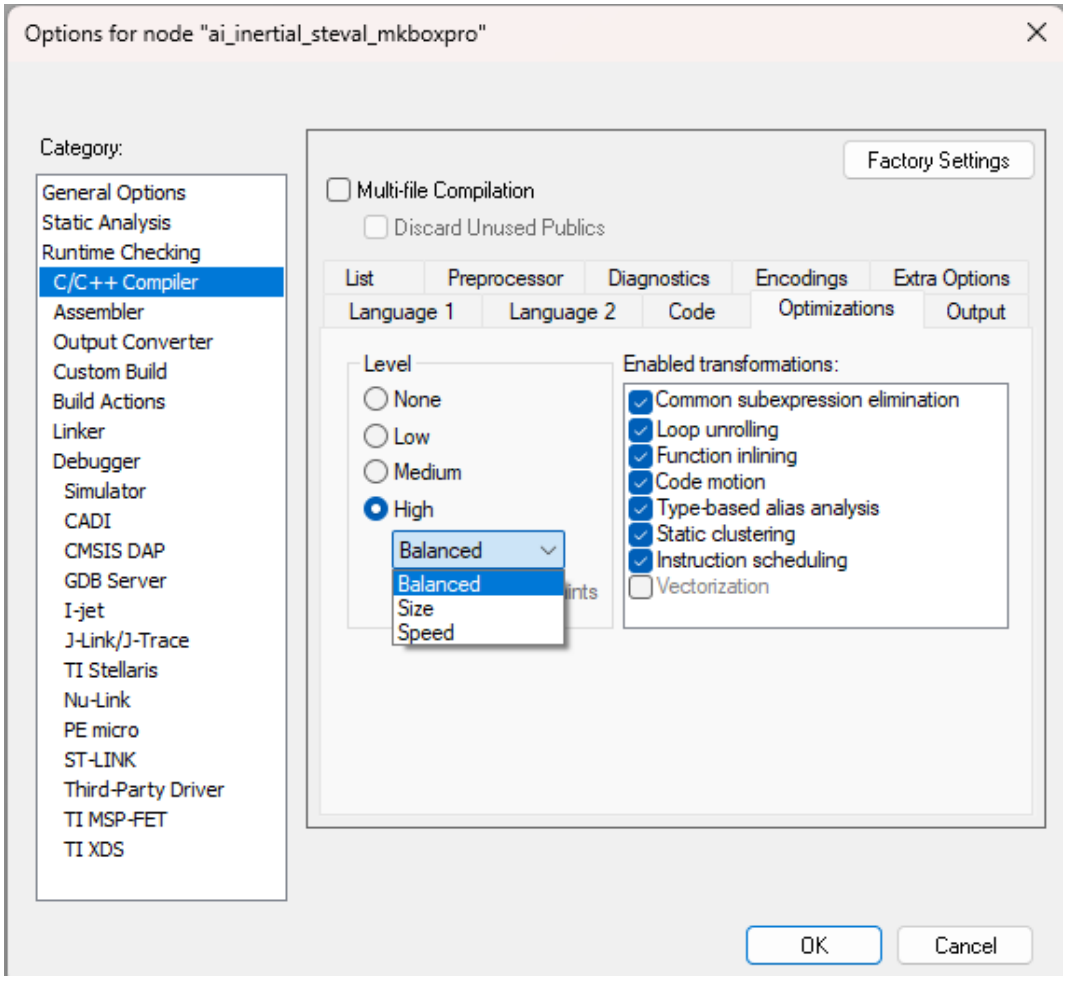


Then compile (in Debug mode) and debug the firmware directly using the STM32CubeIDE or compile (in Release mode) flashing the generated binary using a flashing tool like the STM32CubeProgrammer.

IAR Embedded Workbench IDE

Before proceeding with the compilation, it is required only to update the Optimization settings. As reported in the picture below.

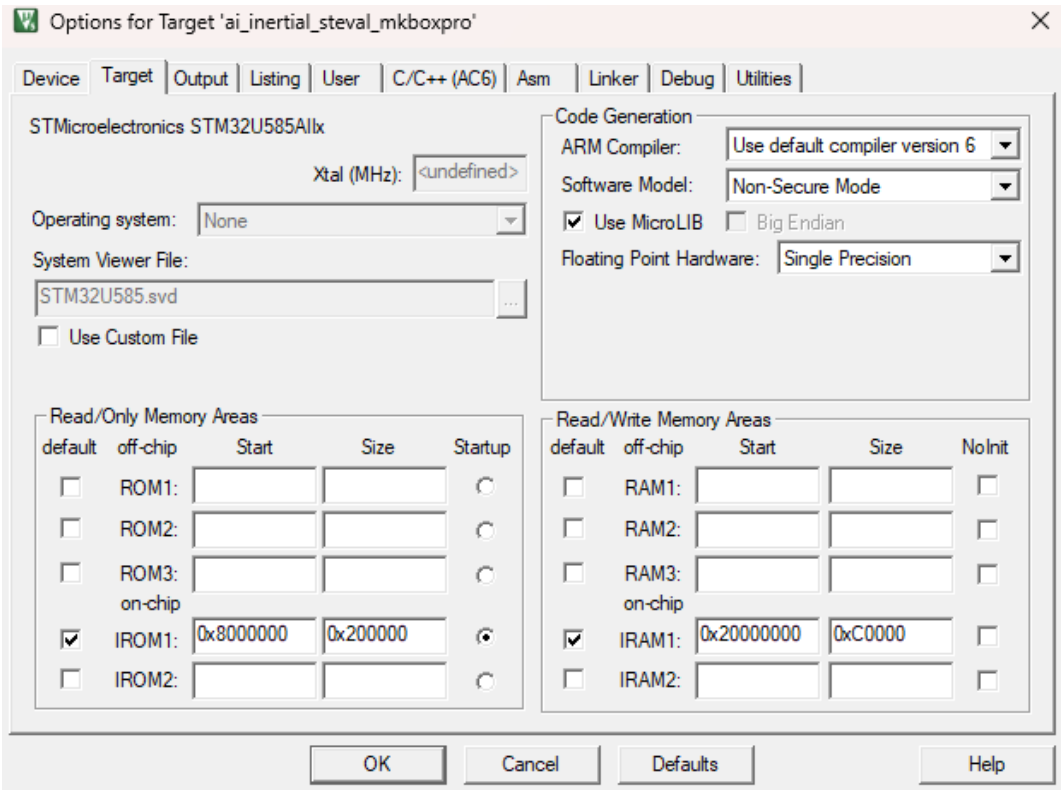
Figure 46. Optimization settings for IAR



MDK-ARM

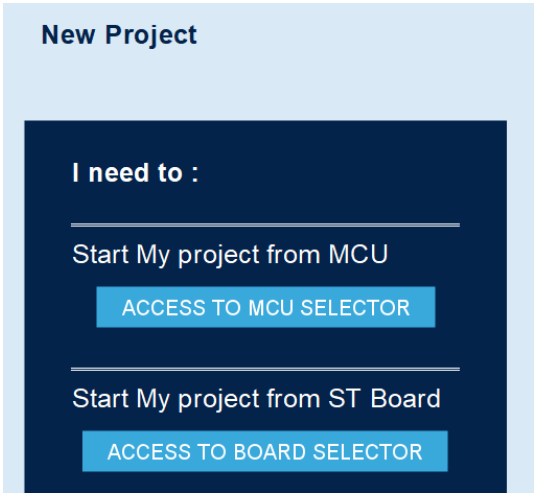
In KEIL it is only required to select the **MicroLIB** in order to avoid unproper behavior of the firmware.

Figure 47. MicroLIB selection



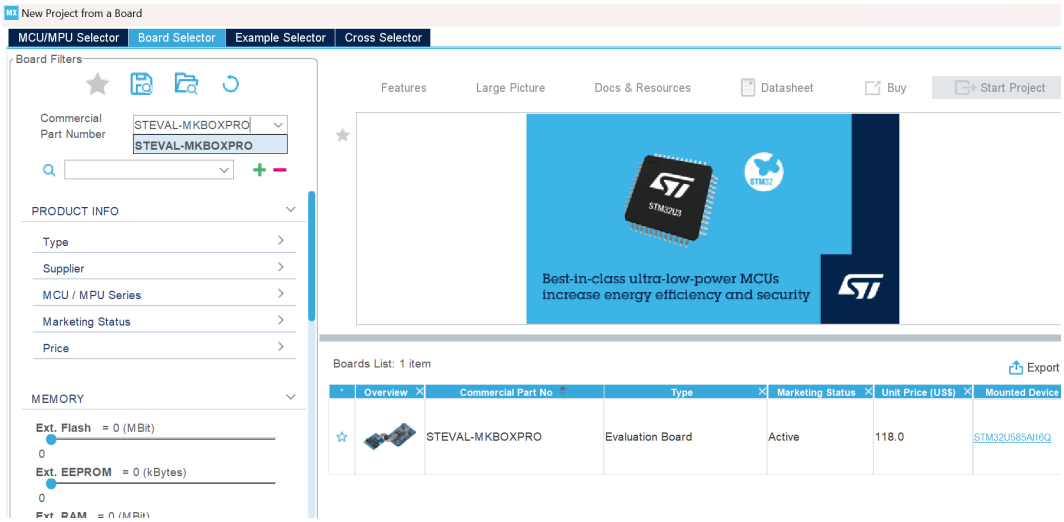
The following section applies only to ai_inertial application, it does not apply to AI SSM application.
The access through the board selector is a bit different with respect to the one from the MCU selector.
First of all start clicking on “Access to board selector”.

Figure 48. Starting point: board selection



Type MKBOXPRO in the Commercial Part Number tab and select the intended Evaluation board from the Board List.

Figure 49. Starting point: board selection

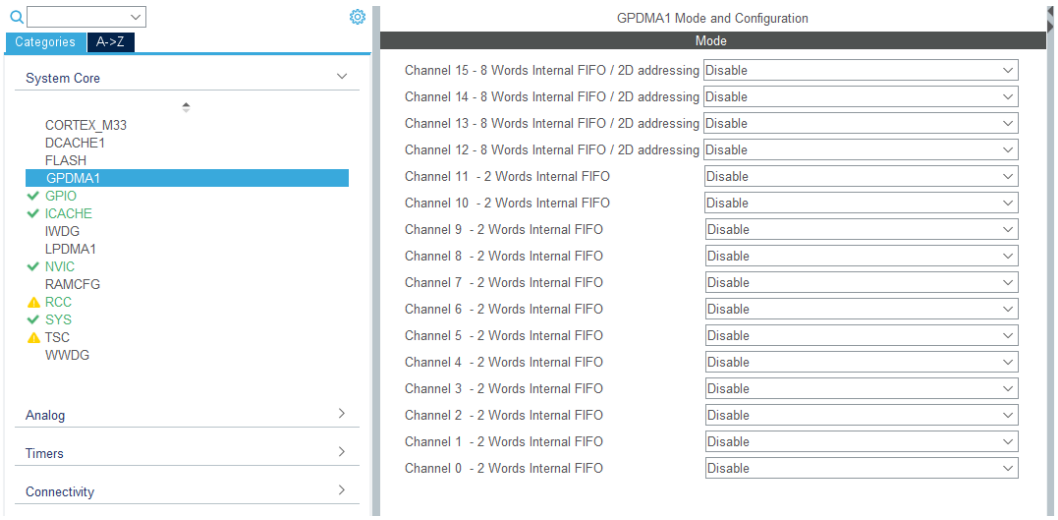


Click “Start Project” and “Yes” to Initialize all the peripherals in Default Mode.

The project will be pre-configured, but there are some alignments that the user has to make to avoid unintended mis-uses of the FP-SNS-STAIOTCFT pack.

First of all de-select all the GPDMA Channels.

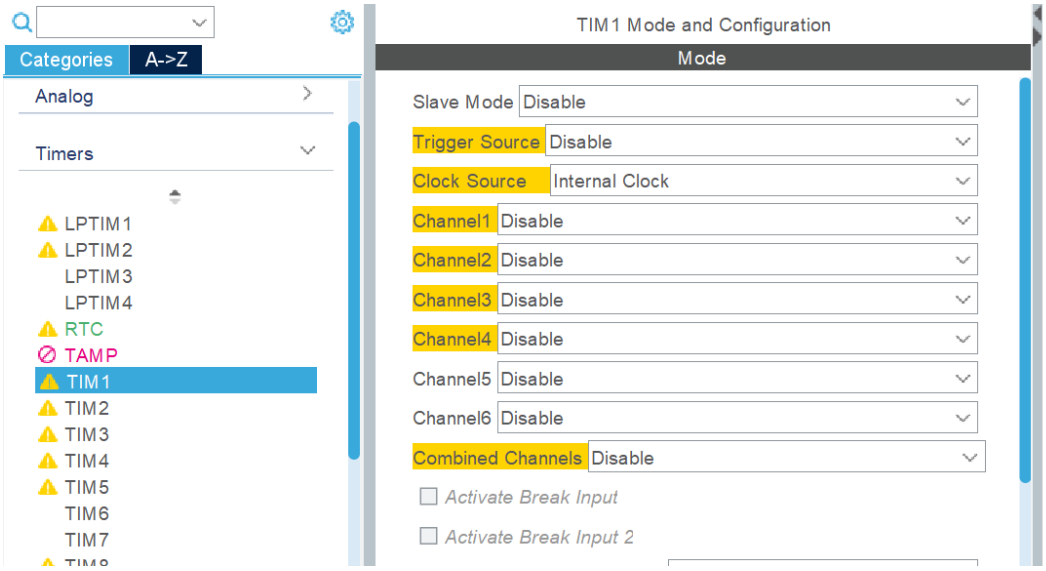
Figure 50. GPDMA1 de-select



In the GPIO section select the missing pins needed for the project: select the PD11 and enable its NVIC interrupt line, pull up PG1 and PH12 from their pin configuration menu and change the output level of PI0 to Low.

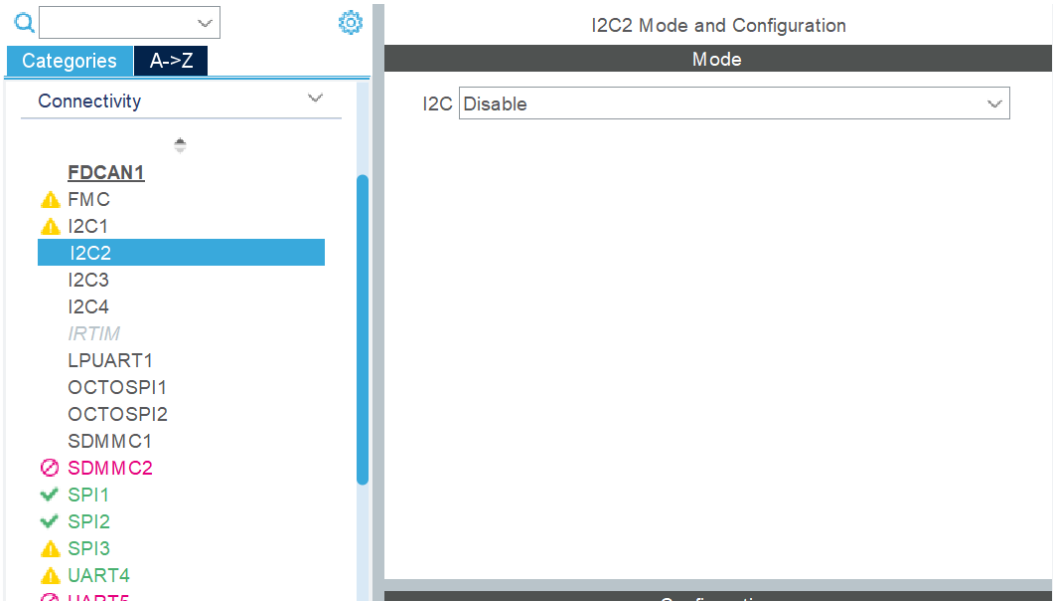
Activate the TIM1 Internal Clock Source and set the same parameters as for the path for MCU explained earlier.

Figure 51. **Internal Clock Source**



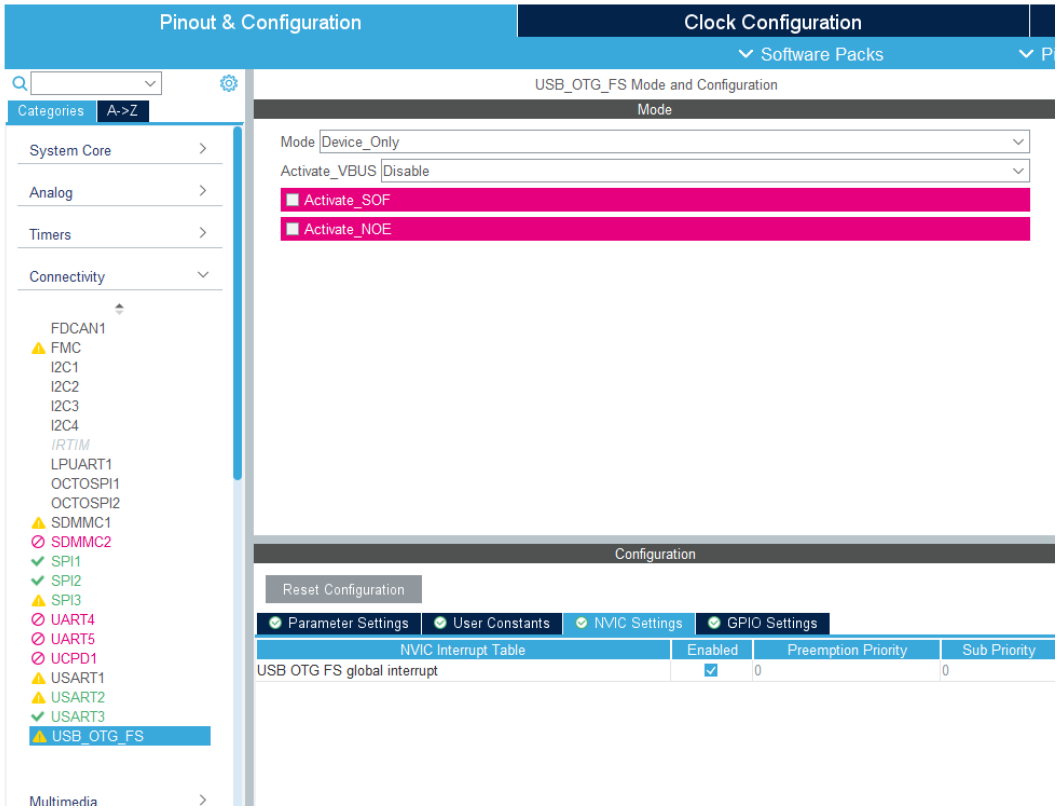
De-select all the I2Cs and the SDMMC1 from the Connectivity section.

Figure 52. **I2Cs and SDMMC1**



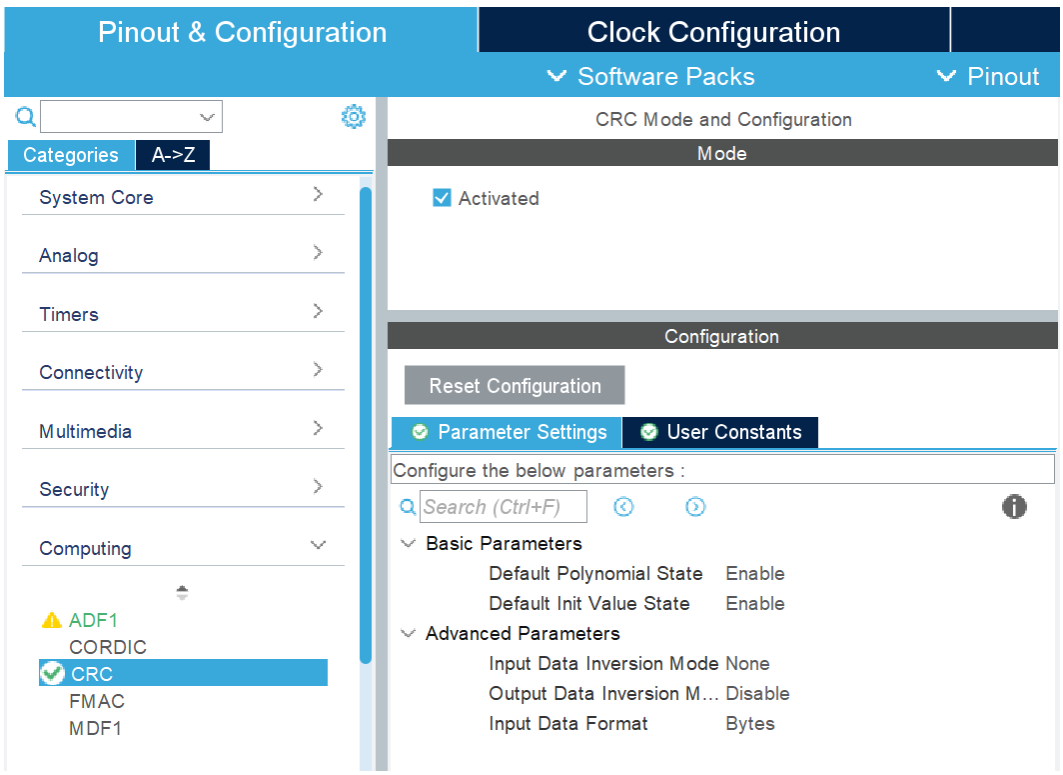
In the USB Configuration, under NVIC settings, enable the global interrupt.

Figure 53. USB Configuration



In the computing section select the CRC.

Figure 54. CRC Selection



Now its possible to select the Software Packs. This procedure is analogous to the one already done in the case starting from the MCU.

Everything now is ready for the project to be generated, thus click GENERATE CODE and go on with the selected IDEs.

The procedure is the same as before.

1.16.2 STEVAL - STWINBX1 (ai_inertial only)

This section outlines how to configure STM32CubeMX with the STEVAL-STWINBX1 development board starting directly from the STM32U585 micro-controller.

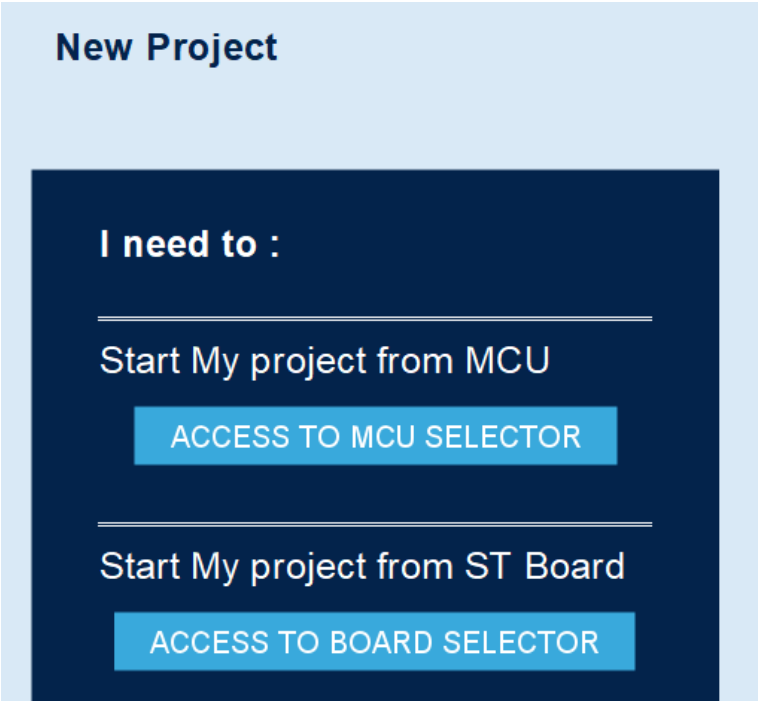
Before starting, if using the ISPU, it is important to properly set up the hardware to ensure that the ISM330IS sensor, which is embedded with the ISPU and connected via flat cable, is correctly connected. Please refer to the picture below for the correct setup.

Figure 55. STEVAL-STWINBX1 development board with ISM330IS sensor connected

It's possible to start a project directly from the MCU or choosing a specific target board.

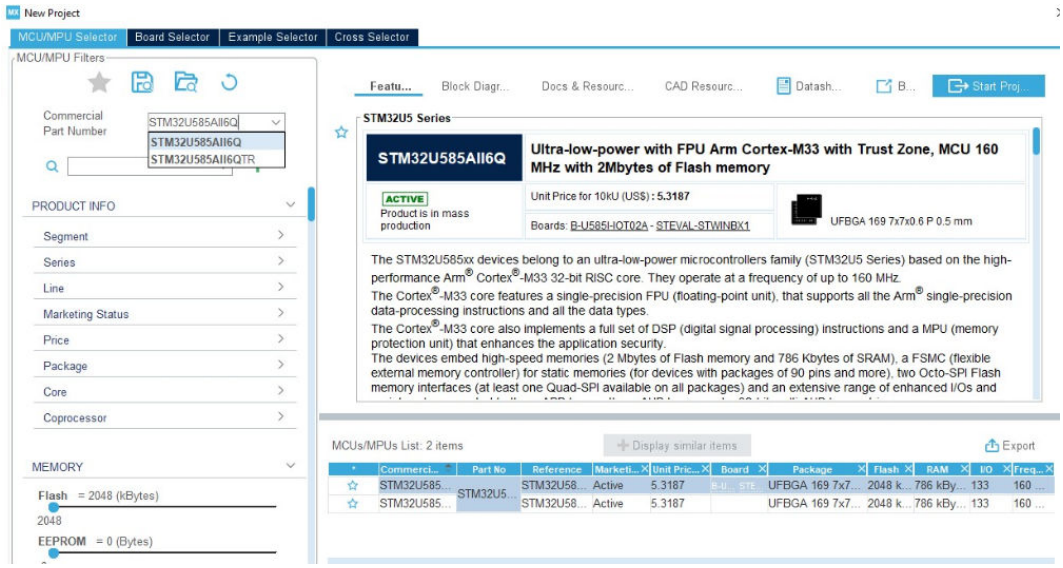
Let's start with the MCU, later we will address the access to the board selector

Figure 56. Starting point: MCU or BOARD selection



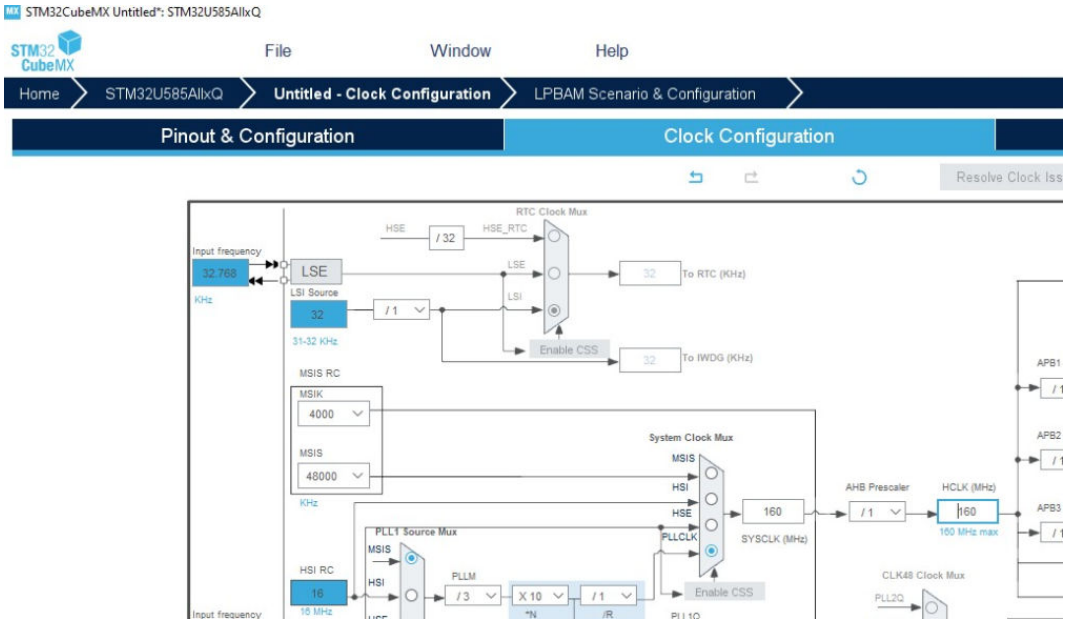
Create a new project for the U585 MCU.

Figure 57. Starting point: Micro-controller selection



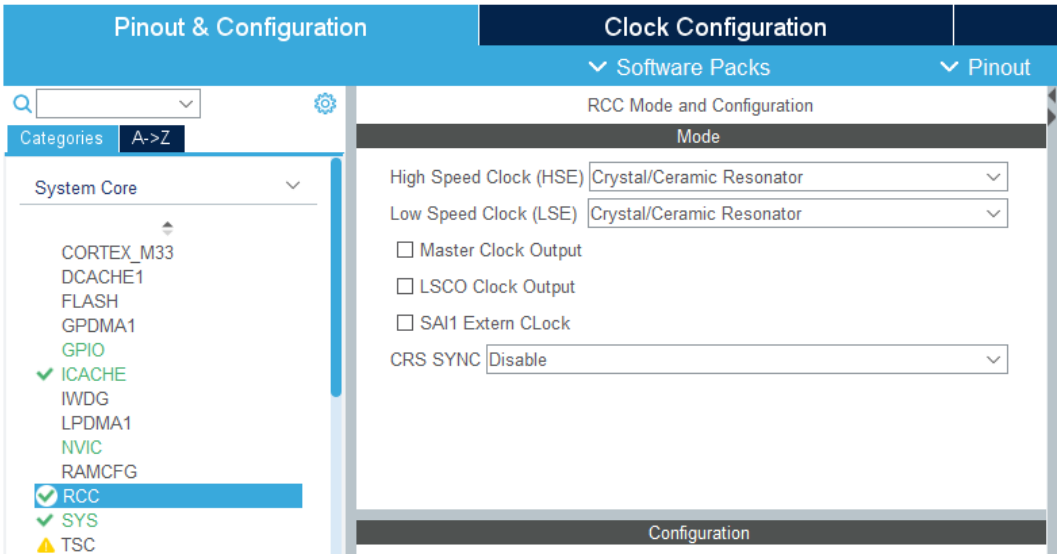
Set the clock **HCLK** to **160MHz** to utilize the maximum frequency.

Figure 58. Clock Frequency selection



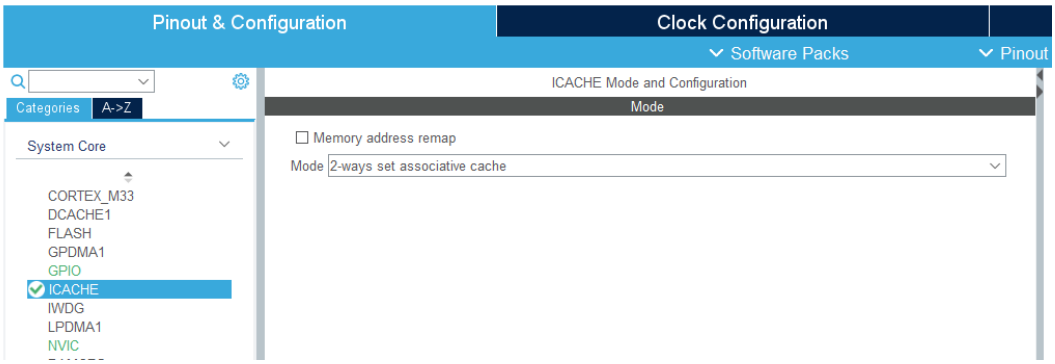
In the **RCC** section set the HSE and the LSE to **Crystal/Ceramic Resonator**.

Figure 59. **HSE and LSE clock selection**



Set the Cache mode to 2-ways set associative cache.

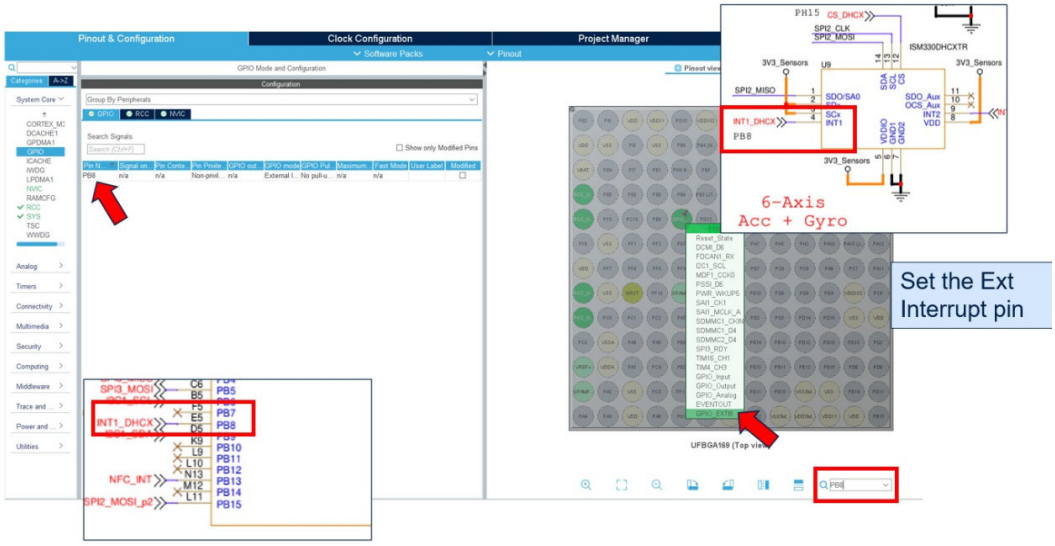
Figure 60. **ICACHE options**



GPIO Configuration

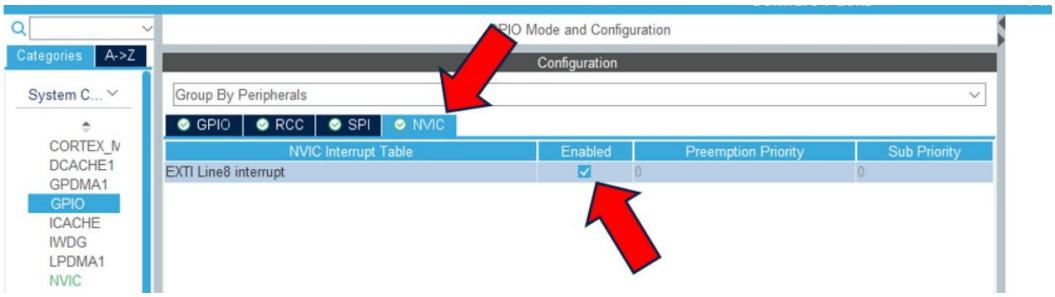
Set the **PB8** pin to **GPIO_EXTI8.** The PB8 is linked to the INT1_DHCX pin, which is excited when the FIFO of the ISM330DHCX is full of samples. For this reason is important to activate the interrupt pin of the sensor and link it to the MCU pin.

Figure 61. GPIO settings



Then going to **NVIC** section enable the interrupt pin.

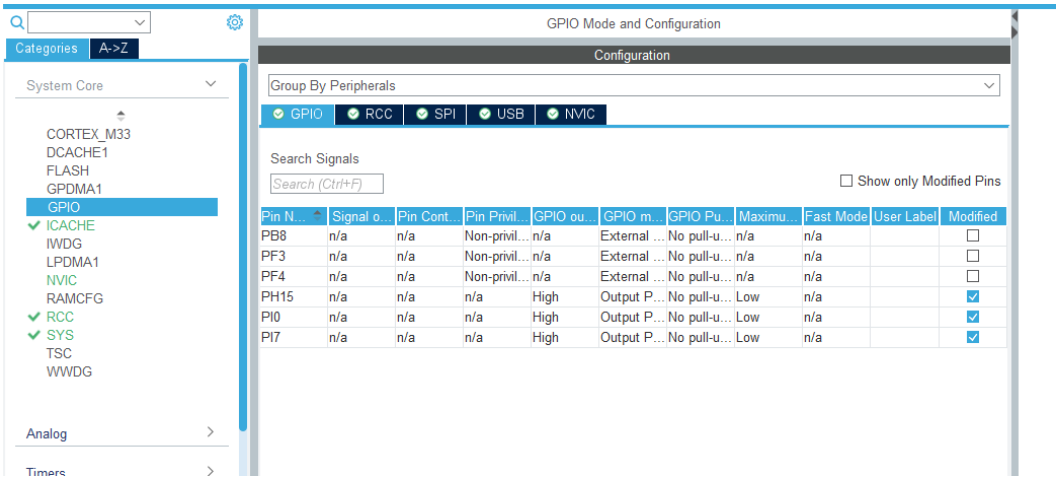
Figure 62. Interrupt from the ISM330DHCX



Same for the **PF3** and **PF4** which are the **INT1_ICLX** pin and **INT2_DHCX** respectively.

Moreover set the **PH15, PI7** and **PI0** to **GPIO_Output**, CS (chip select) of the ISM330DHCX, IIS2ICLX and the ISM330IS sensors respectively, to **High**. This is needed to disable the CS pins, they will be turned on once the SPI2 connection starts.

Figure 63. GPIO settings



Timers

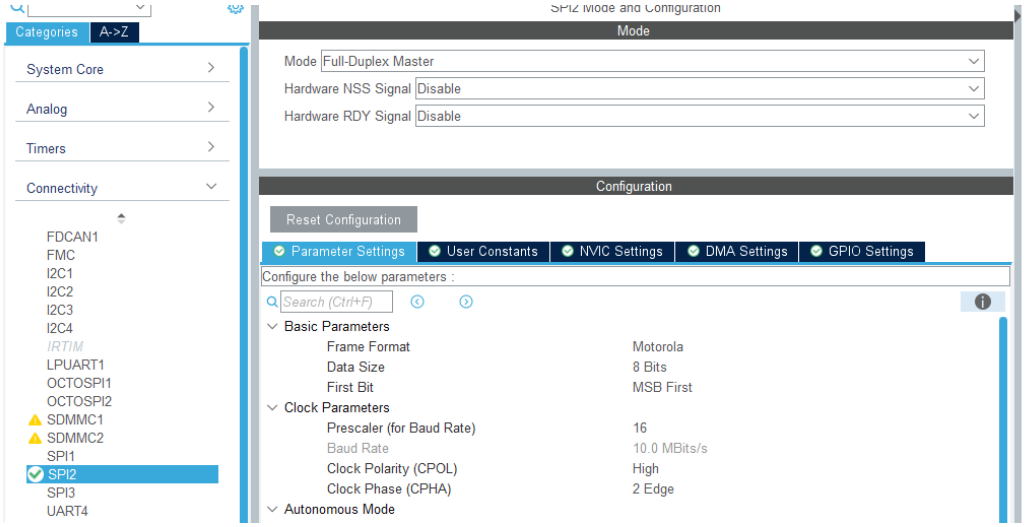
In this section select Tim1, Clock Source: Internal Clock, then in Parameter Settings set the Prescaler to 2400 and the Period to 65534. Then check the TIM1 Update Interrupt in the NVIC settings. This to obtain an interrupt every 1s, since the system clock frequency is 160MHz and to obtain almost the 1 Hz timer frequency the product between Prescaler and period must be equal to the system frequency.

Connectivity Configuration

SPI2

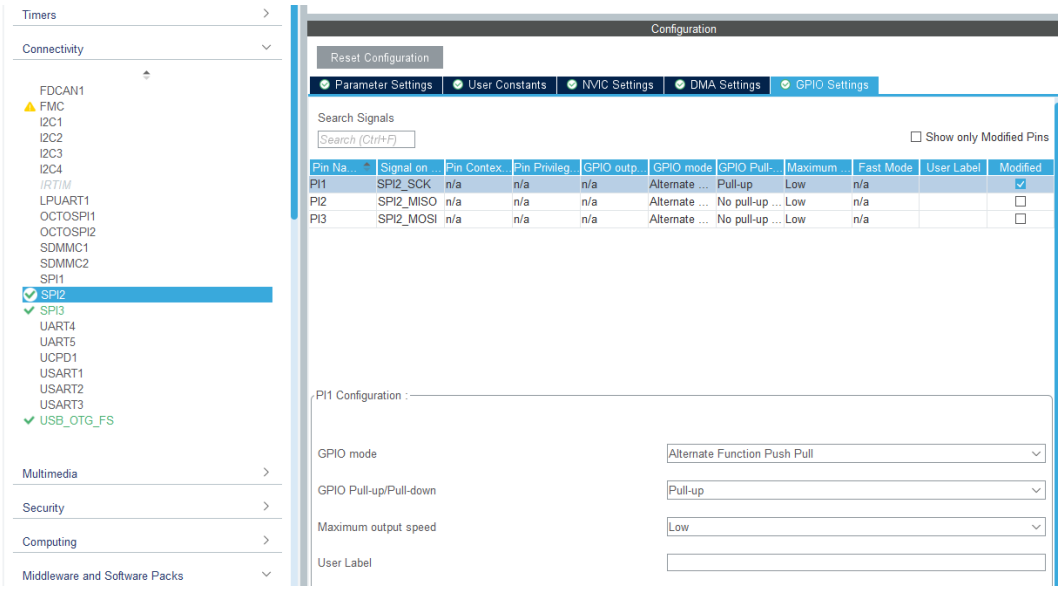
Under the **SPI2** section set the Mode to **Full Duplex Master**. Then in the **Parameter Settings**: set Data Size to 8 Bits, change the Prescaler such that the Baud Rate is lower or equal to 10.0 MBits/s, then choose CPOL High and CPHA 2 Edge. All these settings are standard and are needed to work with the **X-CUBE-MEMS1** pack, with the CUSTOM APIs, more information about this can be retrieved on the X-CUBE-MEMS1 manual (Chapter 7.3, page 64).

Figure 64. SPI2 selection - Parameter settings



In the **GPIO Settings** of the SPI2, change the pins such that **PD3** is the MISO, **PI1** the SCK and **PI3** the MOSI signals. This because the ISM330DHCX sensor is connected to the SPI bus through these pins. Moreover **Pull up** the **SPI2_SCK** pin as reported in the manual discussed earlier.

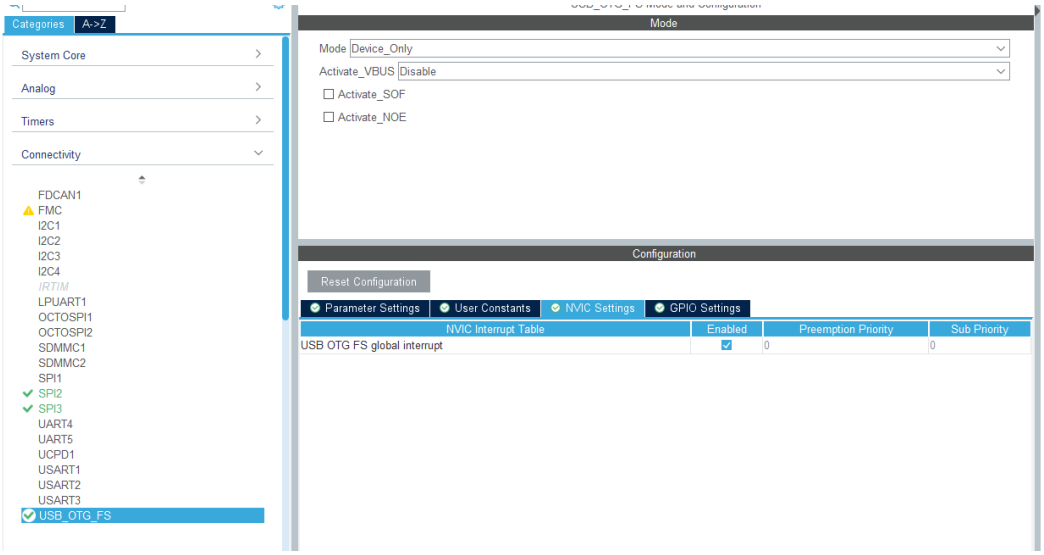
Figure 65. SPI2 selection – GPIO Settings



USB

Enable the **USB_OTG_FS** and select the **Device Only** Mode. Moreover enable the **global interrupt** under the NVIC Settings, the reason for this is to exploit the USB functionality at lower level. The interrupt is needed to receive and transmit data using the USB.

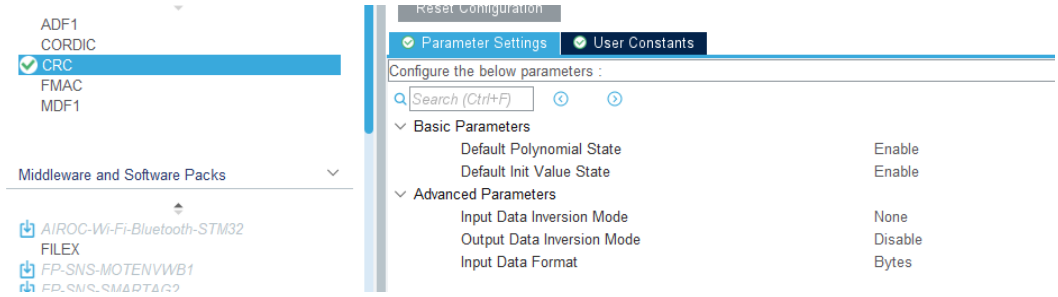
Figure 66. USB selection – NVIC Settings



CRC

The **CRC** must be activated to work with Neural Networks. Overall, using it can help ensuring the accuracy and reliability of trained models.

Figure 67. CRC selection



Software pack selection

In this part 4 software packs must be selected:

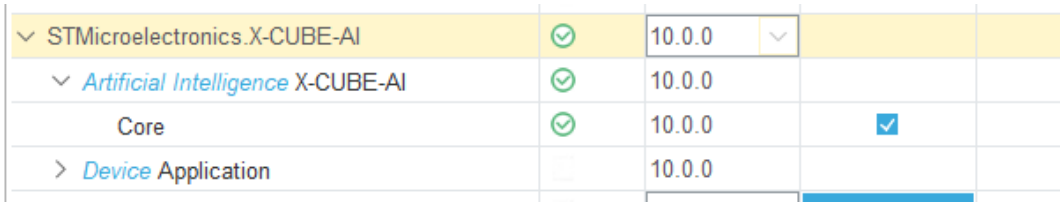
- **X-CUBE-AI**
- **X-CUBE-ISPU**
- **X-CUBE-MEMS1**
- **FP-SNS-STAIOTCFT**

The first three are needed to work with the Function Pack FP-SNS-STAIOTCFT.

Select “Software Packs” and then “Select Components”. From the “Software Packs Component Selector” window, the user has to select all the needed packs. In our case the ones above mentioned.

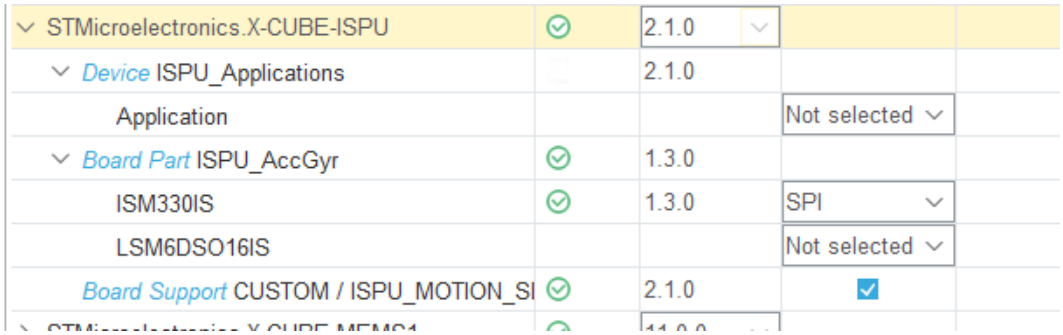
Under the X-CUBE-AI pack select only the **Core**.

Figure 68. **X-CUBE-AI pack selection**



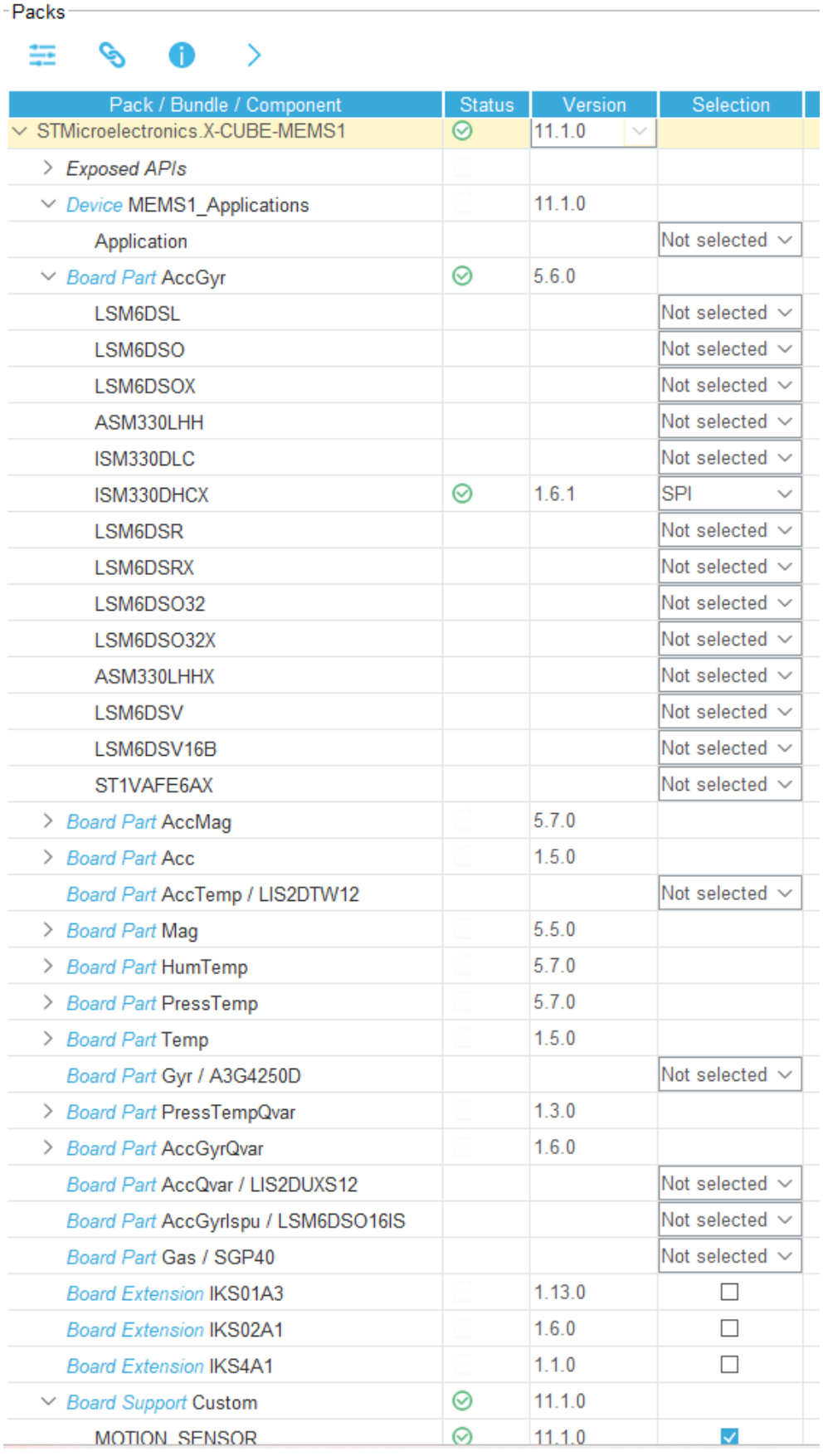
Under the X-CUBE-ISPU pack select the **SPI** for the ISM330IS sensor and the Board Support Custom.

Figure 69. **X-CUBE-ISPU pack selection**



Under the X-CUBE-MEMS1 pack select the **SPI** for the ISM330DHCX sensor and choose **MOTION_SENSOR** for the Board Support Custom.

Figure 70. **X-CUBE-MEMS1 pack selection**



Moreover select the IIS2ICLX sensor, if it is desired.

Figure 71. X-CUBE-MEMS1 pack selection

Board Part Acc	✓	1.4.0	
LIS2DW12			Not selected
LIS2DH12			Not selected
IIS2DLPC			Not selected
AIS2DW12			Not selected
AIS328DQ			Not selected
AIS3624DQ			Not selected
H3LIS331DL			Not selected
IIS2ICLX	✓	1.4.0	SPI
AIS2IH			Not selected

For the FP-SNS-STAIOTCFT select the Application, the sensors required and all the libraries (those include support for the PnPL, pre-processing, AI and USB). The required selections for the STWINBX are provided here below.

Figure 72. FP-SNS-STAIOTCFT pack selection

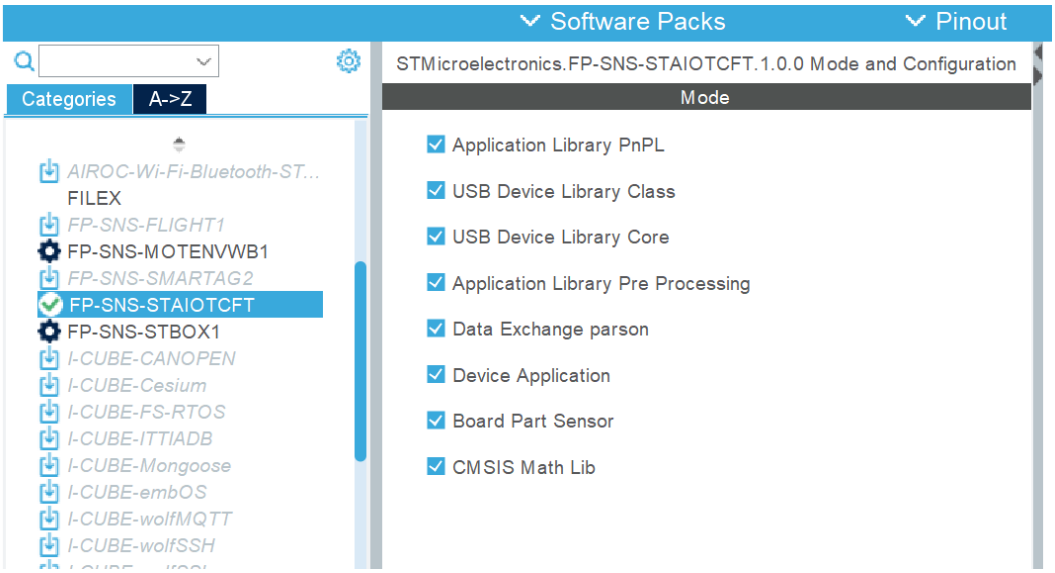
Software Packs Component Selector

Packs

Pack / Bundle / Component	Status	Version	Selection
STMicroelectronics.FP-SNS-MOTENVWB1		1.4.0	
STMicroelectronics.FP-SNS-SMARTAG2		1.2.0	Install
STMicroelectronics.FP-SNS-STAIOTCFT	✓	1.0.0	
Device Application	✓	1.0.0	
Application	✓	1.0.0	AI_Inertial...
Board Part Sensor	✓	1.0.0	
AccGyr / ism330dhcx		1.0.0	☐
ISPU_AccGyr / ism330is	✓	1.0.0	☑
AccGyrQvar / lsm6dsv16x	✓	1.0.0	☑
Acc / iis2iclx		1.0.0	☐
CMSIS Math Lib	✓	1.0.0	
DSP	✓		
Lib	✓	1.0.0	☑
Include	✓	1.0.0	☑
Application Library PnPL	✓	2.2.2	☑
USB_Device_Library Class / CDC	✓	2.11.1	☑
USB_Device_Library Core	✓	2.11.1	☑
Application Library Pre Processing	✓	1.0.0	☑
Data Exchange parson	✓	1.5.2	☑

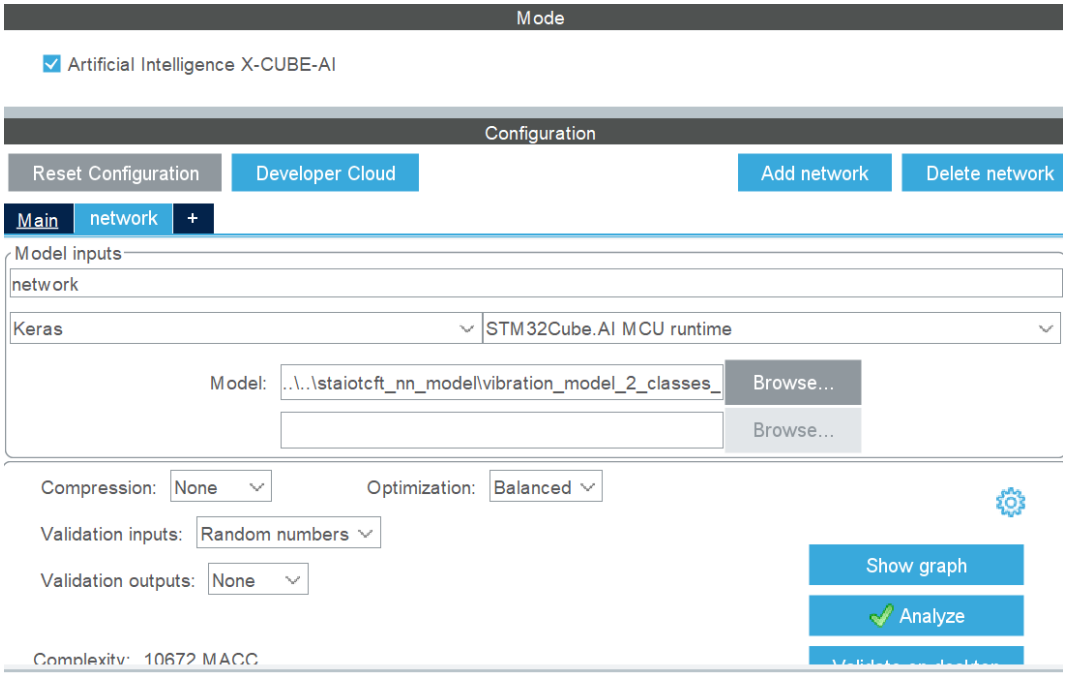
Then the packs must be activated enabling the check boxes.

Figure 73. FP-SNS-STAIOTCFT pack selection



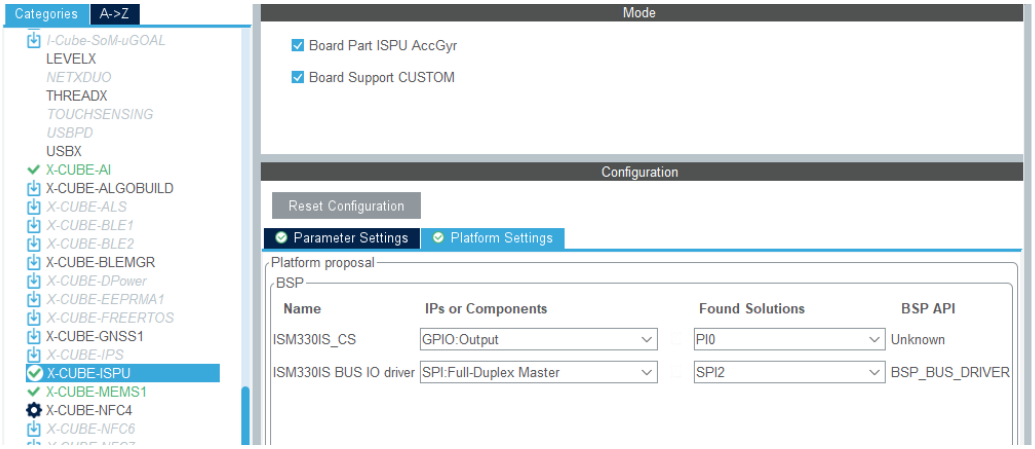
In the X-CUBE-AI section, add the network through the **h5** file (present in the repository under the Projects section in "*staiotcft_nn_model*") and click **Analyze**.

Figure 74. X-CUBE-AI pack selection



In the Platform Settings of the X-CUBE-ISPU connect the sensor to the MCU pins initialized at the beginning (in the GPIO Configuration).

Figure 75. **X-CUBE-ISPU pack selection**



In the Platform Settings of the X-CUBE-MEMS1 connect the sensor to the MCU pins initialized at the beginning (in the GPIO Configuration).

Figure 76. **X-CUBE-MEMS1 pack selection**

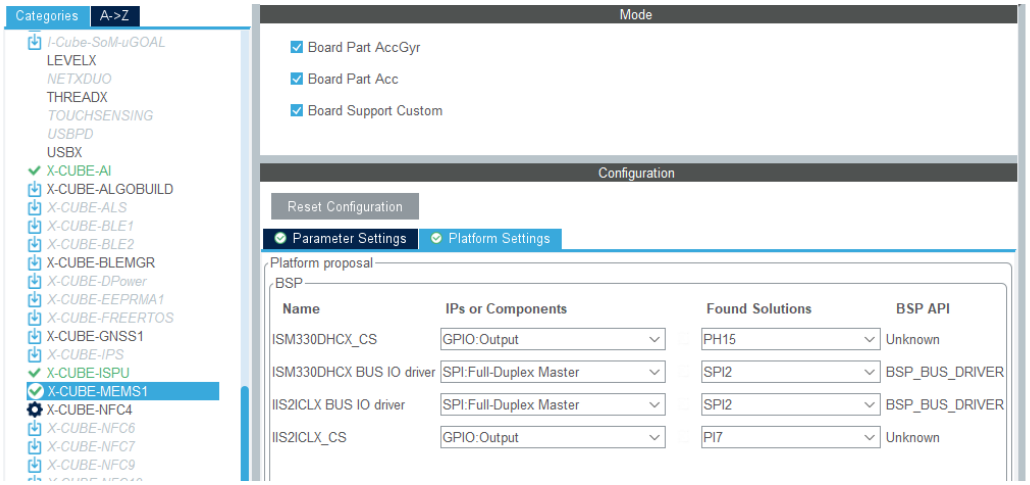
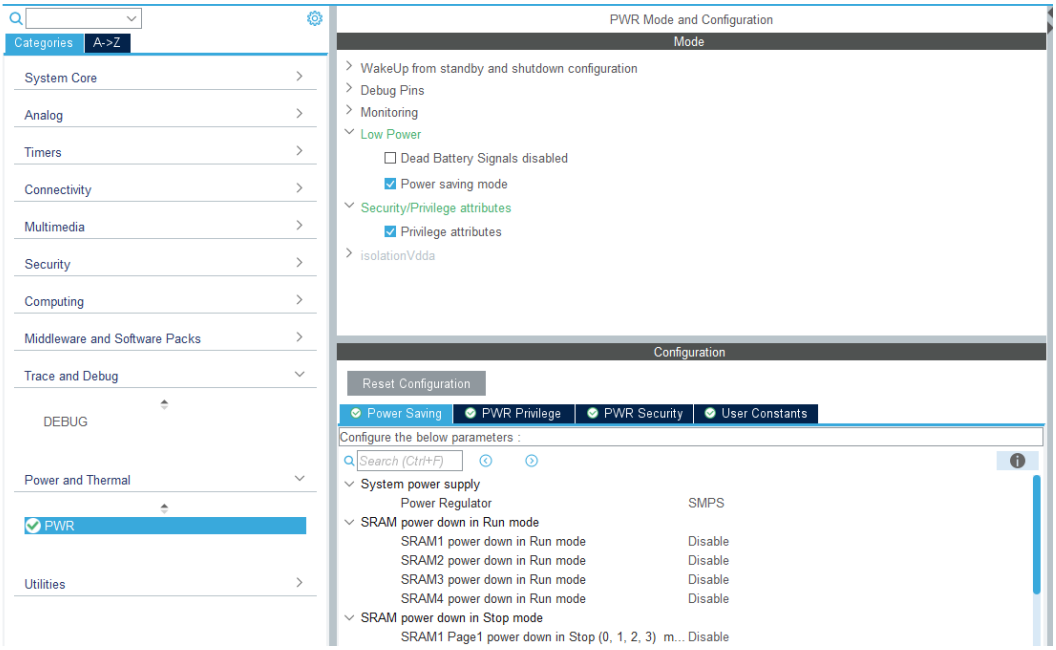


***Figure 77** X-CUBE-MEMS1 pack selection*

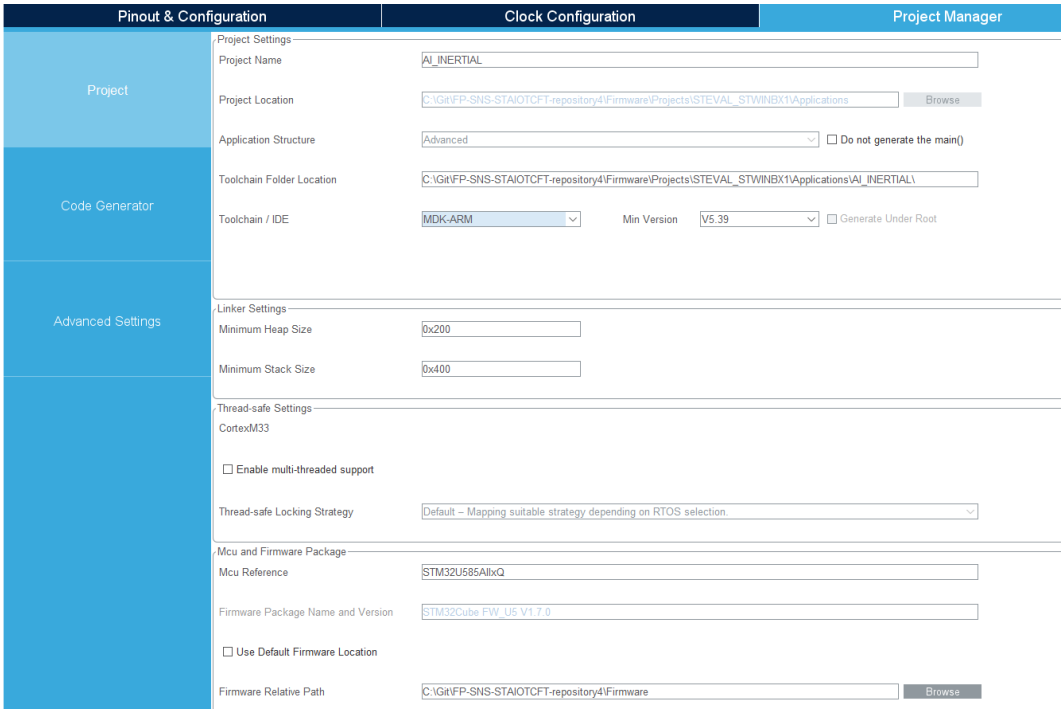
In the Power and Thermal section select **PWR** and configure it like in the picture below. This is required to avoid warning messages in the generation step.

Figure 77. PWR selection



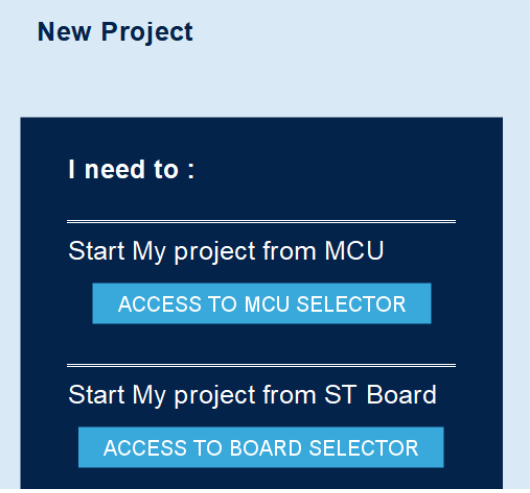
Finally, in the **Project Manager** section, choose Project Name and Location, Toolchain/IDE STM32CubeIDE (Generate Under Root not selected) or EWARM or MDK-ARM. Check the Linker settings as the configuration of the Heap and Stack in the picture below, as well as the Thread-safe settings and the Mcu and Firmware Package. Then click **GENERATE CODE**.

Figure 78. Project generation



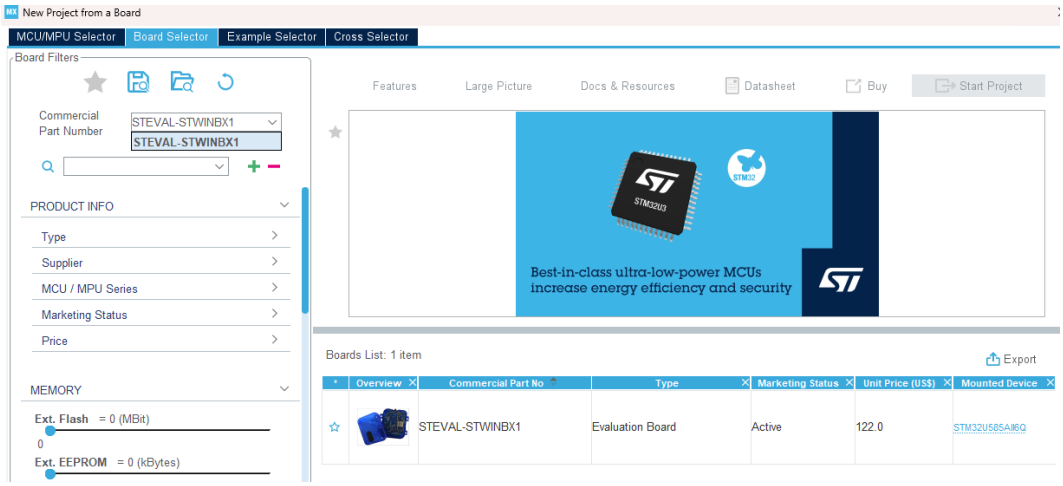
The rest of the steps are exactly the same as for the STEVAL-MKBOXPRO development board.
The access through the board selector is a bit different with respect to the one from the MCU selector.
First of all start clicking on “Access to board selector”.

Figure 79. Starting point: Board selection



Type STWINBX1 in the Commercial Part Number tab and select the intended Evaluation board from the Board List.

Figure 80. Starting point: Board selection

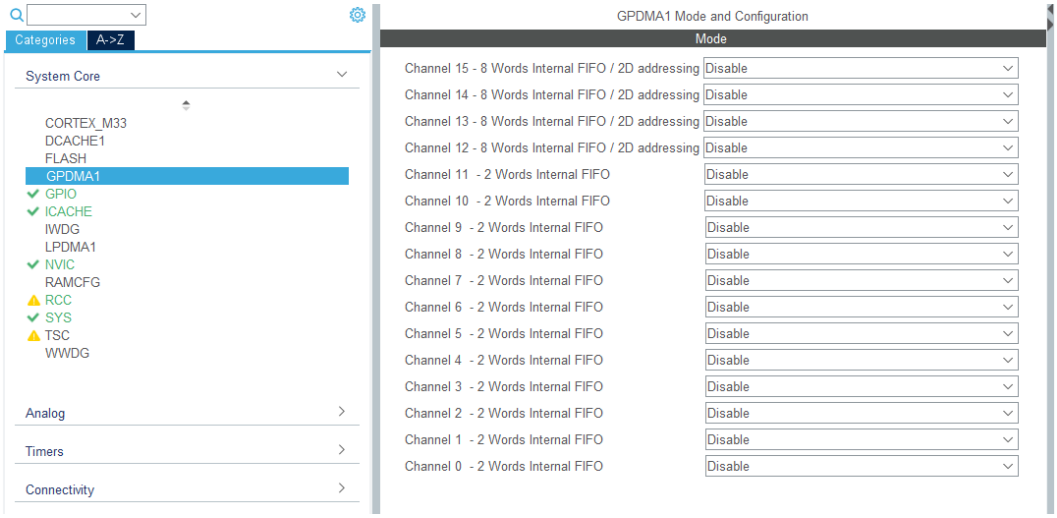


Click “Start Project” and “Yes” to Initialize all the peripherals in Default Mode.

The project will be pre-configured, but there are some alignments that the user has to make to avoid unintended mis-uses of the FP-SNS-STAIOTCFT pack.

First of all de-select all the GPDMA Channels.

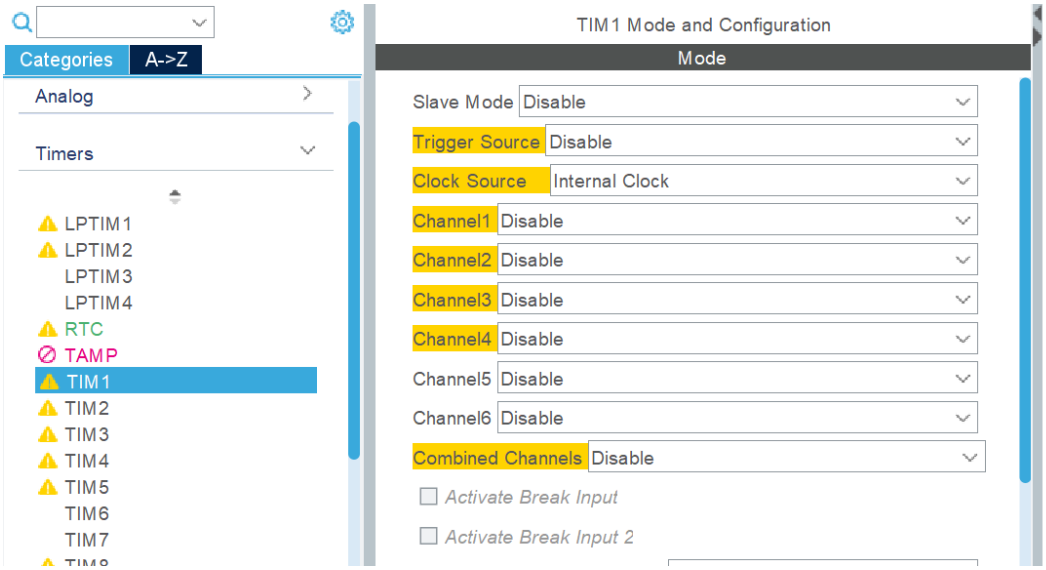
Figure 81. **GPDMA1 de-select**



In the GPIO section leave everything as is. With respect to the MKBOXPRO, the STWINBX1 has every necessary pin already configured.

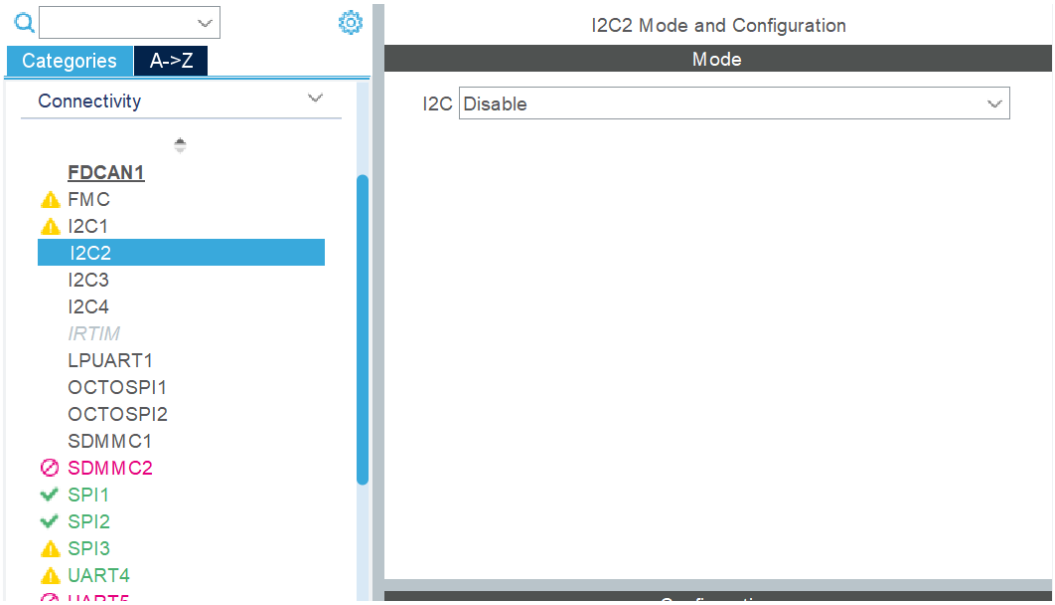
Activate the TIM1 Internal Clock Source and set the same parameters as for the path for MCU explained earlier.

Figure 82. **Internal Clock Source**



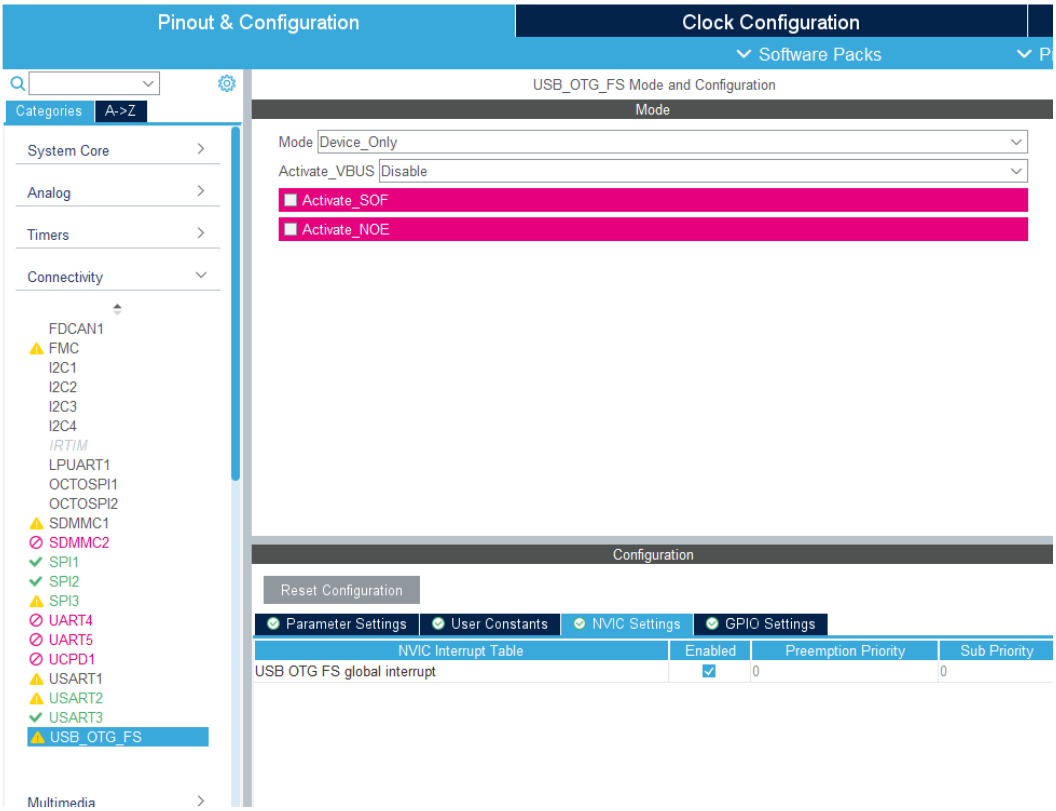
De-select all the I2Cs and the SDMMC1 from the Connectivity section.

Figure 83. **I2Cs and SDMMC1**



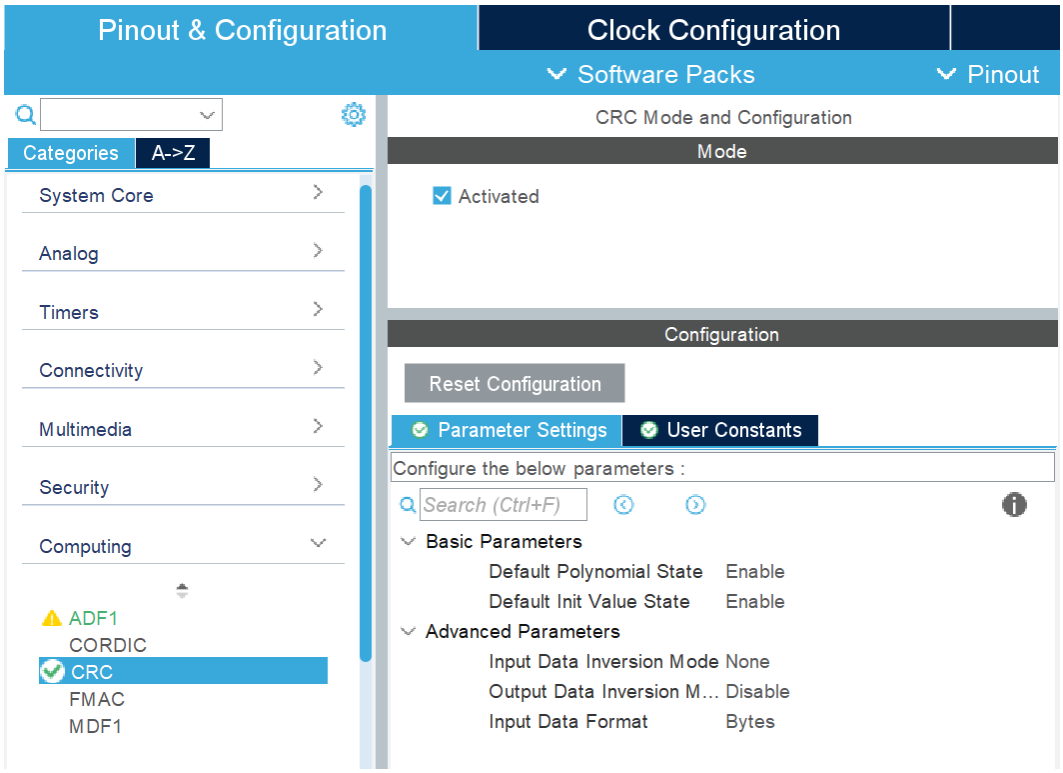
In the USB Configuration, under NVIC settings, enable the global interrupt.

Figure 84. **USB Configuration**



In the computing section select the CRC.

Figure 85. **CRC Selection**



Now its possible to select the Software Packs. This procedure is analogous to the one already done in the case starting from the MCU.

Everything now is ready for the project to be generated, thus click GENERATE CODE and go on with the selected IDEs.

The procedure is the same as before.

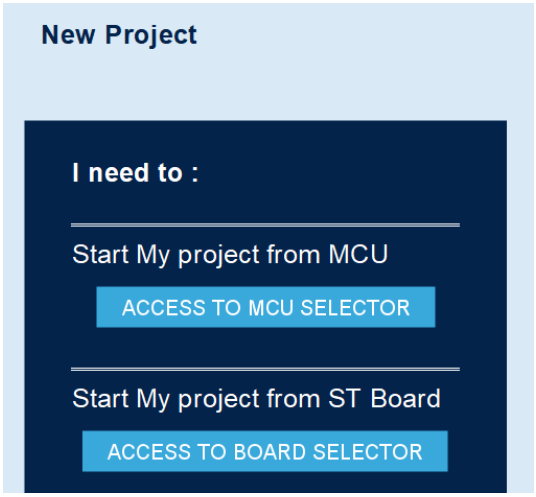
STEVAL – STWINKT1B (ai_inertial only)

This section outlines how to configure STM32CubeMX with the STEVAL-STWINKT1B development board starting directly from the STM32L4R9 micro-controller.

It's possible to start a project directly from the MCU or choosing a specific target board.

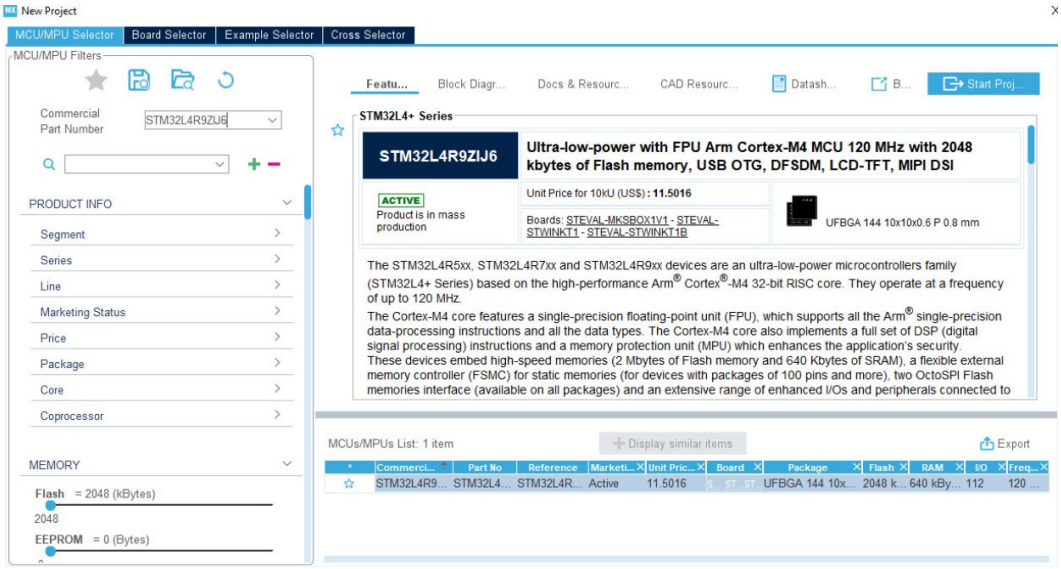
Let's start with the MCU, later we will address the access to the board selector

Figure 86. **Starting point: MCU or BOARD selection**



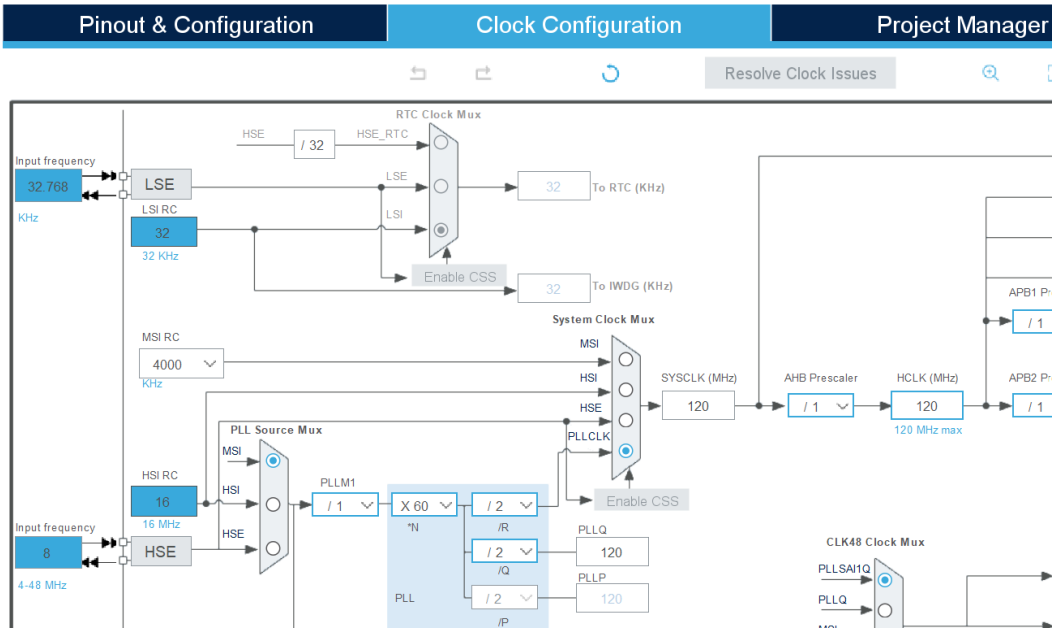
Create a new project for the L4R9 MCU.

Figure 87. Starting point: Micro-controller selection



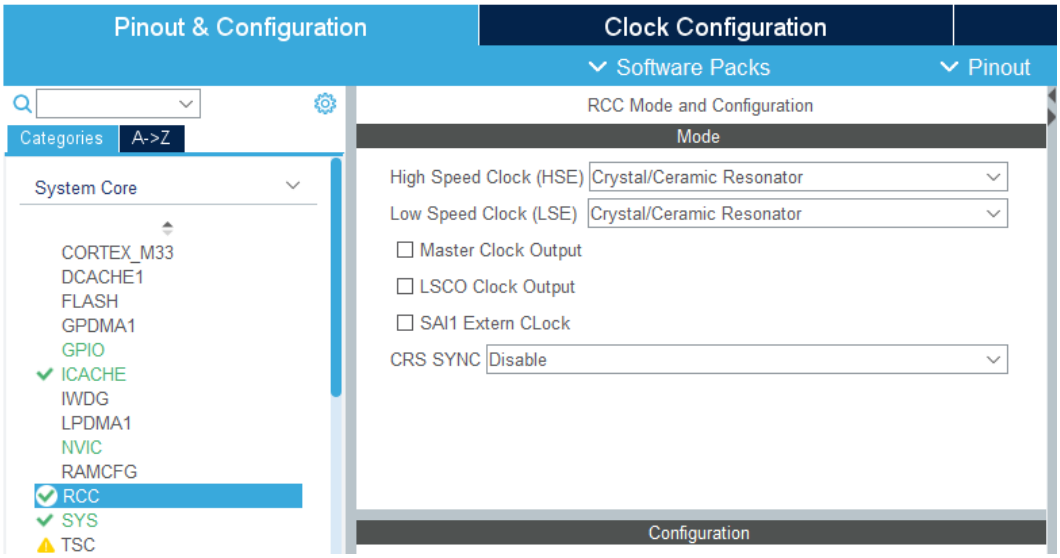
Set the clock **HCLK** to **120MHz** to utilize the maximum frequency.

Figure 88. Clock Frequency selection



In the **RCC** section set the HSE and the LSE to **Crystal/Ceramic Resonator**.

Figure 89. **HSE and LSE clock selection**



GPIO Configuration

Set the **PE8** pin to **GPIO_EXTI8, PF4** to **GPIO_EXTI4** and **PF13** pin to **GPIO_Output**, then going to **NVIC** section enable the interrupt pins. The reason for the use of these pins is explained in the picture below. The PE8 for example is linked to the INT1_DHC pin, which is excited when the FIFO of the ISM330DHCX is full of samples. While the PF4 is connected to the IN2_DHC, excited when the MLC changes result. For this reason is important to activate the interrupt pin of the sensor and link it to the MCU pin.

Figure 90. **GPIO settings**

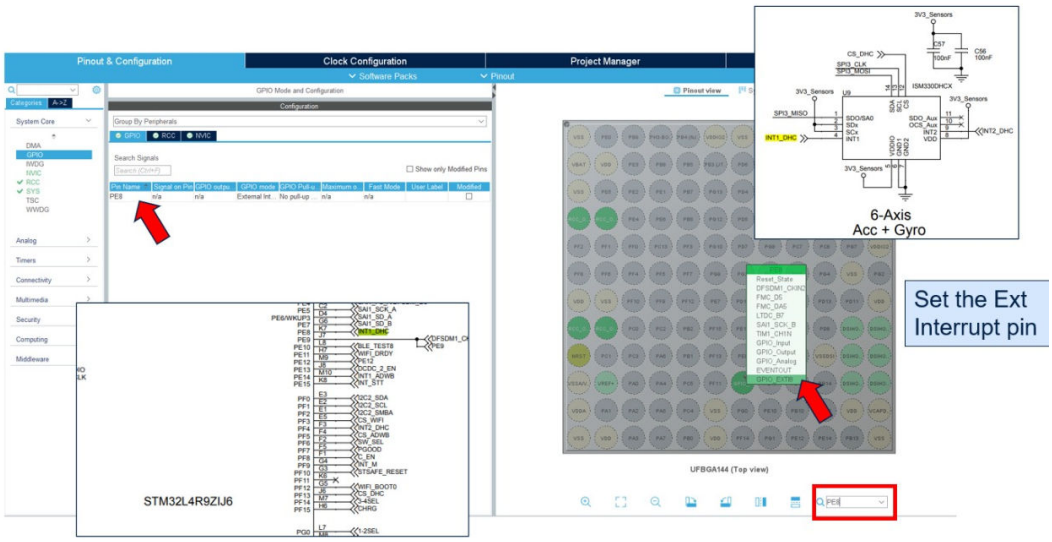


***Figure 91** GPIO settings*

Figure 91. **GPIO settings**

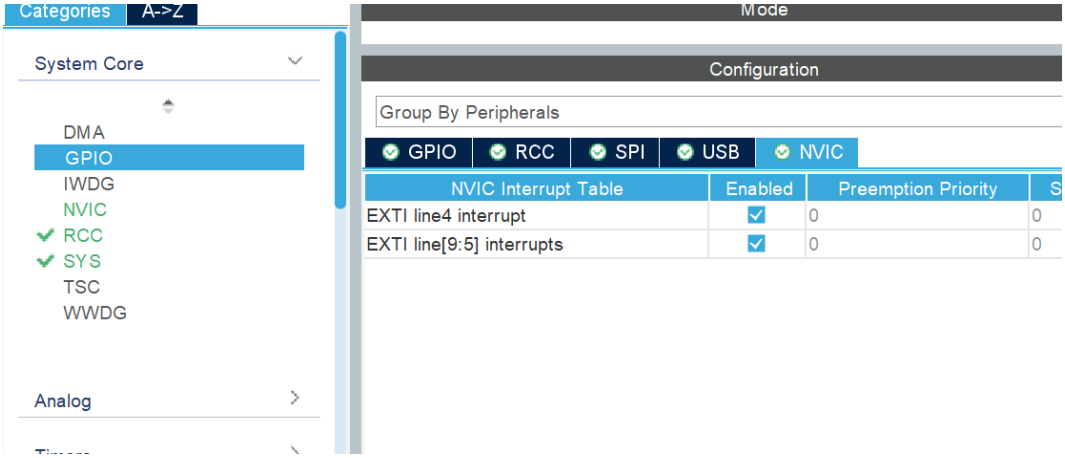


***Figure 92** GPIO settings*

Moreover set the **PF13**, which is the CS (chip select) of the ISM330DHCX sensor, to **High**. This is needed to disable the CS pin, that will be turned on once the SPI3 connection starts.

Figure 92. **GPIO settings**

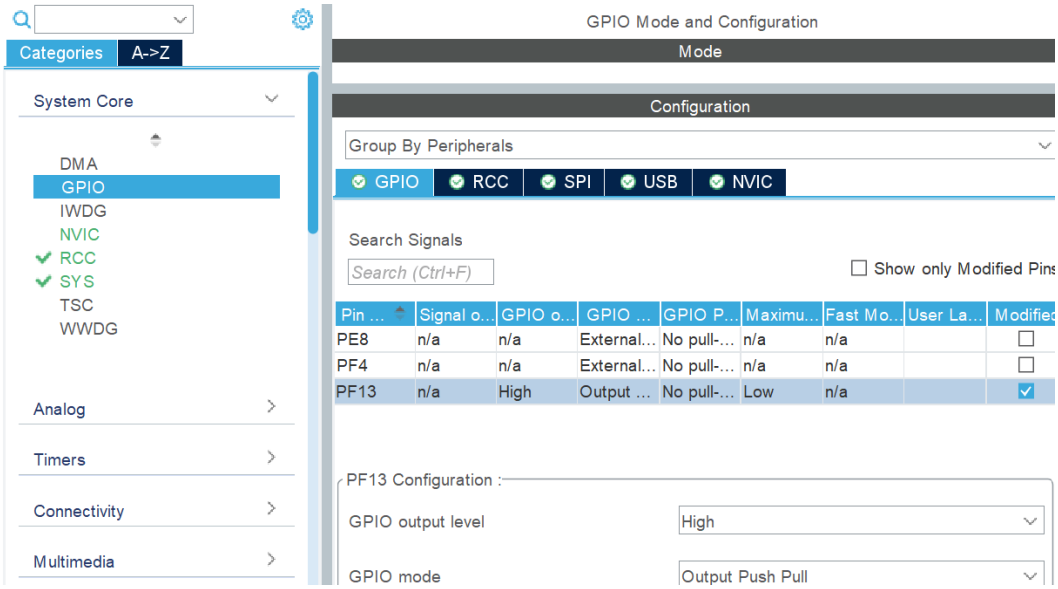


***Figure 93** GPIO settings*

Timers

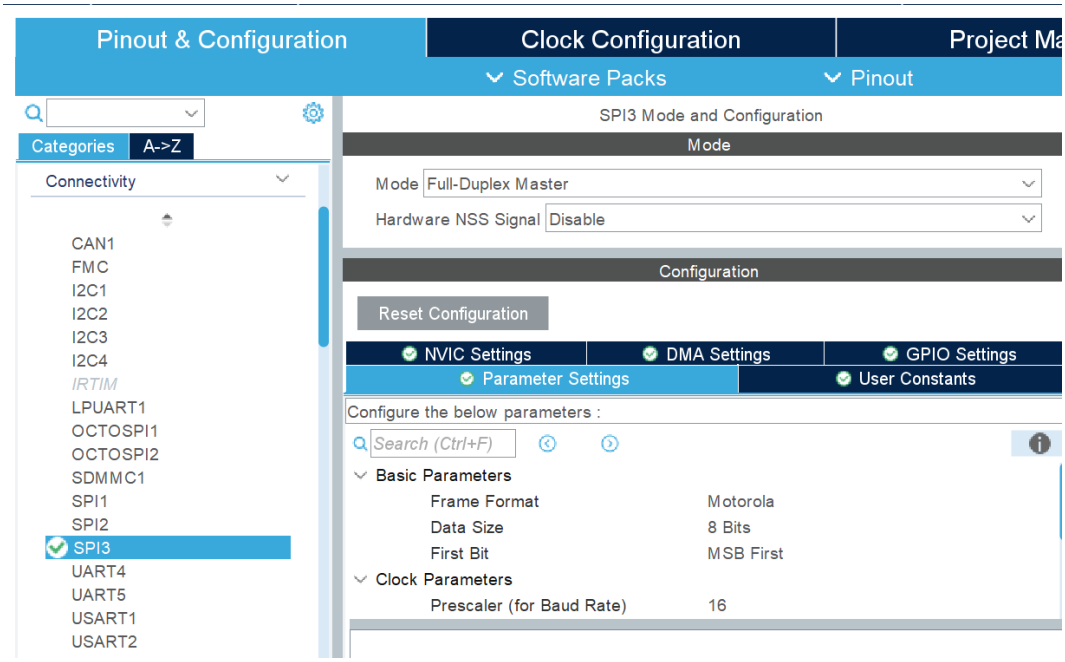
In this section select Tim1, Clock Source: Internal Clock, then in Parameter Settings set the Prescaler to 2400 and the Period to 65534. Then check the TIM1 update interrupt in the NVIC settings. This to obtain an interrupt every 1s, since the system clock frequency is 160MHz and to obtain almost the 1 Hz timer frequency the product between Prescaler and period must be equal to the system frequency.

Connectivity Configuration

SPI3

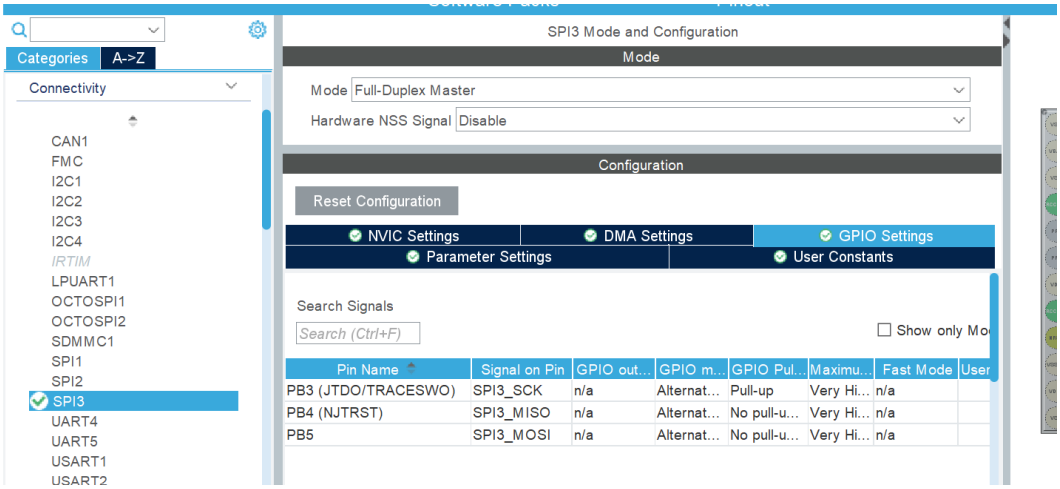
Under the **SPI3** section set the Mode to **Full Duplex Master**. Then in the **Parameter Settings**: set Data Size to 8 Bits, change the Prescaler such that the Baud Rate is lower or equal to 10.0 MBits/s, then choose CPOL High and CPHA 2 Edge. All these settings are standard and are needed to work with the **X-CUBE-MEMS1** pack, with the CUSTOM APIs, more information about this can be retrieved on the X-CUBE-MEMS1 manual (Chapter 7.3, page 64).

Figure 93. **SPI3 selection - Parameter settings**



In the **GPIO Settings** of the SPI3, change the pins such that **PB4** is the MISO, **PB3** the SCK and **PB5** the MOSI signals. This because the ISM330DHCX sensor is connected to the SPI bus through these pins. Moreover **Pull up** the **SPI3_SCK** pin as reported in the manual discussed earlier.

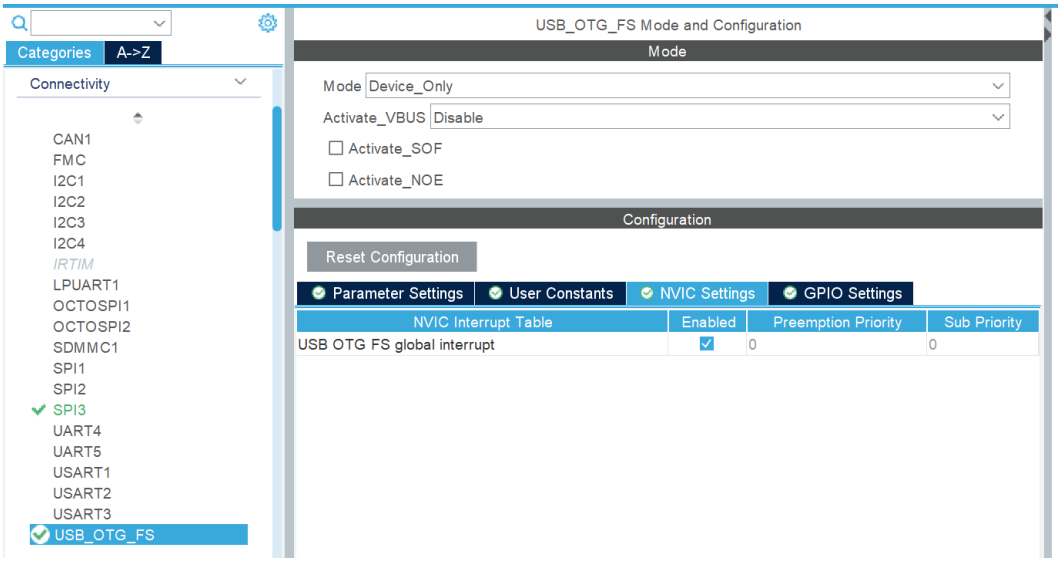
Figure 94. **SPI3 selection - GPIO settings**



USB

Enable the **USB_OTG_FS** and select the **Device Only** Mode. Moreover enable the **global interrupt** under the NVIC Settings, the reason for this is to exploit the USB functionality at lower level. The interrupt is needed to receive and transmit data using the USB.

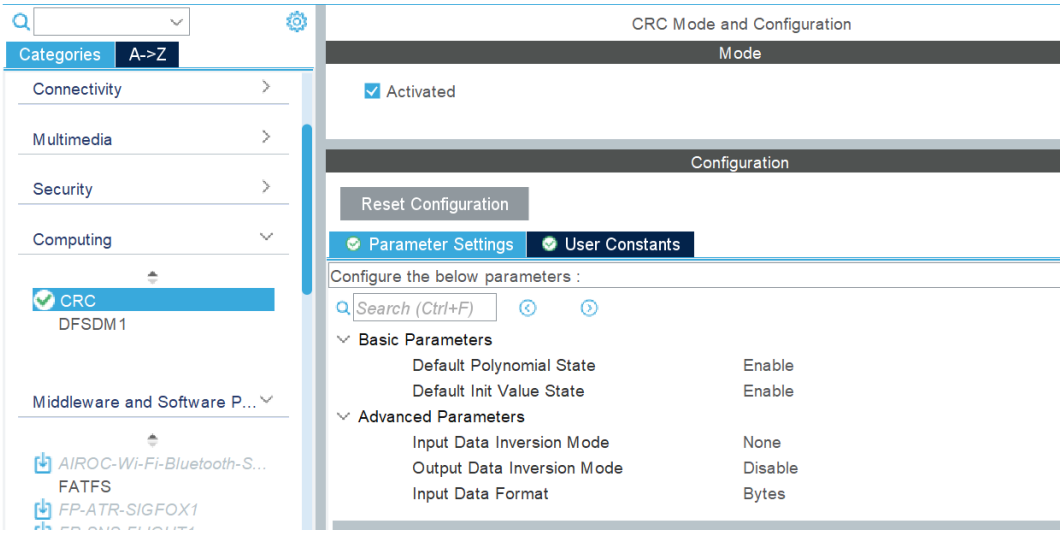
Figure 95. **USB selection**



CRC

The **CRC** must be activated to work with Neural Networks. Overall, using it can help ensuring the accuracy and reliability of trained models.

Figure 96. **CRC selection**



Software pack selection

In this part 3 software packs must be selected:

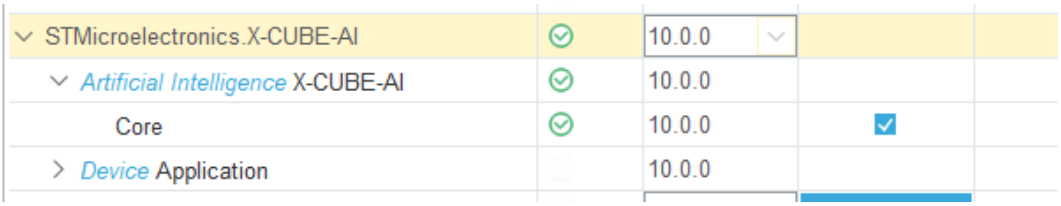
- **X-CUBE-AI**
- **X-CUBE-MEMS1**
- **FP-SNS-STAIOTCFT**

The first two are needed to work with the Function Pack FP-SNS-STAIOTCFT.

Select “Software Packs” and then “Select Components”. From the “Software Packs Component Selector” window, the user has to select all the needed packs. In our case the ones above mentioned.

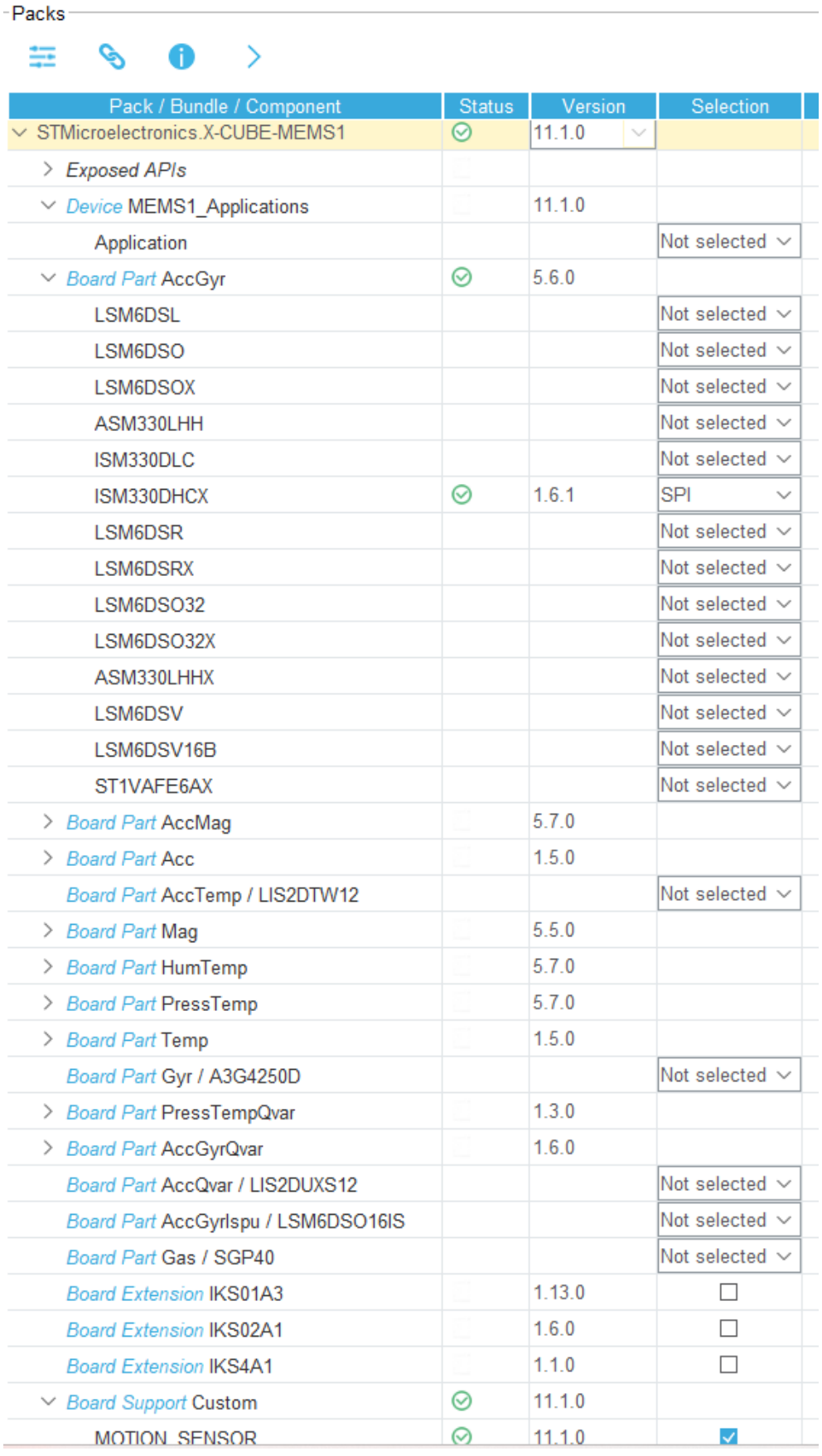
Under the X-CUBE-AI pack select only the **Core**.

Figure 97. **X-CUBE-AI pack selection**



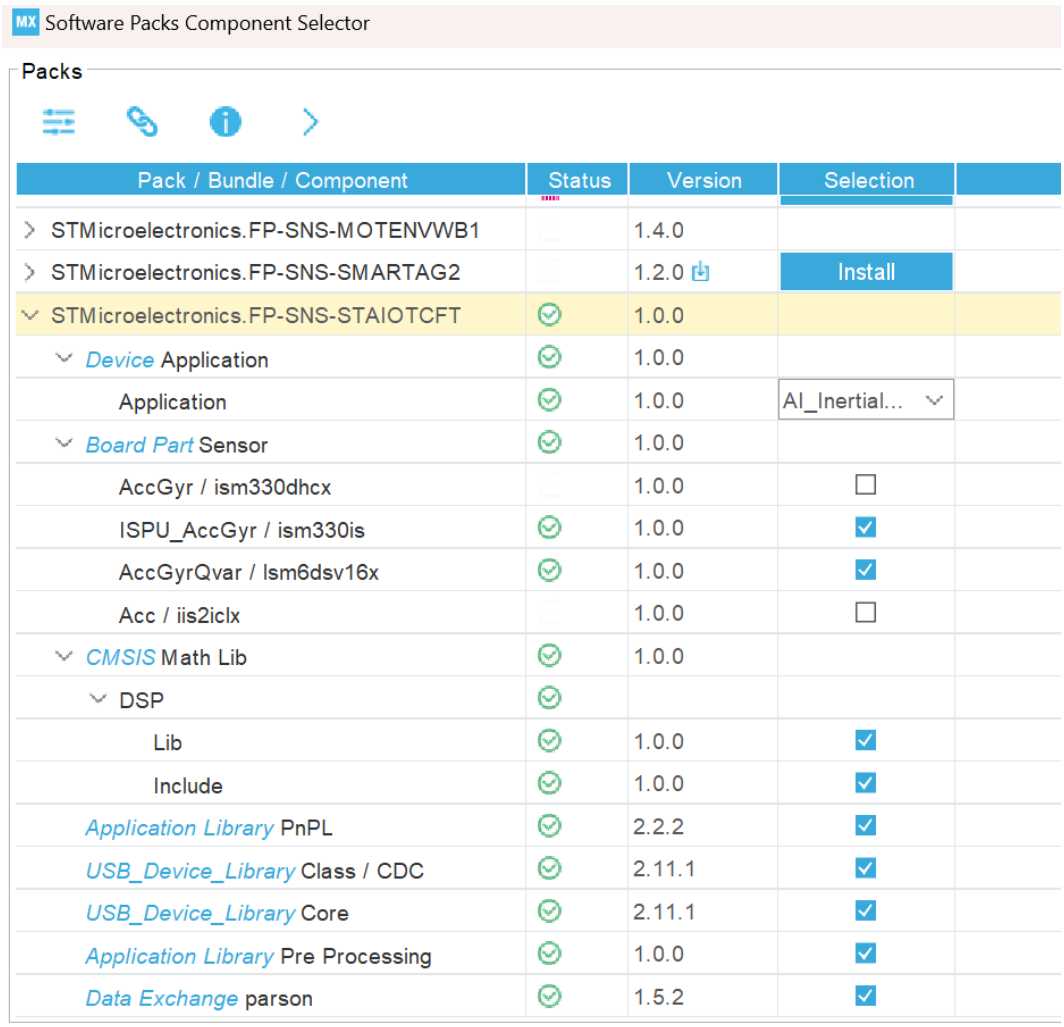
Under the X-CUBE-MEMS1 pack select the **SPI** for the ISM330DHCX sensor and choose **MOTION_SENSOR** for the Board Support Custom.

Figure 98. **X-CUBE-MEMS1 pack selection**



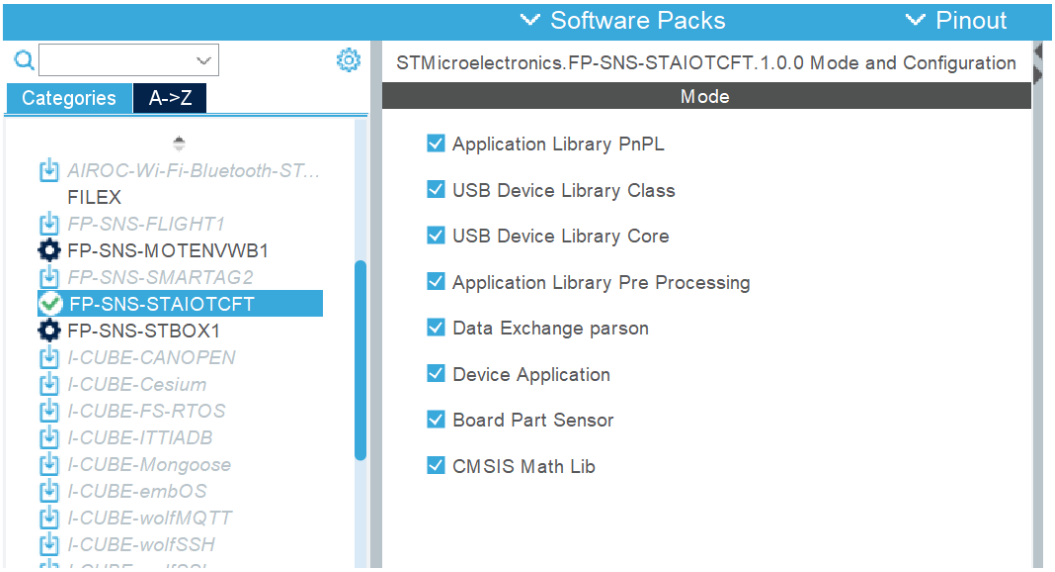
For the FP-SNS-STAIOTCFT select the Application, the sensors required and all the libraries (those include support for the PnPL, pre-processing, AI and USB). The required selections for the STWINKT1B are provided here below.

Figure 99. FP-SNS-STAIOTCFT pack selection



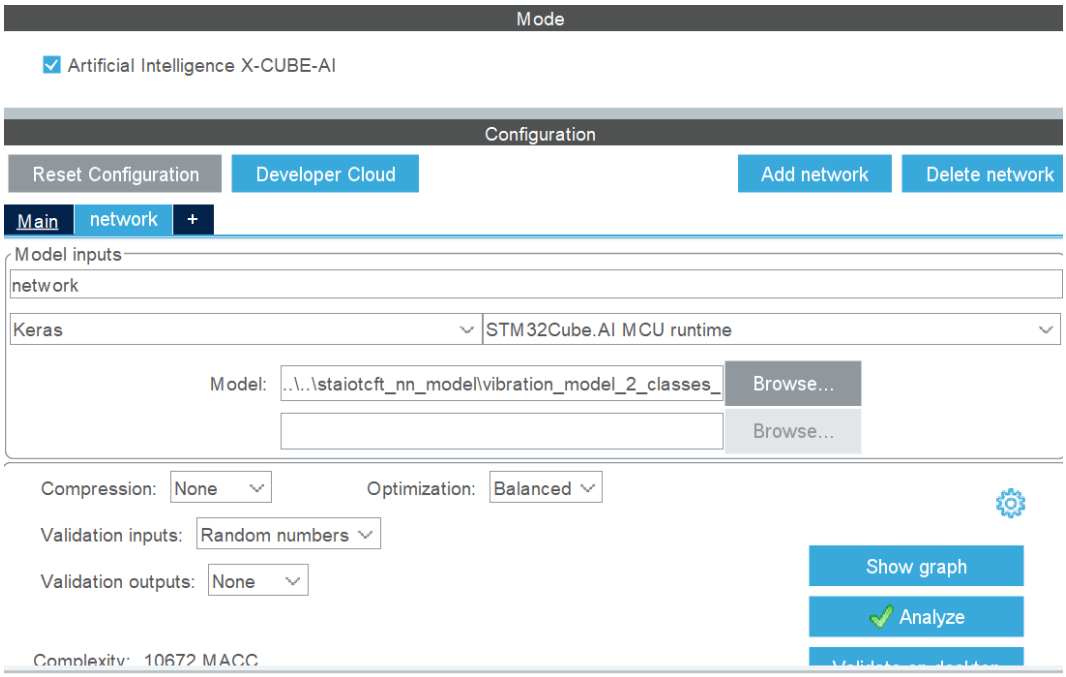
Then the packs must be activated enabling the check boxes.

Figure 100. FP-SNS-STAIOTCFT pack selection



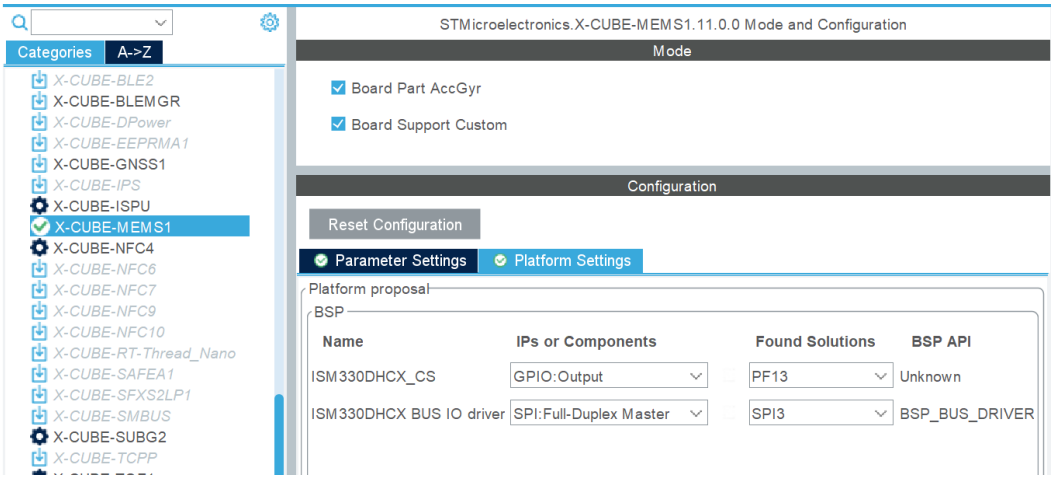
In the X-CUBE-AI section, add the network through the **h5** file (present in the repository under the Projects section in "*staiotcft_nn_model*") and click **Analyze**.

Figure 101. X-CUBE-AI pack selection



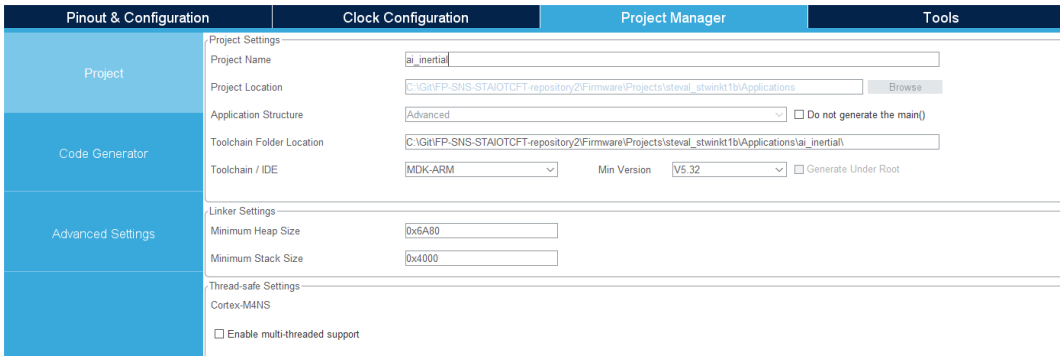
In the Platform Settings of the X-CUBE-MEMS1 connect the sensor to the MCU pins initialized at the beginning (in the GPIO Configuration).

Figure 102. X-CUBE-MEMS1 pack selection



Finally, in the **Project Manager** section, choose Project Name and Location, Toolchain/IDE STM32CubeIDE (Generate Under Root not selected) or EWARM or MDK-ARM. Check the Linker settings as the configuration of the Heap and Stack in the picture below, as well as the Thread-safe settings and the Mcu and Firmware Package. Then click **GENERATE CODE**.

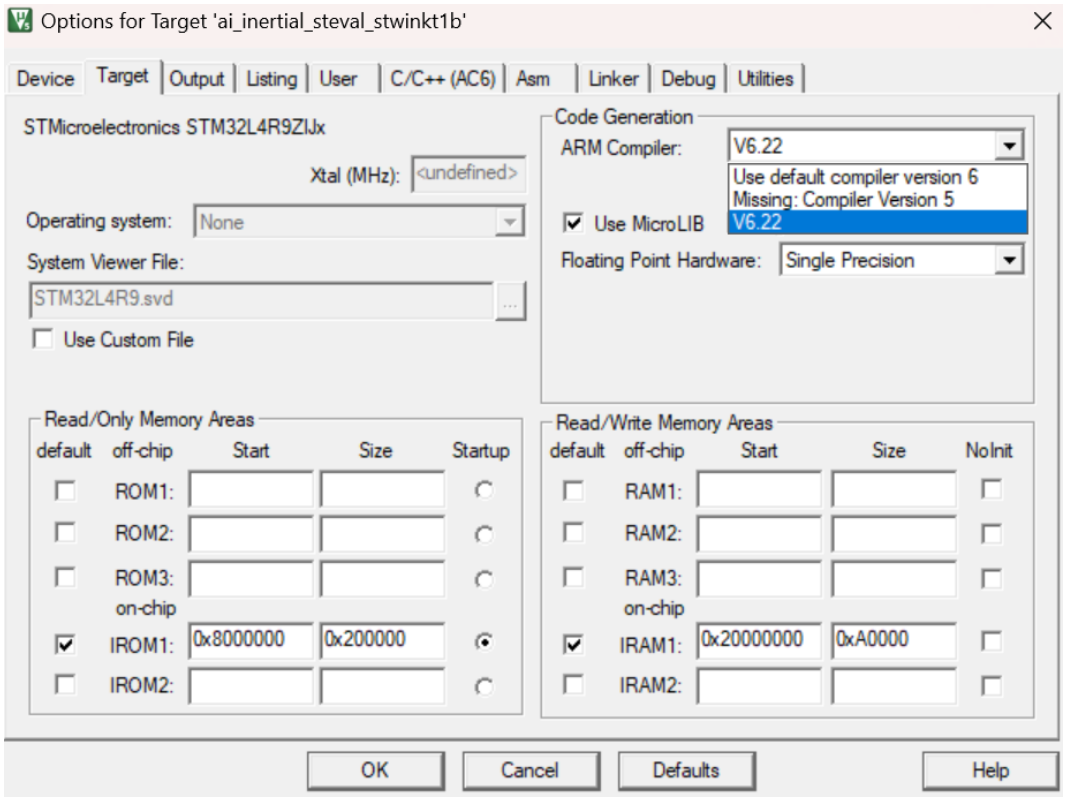
Figure 103. Project generation



The rest of the steps are exactly the same as for the STEVAL-MKBOXPRO and STEVAL-STWINBX1 development boards.

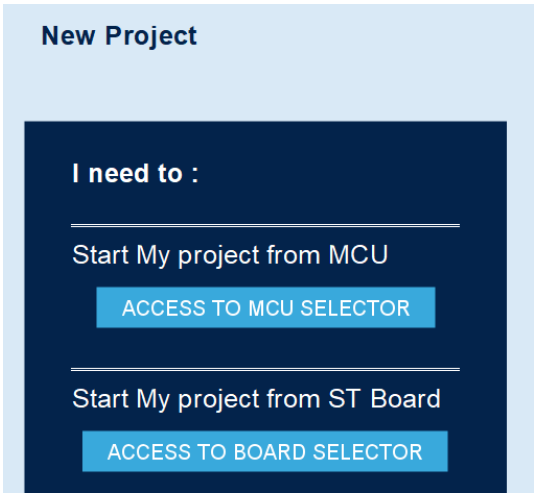
Note: In the MDK-ARM (KEIL) development environment a warning message will appear, this is due to the incompatibility between the compiler version and the generated code from STM32CubeMX for the L4 micro.

Figure 104. **Project generation**



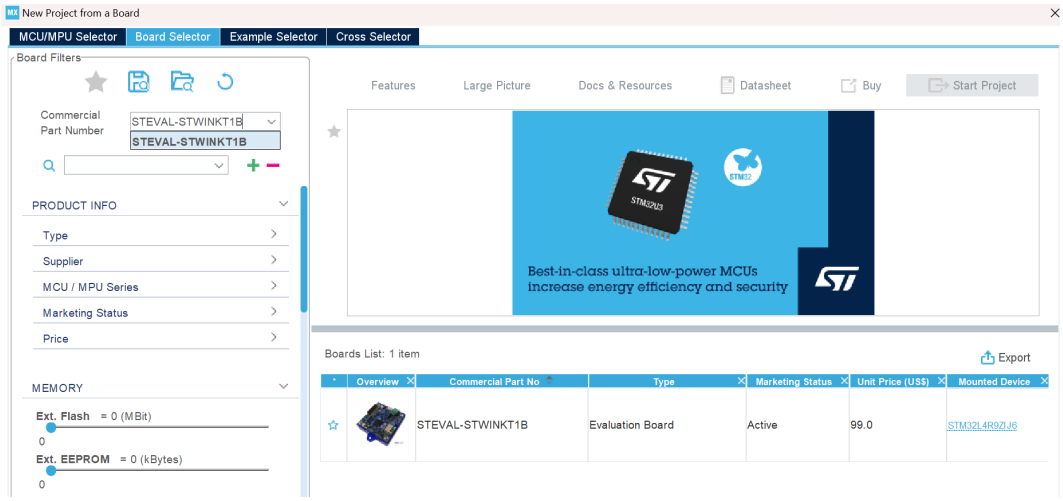
The access through the board selector is a bit different with respect to the one from the MCU selector. First of all start clicking on “Access to board selector”.

Figure 105. **Starting point: Board selection**



Type STWINKT1B in the Commercial Part Number tab and select the intended Evaluation board from the Board List.

Figure 106. Starting point: Board selection

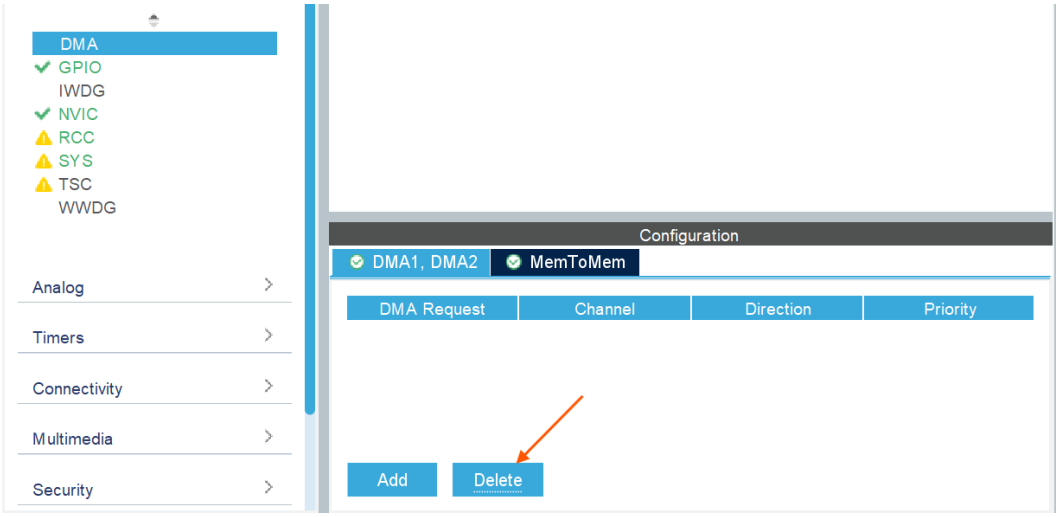


Click “Start Project” and “Yes” to Initialize all the peripherals in Default Mode.

The project will be pre-configured, but there are some alignments that the user has to make to avoid unintended mis-uses of the FP-SNS-STAIOTCFT pack.

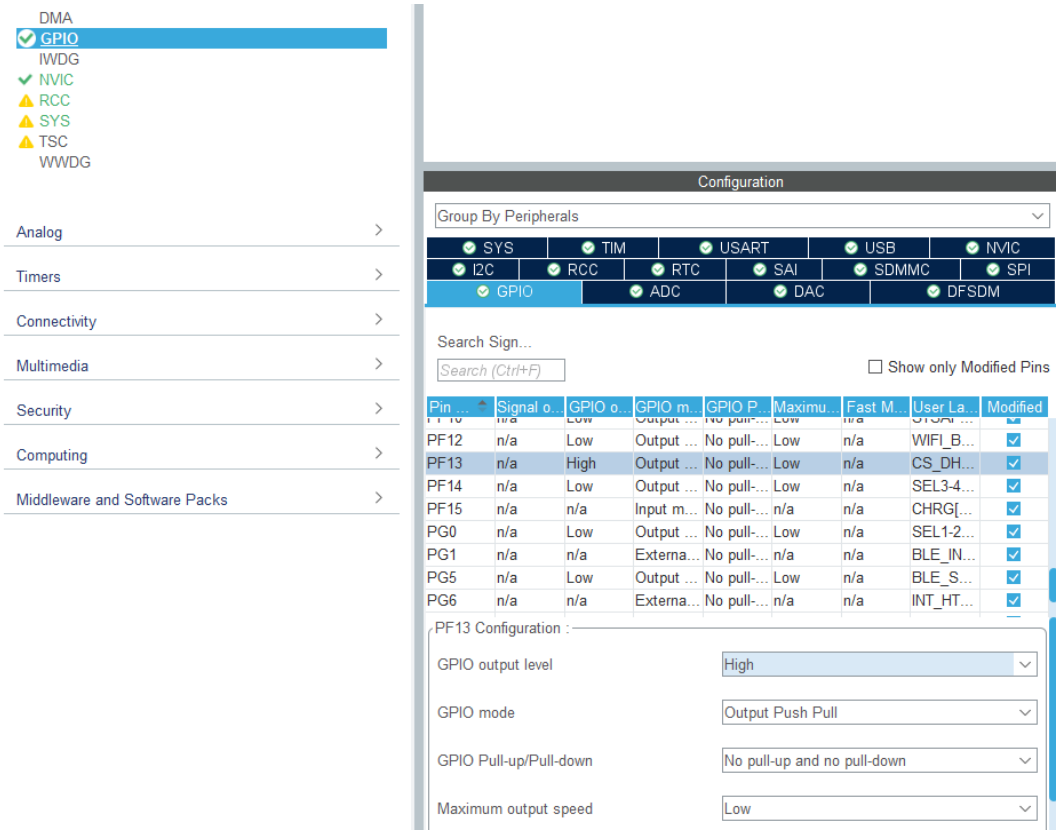
First of all de-select the DMA Channels.

Figure 107. DMA de-select



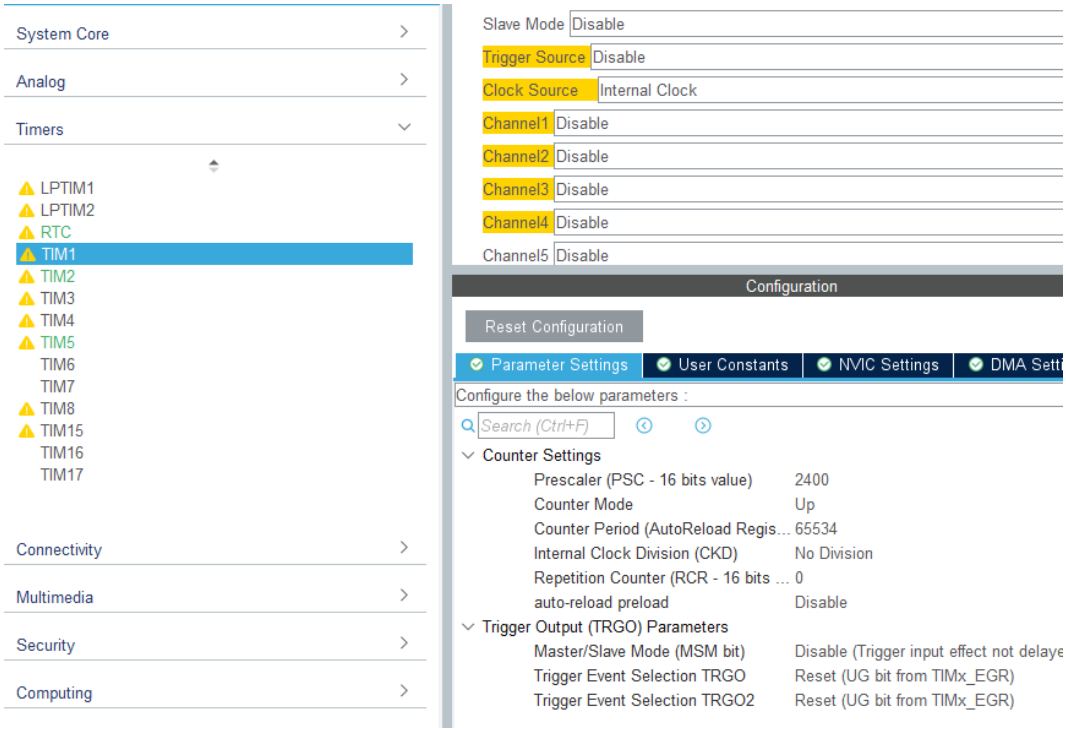
In the GPIO section change only the PF13 GPIO output level to High.

Figure 108. GPIO select



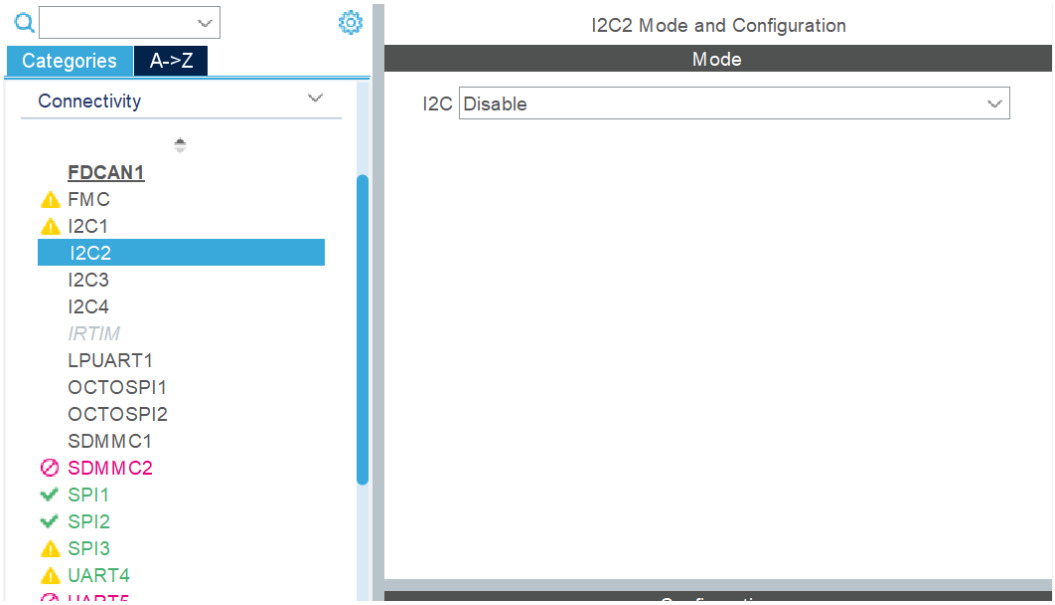
Activate the TIM1 Internal Clock Source and set the same parameters as for the path for MCU explained earlier.

Figure 109. Internal Clock Source



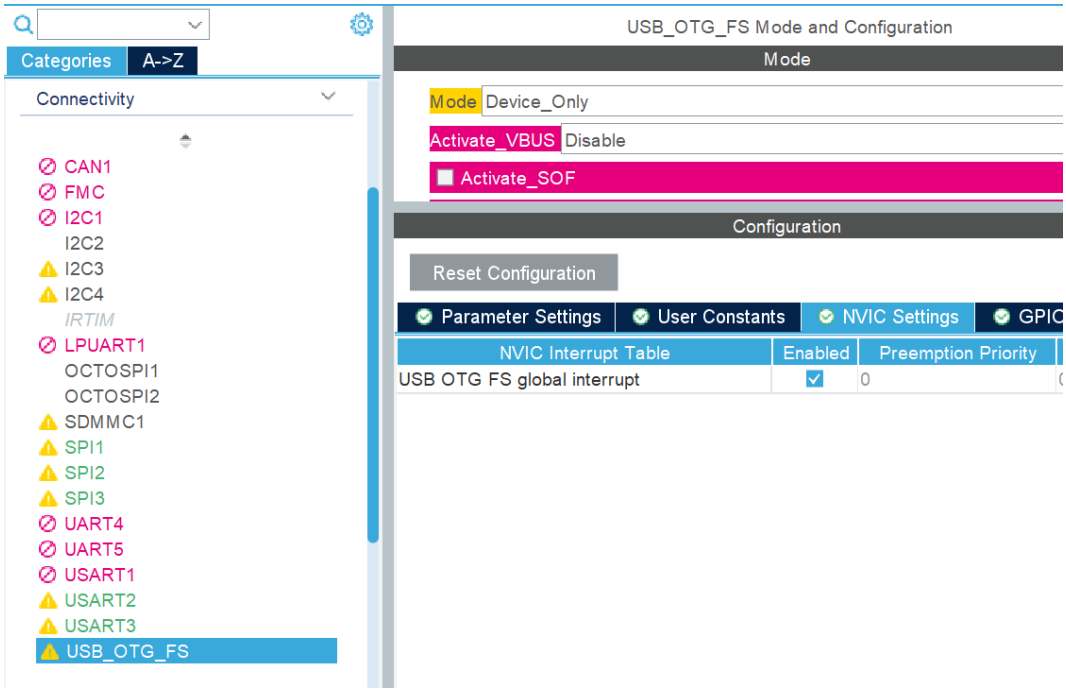
De-select all the I2Cs and the SDMMC1 from the Connectivity section.

Figure 110. **I2Cs and SDMMC1**



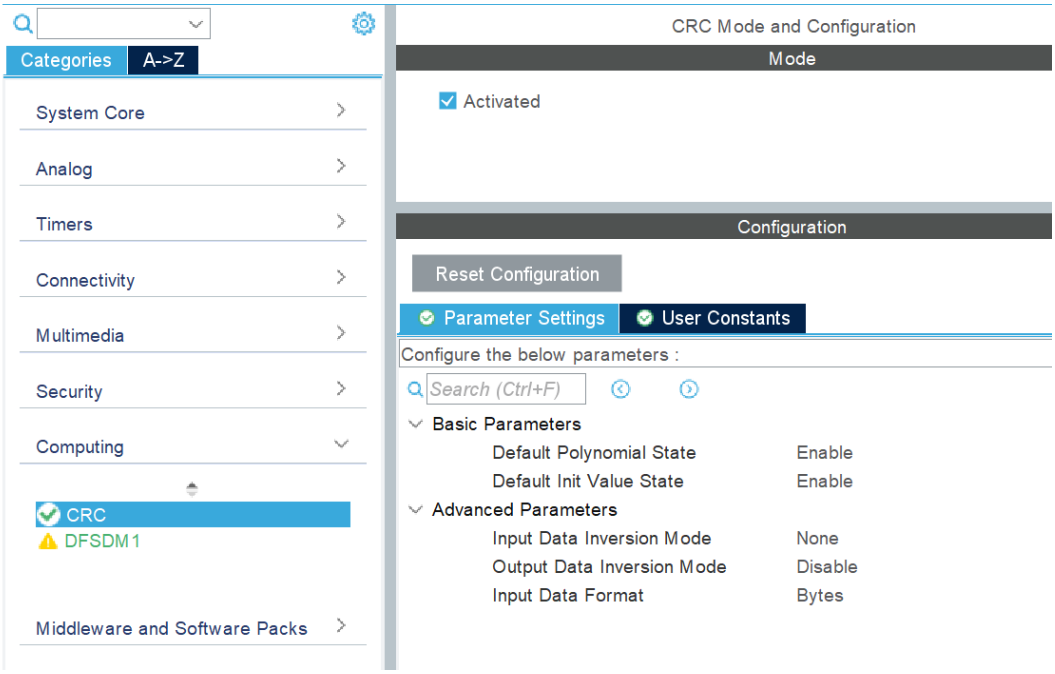
In the USB Configuration, under NVIC settings, enable the global interrupt.

Figure 111. **USB Configuration**



In the computing section select the CRC.

Figure 112. **CRC Selection**



Now its possible to select the Software Packs. This procedure is analogous to the one already done in the case starting from the MCU.

Everything now is ready for the project to be generated, thus click GENERATE CODE and go on with the selected IDEs.

The procedure is the same as before.

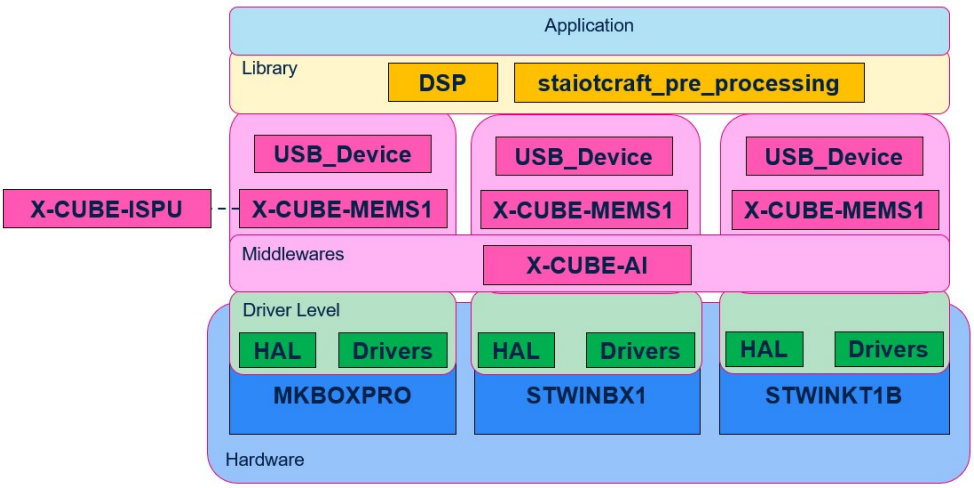
1.17 Application ai_inertial

How does the firmware work?

As mentioned earlier the generated firmware is embedding all the inference modes and functionalities and it's different only from a board perspective. This allows us to focus on one of the development boards, for example the STEVAL-MKBOXPRO. From an application level view point the firmware structure is very simple.

An explanatory image can be found below.

Figure 113. **Firmware structure**



The structure can be subdivided into different conceptual levels:

- **Application**
- **Middlewares and Libraries**
- **Sensor Drivers**
- **Development Boards**

The applicative part is common to all the boards and is crucial for allowing the user to interact with the other firmware levels.

It wraps commands and general functions of the firmware levels below. It provides the user with the capability to communicate via USB using PnPL messages.

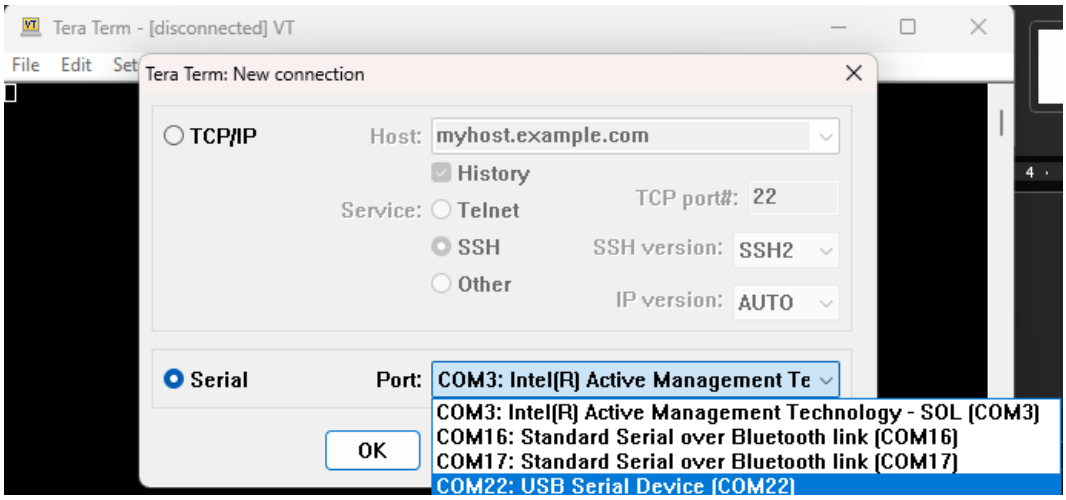
It adopts functions to interact with inertial sensors thanks to the X-CUBE-MEMS1 pack and X-CUBE-ISPU (excluding the STEVAL-STWINKT1B board).

It runs AI inference on the MCU using the X-CUBE-AI pack.

How to interact with the firmware?

After flashing the firmware onto the chosen target (MKBOXPRO, STWINBX1, or STWINKT1B), a reboot is necessary to interact with the firmware. To do this, either turn the board off and on again or unplug and plug the USB cable. Once the board has rebooted, open a terminal application, such as Tera Term, to view the logs and interact with the firmware.

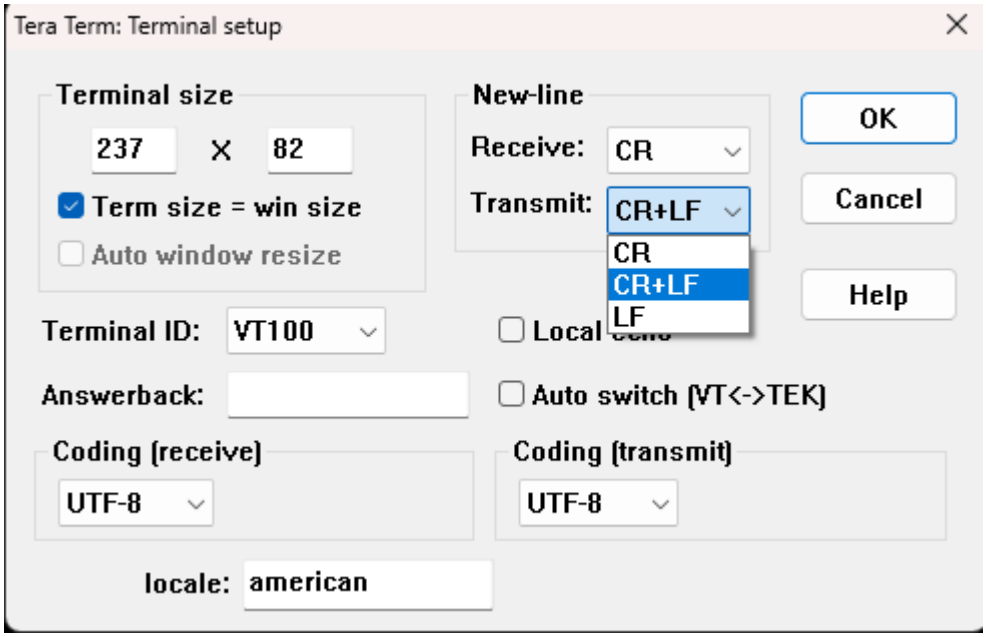
Figure 114. Teraterm: new connection



The first message that appears will help the user providing the list of possible commands and properties.

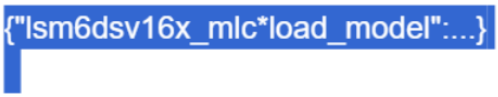
*Note: To send commands to the firmware and trigger any application, you must enable the CR+LF option in Tera Term. To do this, go to **Setup -> Terminal -> Transmit** and enable the CR+LF option. Additionally, ensure that a new-line character is appended after every command. This can be achieved by simply copying the command along with the new line (see the picture below for reference).*

Figure 115. **Teraterm: terminal setup**



Drag and drop the cursor to ensure that the entire command, including the new-line character, is highlighted.

Figure 116. **Command**



Depending on the chosen target, the displayed commands may vary due to the presence of different sensors. For example, in the picture above, the chosen board is the STEVAL-MKBOXPRO, which can be inferred from the clear reference to the LSM6DSV16X inertial sensor in the help message. The help message displays a set of possible commands depending on the chosen application.

In this case, the firmware can start inferring the Neural Network running on the MCU by choosing the "start_inference" command or stop it with the "stop_inference" command. The same applies to the inference of the Machine Learning Core (and if selected, also the ISPU) on the inertial sensor. In the terminal, the firmware starts outputting the results of a classification for an example application, the chosen Smart Asset Tracking, with the labels referenced as follows:

- **0 – Stationary Upright** (board on the table with the ST logo facing up)
- **4 – Stationary Not Upright** (board on the table facing down)
- **8 – Moving** (board lightly moving)
- **12 – Shaken** (board shaken)

Moreover, if a user wants to change the inference application running on the MLC, a "*load_model*" command can be triggered. It is important to respect the different fields of the PnPL message: filename, size (of the content), and content.

The content field must be filled with the text containing the compressed model.

Note: the number of characters in the content field is counted, and will represent the size.

As a reference, here is a list of possible *load_model* commands that can be used for the STEVAL-MKBOXPRO:

LSM6DSV16X

6D Position

```
{"lsm6dsv16x_mlc*load_model":{"arguments":{"filename": "6dposition_model ", "size":485, "cont
ent":" 10001100018004000500174002110 8EA097809030982090309000900090A09010912021108FA095C090309
8E0903099A09030231085C093F090009000984090009000900098809000900090098C090009000900090409000900
0090009080900090009000900090C09000900091F09000231088E090009000900090009000900090009000900090
0100018017400231089A09CD09340905098009CD09340903098109CD0934095609E509CD0934090009A209CD09340
93409E409CD0934090009A209CD0934090009A109CD0934091209E30180170004000510020101005E0201800D0160
1545020100150017001004110 0"}}}
```


Vibration Monitoring


```
{"lsm6dsv16x_mlc*load_model":{"arguments":{"filename":"vibration_monitoring_model","size":316
,"content":"10001100018004000500174002110 8EA096609030970090309000900090A09010912021108FA095C0
90309720903097E09030231085C093F090009030994090009FC0900097C091F090002310872090009000900090009
0009000900090009000900090001000180174002310 87E09AE0927091009C00900093E092109E0018017000400051
0020101005E0201800D0160154502010015001701100 41100"}}}
```


Note: If you try this commands remember to attach the new line character as explained above.

More specifically, here is the list of possible commands for the MKBOXPRO:

```
/* PnPL GET Messages List ------------------------------------------------- */
{"get_status": "all"}
{"get_status": "lsm6dsv16x_acc"}
{"get_status": "lsm6dsv16x_gyro"}
{"get_status": "inference_mcu"}
{"get_status": "lsm6dsv16x_mlc"}
{"get_status": "ism330is_ispu"}
{"get_status": "controller"}
{"get_status": "firmware_info"}
{"get_status": "DeviceInformation"}
/* ----------------------------------------------------------------------- */
/* PnPL SET Messages List ------------------------------------------------- */
{"firmware_info": {"alias": "string_value"}}
/* ----------------------------------------------------------------------- */
/* PnPL COMMAND Messages List --------------------------------------------- */
{"inference_mcu*start_inference": ""}
{"inference_mcu*stop_inference": ""}
{"lsm6dsv16x_mlc*start_inference": ""}
{"lsm6dsv16x_mlc*stop_inference": ""}
{"lsm6dsv16x_mlc*load_model": {"arguments": {"filename": "string_value", "size": integer_valu
e, "content": "string_value"}}}
{"ism330is_ispu*start_inference": ""}
{"ism330is_ispu*stop_inference": ""}
{"ism330is_ispu*load_model": {"arguments": {"filename": "string_value", "size": integer_value
, "content": "string_value"}}}
{"controller*set_dfu_mode": ""}
{"controller*switch_bank": ""}
/* ----------------------------------------------------------------------- */
/* PnPL TELEMETRY Messages List [FW --> Host] ----------------------------- */
{"inference_mcu": {"label_id": integer_value, "accuracy": float_value}}
{"lsm6dsv16x_mlc": {"label_id": integer_value}}
{"ism330is_ispu": {"label_id": integer_value}}
```

When considering another board, the commands remain exactly the same. The only parts that change are the names of the sensors with the MLC (Machine Learning Core) embedded inside and the microcontroller. This consistency ensures that the user experience and interaction with the firmware are uniform across different boards, simplifying the development and deployment process.

More specifically, here is the list of possible commands for the STWINBX1:

```
/* PnPL GET Messages List ------------------------------------------------- */
{"get_status": "all"}
{"get_status": "ism330dhcx_acc"}
{"get_status": "ism330dhcx_gyro"}
{"get_status": "iis2iclx_acc"}
{"get_status": "inference_mcu"}
{"get_status": "ism330dhcx_mlc"}
{"get_status": "iis2iclx_mlc"}
{"get_status": "ism330is_ispu"}
{"get_status": "controller"}
{"get_status": "firmware_info"}
{"get_status": "DeviceInformation"}
```

```
/* ---------------------------------------------------------------------- */

/* PnPL SET Messages List ------------------------------------------------ */
{"firmware_info": {"alias": "string_value"}}
/* ---------------------------------------------------------------------- */

/* PnPL COMMAND Messages List -------------------------------------------- */
{"inference_mcu*start_inference": ""}
{"inference_mcu*stop_inference": ""}
{"ism330dhcx_mlc*start_inference": ""}
{"ism330dhcx_mlc*stop_inference": ""}
{"ism330dhcx_mlc*load_model": {"arguments": {"filename": "string_value", "size": integer_valu
e, "content": "string_value"}}}
{"iis2iclx_mlc*start_inference": ""}
{"iis2iclx_mlc*stop_inference": ""}
{"iis2iclx_mlc*load_model": {"arguments": {"filename": "string_value", "size": integer_value,
 "content": "string_value"}}}
{"ism330is_ispu*start_inference": ""}
{"ism330is_ispu*stop_inference": ""}
{"ism330is_ispu*load_model": {"arguments": {"filename": "string_value", "size": integer_value
, "content": "string_value"}}}
{"controller*set_dfu_mode": ""}
{"controller*switch_bank": ""}
/* ---------------------------------------------------------------------- */

/* PnPL TELEMETRY Messages List [FW --> Host] ---------------------------- */
{"inference_mcu": {"label_id": integer_value, "accuracy": float_value}}
{"ism330dhcx_mlc": {"label_id": integer_value}}
{"iis2iclx_mlc": {"label_id": integer_value}}
{"ism330is_ispu": {"label_id": integer_value}}
```

And STWINKT1B:

```
/* PnPL GET Messages List ------------------------------------------------ */
{"get_status": "all"}
{"get_status": "ism330dhcx_acc"}
{"get_status": "ism330dhcx_gyro"}
{"get_status": "inference_mcu"}
{"get_status": "ism330dhcx_mlc"}
{"get_status": "controller"}
{"get_status": "firmware_info"}
{"get_status": "DeviceInformation"}
/* ---------------------------------------------------------------------- */

/* PnPL SET Messages List ------------------------------------------------ */
{"firmware_info": {"alias": "string_value"}}
/* ---------------------------------------------------------------------- */

/* PnPL COMMAND Messages List -------------------------------------------- */
{"inference_mcu*start_inference": ""}
{"inference_mcu*stop_inference": ""}
{"ism330dhcx_mlc*start_inference": ""}
{"ism330dhcx_mlc*stop_inference": ""}
{"ism330dhcx_mlc*load_model": {"arguments": {"filename": "string_value", "size": integer_valu
e, "content": "string_value"}}}
{"controller*set_dfu_mode": ""}
{"controller*switch_bank": ""}
/* ---------------------------------------------------------------------- */

/* PnPL TELEMETRY Messages List [FW --> Host] ---------------------------- */
{"inference_mcu": {"label_id": integer_value, "accuracy": float_value}}
{"ism330dhcx_mlc": {"label_id": integer_value}}
```

2 System setup guide

2.1 Hardware description

2.1.1 STM32U5 Series MCU

The STM32U5 series offers advanced power-saving microcontrollers, based on Arm Cortex-M33 to meet the most demanding power/performance requirements for smart applications, including wearables, HMI, personal medical devices, home automation, and industrial sensors.

Offering up to 4 MB of flash memory (dual bank) memory and 3 MB of SRAM, the STM32U5 series of microcontrollers takes performance to the next level with advanced graphics capabilities.

The STM32U5 series provides greater integration and memory capabilities to maximize performance. Ultralow power, it is the first 32-bit MCU embedding a flash memory, up to 4 MB.

2.1.2 STEVAL-MKBOXPRO

The STEVAL-MKBOXPRO (SensorTile.box PRO) is the new ready-to-use programmable wireless box kit for developing any IoT application based on remote data gathering and evaluation, exploit the full kit potential by leveraging both motion and environmental data sensing, along with a digital microphone, and enhance the connectivity and smartness of whatever environment you find yourself into.

Figure 117. STEVAL-MKBOXPRO

You can entirely enjoy the SensorTile.box PRO experience regardless of your level of expertise, the box kit could be exploited according to three different modalities:

Entry mode: run a wide range of already embedded IoT applications on your box.

You can download the free STBLESensor app on your smartphone and immediately begin commanding the board with any of the following applications that have been specifically designed to work with the board sensors:

1. Motion: Compass, free-fall detection, level, pedometer, sensor-fusion - quaternion
2. Environmental: Barometer
3. Log: Data recorder
4. AI and MLC: Baby crying detector, human activity recognition
5. User interface: Qtouch
6. Connectivity: NFC tag

Expert mode: build custom applications through the STBLESensor app by selecting specific input data and operating parameters from corresponding available in-box sensors, functions to assess/compute those data, and output types that you need, while leveraging on the available powerful algorithms.

Pro mode: develop quickly your own tailored IoT application taking advantage of STM32 open development environment (ODE) and ST function pack libraries, including sensing AI function pack with neural network libraries, without the need to perform any coding activity.

The SensorTile.box PRO board fits into a small plastic box with a long-life 480 mAh rechargeable battery, now also leveraging a wireless charger and a programmable NFC tag. The board can be easily connected via Bluetooth to the STBLESensor app on your smartphone, from which the box kit can be enjoyed in Entry and Expert mode. In Pro mode, professional users can exploit the firmware programming and debugging interface in the STM32 ODE for developing their firmware from scratch.

All features

- All-in-one sensor node, which fits into the palm of your hand (40x63 mm) for any IoT applications
- Ready-to-go development kit
- Ultralow-power with FPU Arm Cortex-M33 with TrustZone® microcontroller (STM32U585AI)
- microSD card slot for standalone data logging applications
- On-board Bluetooth® LE 5.2 (BlueNRG-LP) and NFC tag (ST25DV04K)
- High precision sensors to gather high-quality data:
  - Low-voltage local digital temperature sensor (STTS22H)
  - Six-axis inertial measurement unit (LSM6DSV16X)
  - Three-axis low-power accelerometer (LIS2DU12)
  - 3-axis magnetometer (LIS2MDL)
  - Pressure sensor (LPS22DF)
  - Digital microphone/audio sensor (MP23DB01HP)
- User interface:
  - Hardware power switch
  - Green and orange system LED to display the power supply state
  - 4 programmable status LEDs (green, red, orange, blue)
  - 2 programmable push-buttons
  - Audio buzzer
  - Reset button
  - Qvar with electrodes for user interface experience
  - Interface J-Link/SWD debug-probe
  - Interface for extension board
  - Socket for DIL24 sensor adapters
- Power and charging options: USB Type-C® charging and connecting, 5 W wireless charging and rechargeable long-life 480 mAh battery

- Develop apps quickly regardless of your level of expertise:
  - Entry mode: wide range of default IoT and wearable applications
  - Expert mode: create your own app in a simple graphic environment
  - Pro mode: develop code in an intuitive way using STM32 Open Development Environment (ODE) and ST function pack libraries
- STBLESensor app on the smartphone (both on the Google Play and Apple Store) allows you to immediately connect to the box kit (the required PIN for pairing with the app is 123456)
- Windows, LINUX, and MacOS ST software compatibility
- Firmware over-the-air (FOTA) upgrade
- Certifications: CE, FCC, UKCA, IC

2.1.3 STEVAL-STWINBX1

The STWIN.box (STEVAL-STWINBX1) is a development kit and reference design that simplifies prototyping and testing of advanced industrial sensing applications in IoT contexts such as condition monitoring and predictive maintenance.

Figure 118. STEVAL-STWINBX1

It is an evolution of the original STWIN kit (STEVAL-STWINKT1B) and features a higher mechanical accuracy in the measurement of vibrations, an improved robustness, an updated bill of materials (BOM) to reflect the latest and best-in-class microcontroller unit (MCU) and industrial sensors, and an easy-to-use interface for external add-ons.

The STWIN.box kit consists of an STWIN.box core system, a 480mAh LiPo battery, an adapter for the ST-LINK debugger, a plastic case, an adapter board for DIL 24 sensors and a flexible cable.

The many on-board industrial-grade sensors and the ultra-low-power MCU enable applications that feature: Ultralow power, nine DoF motion sensing, wide-bandwidth vibration analysis, audio and ultrasound acoustic inspection, very precise local temperature, and environmental monitoring.

A rich set of software packages is available in source code. Optimized firmware libraries and a complete companion cloud application help to speed up the design cycle to develop end-to-end solutions.

The kit supports a broad range of connectivity options. For wired connectivity, it includes a USB Type-C® port that can be used for power supply, data transfer and STM32 programming via DFU, and an RS-485 transceiver. For wireless connectivity, the kit offers Bluetooth® LE, Wi-Fi, and NFC options.

The STWIN.box also includes a 34-pin expansion connector for small form-factor daughter boards associated with the STM32 family, such as the STEVAL-C34KAT1, STEVAL-C34KAT2, and STEVAL-PDETECT1 expansion boards.

The STWIN.box is suitable for field trials, demonstrations, and proof of concept for industrial IoT applications that use ST software and third-party software.

All features

- Multisensing wireless platform for vibration monitoring and ultrasound detection
- Built around STWIN.box core system board with processing, sensing, connectivity, and expansion capabilities
- Ultralow power Arm® Cortex®-M33 with FPU and TrustZone at 160 MHz, 2048 KB flash memory (STM32U585AI)
- microSD card slot for standalone data logging applications
- On-board Bluetooth® Low Energy v5.0 wireless technology (BlueNRG-M2), Wi-Fi (EMW3080), and NFC (ST25DV64K)
- Option to implement authentication and brand protection secure solution with STSAFE-A110
- Wide range of industrial IoT sensors:
  - Ultrawide bandwidth (up to 6 kHz), low-noise, 3-axis digital vibration sensor (IIS3DWB)
  - 3-axis accelerometer + 3-axis gyroscope iNEMO inertial measurement unit (ISM330DHCX) with machine learning core
  - High-performance ultralow-power 3-axis accelerometer for industrial applications (IIS2DLPC)
  - Ultralow power 3-axis magnetometer (IIS2MDC)
  - High-accuracy, high-resolution, low-power, 2-axis digital inclinometer with embedded machine learning core (IIS2ICLX)
  - Dual full-scale, 1.26 bar and 4 bar, absolute digital output barometer in full-mold package (ILPS22QS)
  - Low-voltage, ultra low-power, 0.5°C accuracy I²C/SMBus 3.0 temperature sensor (STTS22H)
  - Industrial grade digital MEMS microphone (IMP34DT05)
  - Analog MEMS microphone with frequency response up to 80 kHz (IMP23ABSU)
- Expandable via a 34-pin FPC connector

2.1.4 STEVAL-STWINKT1B

The STWIN SensorTile wireless industrial node (STEVAL-STWINKT1B) is a development kit and reference design that simplifies prototyping and testing of advanced industrial IoT applications such as condition monitoring and predictive maintenance.

Figure 119. **STEVAL-STWINKT1B**

The kit features a core system board with a range of embedded industrial-grade sensors and an ultralow-power microcontroller for vibration analysis of 9-DoF motion sensing data across a wide range of vibration frequencies, including very high frequency audio and ultrasound spectra, and high precision local temperature and environmental monitoring.

The development kit is complemented with a rich set of software packages and optimized firmware libraries, as well as a cloud dashboard application, all provided to help speed up design cycles for end-to-end solutions.

The kit supports Bluetooth® LE wireless connectivity through an on-board module, and Wi-Fi connectivity through a special plugin expansion board (STEVAL-STWINWFV1). Wired connectivity is also supported via an on-board RS-485 transceiver. The core system board also includes an STMod+ connector for compatible, low cost, small form factor daughter boards associated with the STM32 family, such as the LTE cell pack.

Apart from the core system board, the kit is provided complete with a 480 mAh LiPo battery, an STLINK-V3MINI debugger and a plastic box.

All features

- Multisensing wireless platform implementing vibration monitoring and ultrasound detection
- Updated version of STEVAL-STWINKT1, now including STSAFE-A110 populated, BLUENRG-M2SA module, and IMP23ABSU MEMS microphone
- Built around STWIN core system board with processing, sensing, connectivity, and expansion capabilities
- Ultralow-power Arm Cortex-M4 MCU at 120 MHz with FPU, 2048 KB flash memory (STM32L4R9)
- MicroSD card slot for standalone data logging applications
- On-board Bluetooth® low energy v5.0 wireless technology and Wi-Fi (with STEVAL-STWINWFV1 expansion board), and wired RS-485 and USB OTG connectivity
- Option to implement authentication and brand protection secure solution with STSAFE-A110

- Wide range of industrial IoT sensors:
  - ultrawide bandwidth (up to 6 kHz), low-noise, 3-axis digital vibration sensor (IIS3DWB)
  - 3D accelerometer + 3D Gyro iNEMO inertial measurement unit (ISM330DHCX) with machine learning core
  - ultralow-power high performance MEMS motion sensor (IIS2DH)
  - ultralow-power 3-axis magnetometer (IIS2MDC)
  - digital absolute pressure sensor (LPS22HH)
  - low-voltage digital local temperature sensor (STTS751)
  - industrial grade digital MEMS microphone (IMP34DT05)
  - analog MEMS microphone with frequency response up to 80 kHz (IMP23ABSU)
- Modular architecture, expandable via on-board connectors:
  - STMOD+ and 40-pin flex general purpose expansions
  - 12-pin male plug for connectivity expansions
  - 12-pin female plug for sensing expansions

2.1.5 STEVAL-MKI230KA adapter with DIL24

The STEVAL-MKI230KA evaluation kit consists of the STEVAL-MKI230A main sensing board, with a square PCB, which mounts the ISM330IS 3-axis accelerometer and 3-axis gyroscope with embedded ISPU, the STEVAL-MKIGIBV5 adapter board, and a flat cable. The main board is connected to the adapter board through the flat cable.

Figure 120. STEVAL-MKI230KA

The presence of the square PCB allows placing the sensor directly in the system where the measurement should be performed, which could be in a different position from the main board. The ISM330IS is soldered exactly in the center of the board and can be plugged into a standard DIL24 socket through the STEVAL-MKIGIBV5 adapter board.

The kit provides the complete ISM330IS pinout and comes ready to use with the required decoupling capacitors on the VDD power supply line.

This adapter is also supported by the STEVAL-MKI109V3 and STEVAL-MKI109D evaluation platforms that include a high-performance 32-bit microcontroller functioning as a bridge between the sensor and a PC, on which it is possible to use the downloadable MEMS Studio graphical user interface or dedicated software routines for customized applications.

All features

- User-friendly ISM330IS board
- Complete ISM330IS pinout for a standard DIL24 socket
- Fully compatible with the STEVAL-MKI109V3 and STEVAL-MKI109D evaluation platforms
- RoHS compliant

2.2 Hardware setup

The following hardware components are needed:

- For the STEVAL-MKBOXPRO
  - STEVAL-MKBOXPRO development board
  - USB - USB-C cable
  - MKIGIVB5 adapter with DIL24 and flat cable
  - MKI230AA adapter with ISM330IS sensor
- For the STEVAL-STWINBX1
  - STEVAL-STWINBX1 development board
  - USB - USB-C cable
  - Flat cable with ISM330IS sensor
- For the STEVAL-STWINKT1B
  - STEVAL-STWINKT1B development board
  - USB – USB-micro cable

2.3 Software setup

The following software components are required for the setup of a suitable development environment to create applications for the STEVAL-MKBOXPRO/STWINBX1/STWINKT1B boards, with the expansion board if needed:

- FP-SNS-STAIOTCFT: an STM32Cube function pack compliant with ST AIoT Craft web application. The firmware and related documentation are available on www.st.com.
- Development tool-chain and compiler. The STM32Cube expansion software supports the three following environments to select from:
  - IAR Embedded Workbench for Arm (EWARM) toolchain + ST-LINK
  - RealView Microcontroller Development Kit (MDK-ARM) toolchain + ST-LINK
  - System Workbench for STM32 + ST-LINK

2.4 Debug

The developer can download the relevant version of the ST-LINK/V2-1 USB driver by searching STSW-LINK008 or STSW-LINK009 on www.st.com (depending on your Windows version).

The ST-LINK/V2 is an in-circuit debugger and programmer for the STM8 and STM32 microcontrollers. The single-wire interface module (SWIM) and JTAG/serial wire debugging (SWD) interfaces are used to communicate with any STM8 or STM32 microcontroller located on an application board. In addition to providing the same functionalities as the ST-LINK/V2, the ST-LINK/V2-ISOL features digital isolation between the PC and the target application board. It also withstands voltages of up to 1000 Vrms.

Revision history

Table 2. **Document revision history**

Date	Version	Changes
03-Jun-2025	1	Initial release.
09-Dec-2025	2	Updated Section Introduction, Section 1.1: Overview, Figure 1. FP-SNS-STAIOTCFT software architecture, Section 1.3: Folder structure, Section 1.10: Executing commands and Section 1.11.1: Customizing inference applications. Added Section 1.13: Application AI SSM, Section 1.14: Installing the FP-SNS-STAIOTCFT pack in STM32CubeMX, Section 1.15: Starting a new project, Section 1.16: STM32 Configuration steps and Section 1.17: Application ai_inertial.

Contents

List of figures

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries (“ST”) reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice.

In the event of any conflict between the provisions of this document and the provisions of any contractual arrangement in force between the purchasers and ST, the provisions of such contractual arrangement shall prevail.

The purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgment.

The purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of the purchasers’ products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

If the purchasers identify an ST product that meets their functional and performance requirements but that is not designated for the purchasers' market segment, the purchasers shall contact ST for more information.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.